

[Draw your reader in with an engaging abstract. It is typically a short summary of the document. When you're ready to add your content, just click here and start typing.]

ALL MNC CODING

Collection questions

Praful Sharma

Contents

-----Accenture-----	1
-----Capgemini-----	54
-----Cognizant-----	108
-----Tech Mahindra-----	159
-----TCS-----	210
-----HCL-----	308

-----Accenture-----

1. Write a program to find the smallest number in a given array of integers. The first line of the input will be an integer indicating how many array elements are being input. If we let n denote this number, the following n lines of input will have one integer each.

(2022 Hirepro)

Sample Input:

3

2

10

-1

Output:

-1

Solutions :

C++

```
#include <iostream>
```

```
#include <limits.h>
```

```
using namespace std;
```

```
int main() {
```

```
    int n;
```

```

cin >> n;
int min_num = INT_MAX;
for (int i = 0; i < n; ++i) {
    int num;
    cin >> num;
    if (num < min_num) {
        min_num = num;
    }
}
cout << min_num << endl;
return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();
        int min_num = Integer.MAX_VALUE;
        for (int i = 0; i < n; ++i) {
            int num = scanner.nextInt();
            if (num < min_num) {
                min_num = num;
            }
        }
        System.out.println(min_num);
    }
}

```

Python

```

n = int(input())

```

```

min_num = float('inf')
for _ in range(n):
    num = int(input())
    if num < min_num:
        min_num = num
print(min_num)

```

2.

I. Edward's Birthday

Problem statement

It is Edward's birthday today. His friends have bought him a huge circular cake. Edward wants to find out the maximum number of pieces he can get by making exactly N straight vertical cuts on the cake. Your task is to write a function that returns the maximum number of pieces that can be obtained by making N number of cuts.

Note: Since the answer can be quite large, modulo it by 1000000007

Input Specification:

input1: An integer N denoting the number of cuts

Output Specification:

Return the maximum number of pieces that can be obtained by making N cuts on the cake

Input	Output	Explanation
1	2	Given the above scenario, if we make 1 cut on the cake, the maximum number of pieces of cake we obtain is 2 and $2\%1000000007=2$. Hence, 2 is returned as output.
5	16	Given the above scenario, if we make 5 cuts on the cake, the maximum number of pieces of cake we obtain is 16 and $16\%1000000007=16$. Hence, 16 is returned as output.

Solutions:

C++

```

#include <iostream>
using namespace std;

```



```

const int MOD = 1000000007;
int maxCakePieces(int N) {
    long long n = N;
    long long pieces = ((n * (n + 1)) / 2) + 1;
    return pieces % MOD;
}
int main() {
    int N;
    cin >> N;
    cout << maxCakePieces(N) << endl;
    return 0;}

```

Java

```

import java.util.Scanner;
public class Main {
    static final int MOD = 1000000007;
    public static int maxCakePieces(int N) {
        long n = N;
        long pieces = ((n * (n + 1)) / 2) + 1;
        return (int)(pieces % MOD);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        System.out.println(maxCakePieces(N));
    }
}

```

Python

```

def max_cake_pieces(N):
    MOD = 10**9 + 7
    pieces = (N * (N + 1)) // 2 + 1
    return pieces % MOD

# Input
N = int(input())

```

```
print(max_cake_pieces(N))
```

3. Regions on a Plane

Problem statement :

Mr. Professor is a great scientist, but he is not able to find a solution to one problem. There are N straight lines that are not parallel, and no three lines go through the same point. The lines divide the plane into M regions. Write a function to find out the maximum number of such regions he can get on the plane.

Input Specification:

input1: An integer N representing the number of straight lines ($0 \leq N \leq 100$)

Output Specification return the maximum number of region

Return the maximum number of regions

Example:

Input	Output	Explanation
2	4	Given the above scenario, 2 lines divide the plane into 4 regions. Therefore, 4 is returned as the output.
3	7	Given the above scenario, 3 lines divide the plane into 7 regions. Therefore, 7 is returned as the output.

Solutions

C++

```
#include <iostream>
using namespace std;
int maxRegions(int N) {
    return (N * (N + 1)) / 2 + 1;
}
int main() {
    int N;
    cin >> N;
    cout << maxRegions(N) << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;

public class Main {

    public static int maxRegions(int N) {

        return (N * (N + 1)) / 2 + 1;

    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt();

        System.out.println(maxRegions(N));}}
```

Python

```
def max_regions(N):

    return (N * (N + 1)) // 2 + 1

# Input

N = int(input())

print(max_regions(N))
```

4.

3. Sum of odd integers in array

Problem statement

An odd number is an integer which is not a multiple of 2.

You are required to implement the following function:

Int SumOddIntegers(int arr[], int n);

The function accepts an integer array 'arr' of length 'n' as its argument. You are required to calculate the sum of all odd integers in an array 'an' and return the same.

Note:

Array can have negative integers n > 0

Computed values lie within integer range

Example:

Input:

arr: 1 4 6 7 10 12 11 5

n: 8

Output:

24

Explanation:

The odd integers in array {1, 4, 6, 7, 10, 12, 11, 5} are {1, 7, 11, 5} and their sum is 24, hence 24 is returned.

The custom input format for the above case:

8

1 4 6 7 10 12 11 5

(The first line represents 'n' the second line represents the elements of the array)

Sample Input

arr: 2 4 9 7 11 13 25 31 6 8 10 24

n: 12

Sample Output

96

The custom input format for the above case:

12

2 4 9 7 11 13 25 31 6 8 10 24

Solutions

C++

```
#include <iostream>
using namespace std;
int SumOddIntegers(int arr[], int n) {
    int sum = 0;
    for(int i = 0; i < n; ++i) {
        if (arr[i] % 2 != 0) {
            sum += arr[i];
        }
    }
    return sum;
}
int main() {
    int n;
    cin >> n;
    int arr[n];
    for(int i = 0; i < n; ++i) {
        cin >> arr[i];
    }
    cout << SumOddIntegers(arr, n) << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;
public class SumOddIntegers {
    public static int sumOddIntegers(int[] arr, int n) {
        int sum = 0;
        for(int i = 0; i < n; i++) {
            if (arr[i] % 2 != 0) {
                sum += arr[i];
            }
        }
    }
}
```

```

    }
    return sum;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int[] arr = new int[n];
    for(int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
    }
    System.out.println(sumOddIntegers(arr, n));
    sc.close();
}
}

```

Python

```

def sum_odd_integers(arr, n):
    return sum(x for x in arr if x % 2 != 0)

# Read input
n = int(input())
arr = list(map(int, input().split()))

# Output result
print(sum_odd_integers(arr, n))

```

5.

4. Inversion count in array Problem statement

Let j and k be two indices in an array A .

If $j < k$ and $A[j] > A[k]$, then the pair (j,k) is known as an "Inversion pair".

You are required to implement the following function:

```
int InversionCount(int *A, int n);
```

The function accepts an array 'A' of 'n' unique integers as its argument. You are required to calculate the number of 'Inversion pair' in an array A, and return.

Note:

If 'A' is NULL (None in case of python), return -1

If 'n' < 2, return 0

Example:

Input:

A: 1 20 6 4 5

n: 5

Output:

5

Explanation:

The inversion pair in array A are (20,6),(20,4),(20,5),(6,4) and (6,5), the count of the inversions are 5, hence 5 is returned.

The custom input format for the above case:

5

1 20 6 4 5

(The first line represents the size of the array, the second line represents the elements of the array)

Sample Input

A: 13 10 9 6 21 15 14

n: 7

Sample Output

9

The custom input format for the above case: 7

13 10 9 6 21 15 14

Solutions

C++

```
#include <iostream>
```

```
using namespace std;
```

```
int merge(int arr[], int temp[], int left, int mid, int right) {
```

```
    int i = left, j = mid, k = left;
```

```
    int inv_count = 0;
```

```
    while ((i <= mid - 1) && (j <= right)) {
```

```
        if (arr[i] <= arr[j]) {
```

```
            temp[k++] = arr[i++];
```

```
        } else {
```

```
            temp[k++] = arr[j++];
```

```
            inv_count += (mid - i); // All remaining elements in left are greater
```

```
        }
```

```
    }
```

```
    while (i <= mid - 1)
```

```
        temp[k++] = arr[i++];
```

```

    while (j <= right)
        temp[k++] = arr[j++];
    for (int i = left; i <= right; i++)
        arr[i] = temp[i];
    return inv_count;
}

int mergeSort(int arr[], int temp[], int left, int right) {
    int mid, inv_count = 0;
    if (right > left) {
        mid = (right + left) / 2;
        inv_count += mergeSort(arr, temp, left, mid);
        inv_count += mergeSort(arr, temp, mid + 1, right);
        inv_count += merge(arr, temp, left, mid + 1, right);
    }
    return inv_count;
}

int InversionCount(int arr[], int n) {
    if (arr == nullptr) return -1;
    if (n < 2) return 0;
    int temp[n];
    return mergeSort(arr, temp, 0, n - 1);
}

int main() {
    int n;
    cin >> n;
    int arr[n];
    for(int i = 0; i < n; ++i)
        cin >> arr[i];
    cout << InversionCount(arr, n) << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class InversionCount {

    public static int mergeSort(int[] arr, int[] temp, int left, int right) {
        int mid, invCount = 0;
        if (right > left) {
            mid = (right + left) / 2;
            invCount += mergeSort(arr, temp, left, mid);
            invCount += mergeSort(arr, temp, mid + 1, right);
            invCount += merge(arr, temp, left, mid + 1, right);
        }
        return invCount;
    }

    public static int merge(int[] arr, int[] temp, int left, int mid, int right) {
        int i = left, j = mid, k = left, invCount = 0;
        while (i <= mid - 1 && j <= right) {
            if (arr[i] <= arr[j]) {
                temp[k++] = arr[i++];
            } else {
                temp[k++] = arr[j++];
                invCount += (mid - i);
            }
        }
        while (i <= mid - 1)
            temp[k++] = arr[i++];
        while (j <= right)
            temp[k++] = arr[j++];
        for (i = left; i <= right; i++)
            arr[i] = temp[i];
        return invCount;
    }

    public static int inversionCount(int[] arr, int n) {

```



```

    if (arr == null) return -1;
    if (n < 2) return 0;
    int[] temp = new int[n];
    return mergeSort(arr, temp, 0, n - 1);
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int[] arr = new int[n];
    for (int i = 0; i < n; i++)
        arr[i] = sc.nextInt();
    System.out.println(inversionCount(arr, n));
    sc.close();
}

```

Python

```

def merge(arr, temp, left, mid, right):
    i = left
    j = mid
    k = left
    inv_count = 0
    while i <= mid - 1 and j <= right:
        if arr[i] <= arr[j]:
            temp[k] = arr[i]
            i += 1
        else:
            temp[k] = arr[j]
            inv_count += (mid - i)
            j += 1
        k += 1
    while i <= mid - 1:
        temp[k] = arr[i]
        i += 1

```

```

        k += 1
    while j <= right:
        temp[k] = arr[j]
        j += 1
        k += 1
    for i in range(left, right + 1):
        arr[i] = temp[i]
    return inv_count

def merge_sort(arr, temp, left, right):
    inv_count = 0
    if right > left:
        mid = (left + right) // 2
        inv_count += merge_sort(arr, temp, left, mid)
        inv_count += merge_sort(arr, temp, mid + 1, right)
        inv_count += merge(arr, temp, left, mid + 1, right)
    return inv_count

def inversion_count(arr, n):
    if arr is None:
        return -1
    if n < 2:
        return 0
    temp = [0]*n
    return merge_sort(arr, temp, 0, n - 1)

n = int(input())
arr = list(map(int, input().split()))
print(inversion_count(arr, n))
6.

```

02. The Energy and Jumps

20 marks

Problem Statement

You want to defeat your enemy one last time. Your enemy is much stronger this time, and it will require exactly T units of energy to defeat him. Initially, you do not have any energy. There are exactly N cities from where you can collect the energy. Initially, you will enter the first city. Each city will give you some energy. The energy gains from the cities are in non-decreasing order. Every hour you can do one of the two following things:

- If you are in some city u , you can collect $E[u]$ units of energy.
- If you are in some city u and have at least $J[u]$ units of energy, you can spend $J[u]$ units of energy and jump to the city $u+1$.

Your task is to find out the minimum number of hours required to defeat your enemy.

Input Format:

The input consists of three lines:

- The first line contains two space-separated integers, N and T , denoting the number of cities and the energy required to defeat the enemy respectively.
- The second line contains N space-separated integers denoting the elements of array E .
- The third line contains $N-1$ space-separated integers denoting the elements of the array J .

The input will be read from the STDIN by the candidate

Output Format:

Print the minimum number of hours required to defeat your enemy.

The output will be matched to the candidate's output printed on the STDOUT

Constraints:

- $2 \leq N \leq 2 \cdot 10^3$
- $1 \leq T \leq 10^3$
- $1 \leq E[i] \leq 10^3$
- $1 \leq J[i] \leq 10^3$

Example:

Input:

2 13
2 13
2 7

Chat

5

Output:

6

Explanation:

In the first hour, we are in city 1, and we collected 2 units of energy. Now we have 2 units of energy.

In the second hour, we are on city 1 and collected 2 units of energy. Now we have 4 units of energy.

In the third hour, we are in city 1, and we collected 2 units of energy. Now we have 6 units of energy.

In the fourth hour, we are in city 1 and move to city 2 after earning 2 units of energy. Now we have 1 unit of energy after moving to the next city burning 5 units of energy.

In the fifth hour, we are in city 2, and we collected 7 units of energy. Now we have 8 units of energy.

In the sixth hour, we are in city 2, and we collected 7 units of energy. Now we have 15 units of energy.

It will take a minimum of 6 hours to defeat the enemy.

Sample input

3 7
5 5 6
2 1

Sample Output

2

Solutions:

C++

```
#include <iostream>
```

```
#include <vector>
```

```
#include <queue>
```

```
#include <set>
```

```
using namespace std;
```

```

struct State {
    int city, energy, hours;
};

int minHours(int n, int T, vector<int>& E, vector<int>& J) {
    queue<State> q;
    set<pair<int, int>> visited;
    q.push({0, 0, 0});
    while (!q.empty()) {
        auto [city, energy, hours] = q.front();
        q.pop();
        if (energy >= T)
            return hours;
        // Collect energy
        int new_energy = energy + E[city];
        if (visited.find({city, new_energy}) == visited.end()) {
            visited.insert({city, new_energy});
            q.push({city, new_energy, hours + 1});
        }
        // Jump
        if (city < n - 1 && energy >= J[city]) {
            int next_city = city + 1;
            int after_jump_energy = energy - J[city];
            if (visited.find({next_city, after_jump_energy}) == visited.end()) {
                visited.insert({next_city, after_jump_energy});
                q.push({next_city, after_jump_energy, hours + 1});
            }
        }
    }

    return -1;
}

int main() {

```

```

int n, T;
cin >> n >> T;
vector<int> E(n);
vector<int> J(n - 1);
for (int i = 0; i < n; ++i) cin >> E[i];
for (int i = 0; i < n - 1; ++i) cin >> J[i];
cout << minHours(n, T, E, J) << endl;
return 0;
}

```

Java

```

import java.util.*;

class Main {
    static class State {
        int city, energy, hours;
        State(int c, int e, int h) {
            city = c; energy = e; hours = h;
        }
    }

    public static int minHours(int n, int T, int[] E, int[] J) {
        Queue<State> q = new LinkedList<>();
        Set<String> visited = new HashSet<>();
        q.add(new State(0, 0, 0));
        while (!q.isEmpty()) {
            State s = q.poll();
            if (s.energy >= T)
                return s.hours;
            // Collect
            int newEnergy = s.energy + E[s.city];
            String collectKey = s.city + "_" + newEnergy;
            if (!visited.contains(collectKey)) {
                visited.add(collectKey);
                q.add(new State(s.city, newEnergy, s.hours + 1));
            }
        }
    }
}

```

```

    }

    // Jump
    if (s.city < n - 1 && s.energy >= J[s.city]) {
        int nextCity = s.city + 1;
        int jumpEnergy = s.energy - J[s.city];
        String jumpKey = nextCity + "_" + jumpEnergy;
        if (!visited.contains(jumpKey)) {
            visited.add(jumpKey);
            q.add(new State(nextCity, jumpEnergy, s.hours + 1));
        }
    }
}
return -1;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt(), T = sc.nextInt();
    int[] E = new int[n], J = new int[n - 1];
    for (int i = 0; i < n; i++) E[i] = sc.nextInt();
    for (int i = 0; i < n - 1; i++) J[i] = sc.nextInt();
    System.out.println(minHours(n, T, E, J));
}
}

```

Python

```

from collections import deque

def min_hours_to_defeat_enemy(n, T, E, J):
    visited = set()
    q = deque()
    q.append((0, 0, 0)) # (city, energy, hours)
    while q:
        city, energy, hours = q.popleft()

```

```

# If required energy is reached
if energy >= T:
    return hours

# Option 1: Collect energy in current city
new_energy = energy + E[city]
if (city, new_energy) not in visited:
    visited.add((city, new_energy))
    q.append((city, new_energy, hours + 1))

# Option 2: Jump to next city if energy is enough
if city < n - 1 and energy >= J[city]:
    next_city = city + 1
    jump_energy = energy - J[city]
    if (next_city, jump_energy) not in visited:
        visited.add((next_city, jump_energy))
        q.append((next_city, jump_energy, hours + 1))

return -1 # Not reachable

# Input Reading Section
if __name__ == "__main__":
    # Read N and T
    n, T = map(int, input().split())
    # Read energy values of N cities
    E = list(map(int, input().split()))
    # Read jump costs for N-1 cities
    J = list(map(int, input().split()))
    # Compute and print result
    result = min_hours_to_defeat_enemy(n, T, E, J)
    print(result)

```

7.

C++

```
#include <iostream>
#include <vector>
#include <map>
#include <algorithm>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<int> marks(n);
    map<int, int> freq;
    for (int i = 0; i < n; ++i) {
        cin >> marks[i];
        freq[marks[i]]++;
    }
    // Get unique marks in descending order
    vector<int> unique_marks;
```

02. **Problem Statement**

There are n students in a class. You are given the marks of all the students represented by an array `marks`. You want to maintain the equality in each section of the class. For that you have to remove the students having marks equal to the marks of two toppers.

For example if `marks = {1, 2, 3, 4, 3}`, here you have two toppers having marks 3 and 4. But two students got 3 marks, so you have to remove this student and send him to another section. Your task is to find the number of students you need to remove to maintain the equality.

Input Format

The input consists of two lines:

- The first line contains a single integer n .
- The second line contains n space-separated integers denoting the array `marks`.

The input will be read from the STDIN by the candidate

Output Format:

Print a number that represents the number of students to be removed to maintain the equality.

The output will be matched to the candidate's output printed on the STDOUT

Constraints:

- $2 \leq n \leq 10^5$
- $1 \leq \text{marks}[i] \leq 100$

Example:

Input:

```
5
90 80 90 80 70
```

Output:

```
2
```

Explanation:

We have two toppers having marks 80 and 90. But there are two more students having the same marks. So, the answer is 2.

Sample input

```
2
78 90
```

Sample Output

```
for (auto& p : freq) unique_marks.push_back(p.first);
```



```

sort(unique_marks.rbegin(), unique_marks.rend());
int remove_count = 0;
for (int i = 0; i < 2 && i < unique_marks.size(); ++i) {
    remove_count += freq[unique_marks[i]] - 1;
}
cout << remove_count << endl;
return 0;
}

```

Java

```

import java.util.*;

public class RemoveStudents {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] marks = new int[n];
        Map<Integer, Integer> freq = new HashMap<>();
        for (int i = 0; i < n; i++) {
            marks[i] = sc.nextInt();
            freq.put(marks[i], freq.getDefault(marks[i], 0) + 1);
        }
        // Sort keys in descending order
        List<Integer> uniqueMarks = new ArrayList<>(freq.keySet());
        Collections.sort(uniqueMarks, Collections.reverseOrder());
        int remove = 0;
        for (int i = 0; i < Math.min(2, uniqueMarks.size()); i++) {
            remove += freq.get(uniqueMarks.get(i)) - 1;
        }
        System.out.println(remove);
    }
}

```

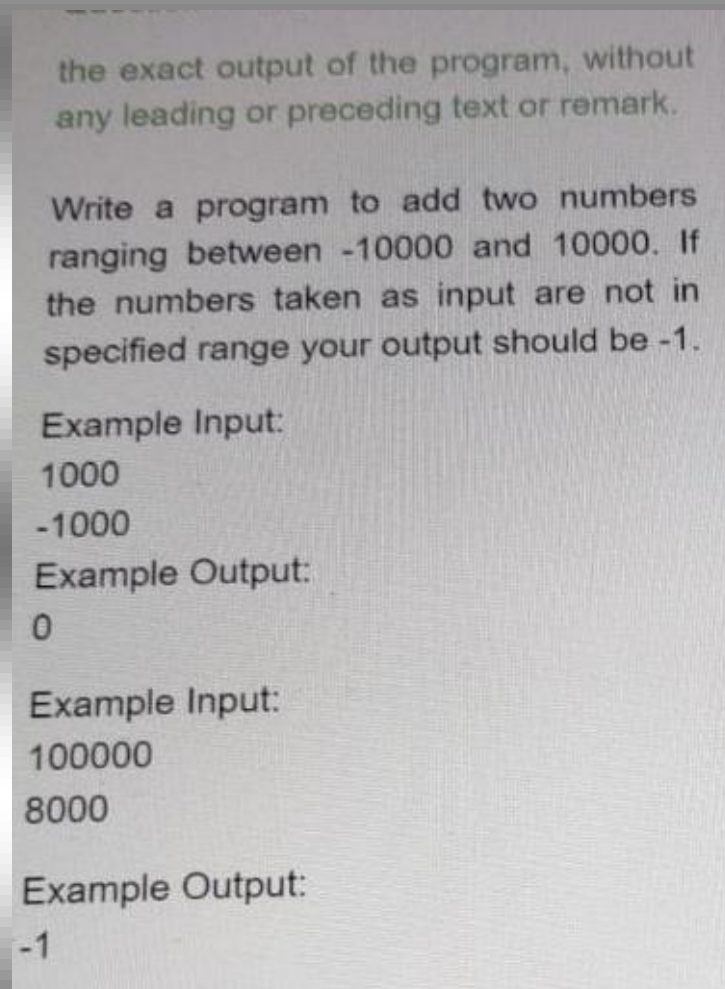
Python

```

from collections import Counter
def remove_students_for_toppers(n, marks):
    freq = Counter(marks)
    top_two = sorted(set(marks), reverse=True)[:2]
    total_remove = sum(freq[m] - 1 for m in top_two)
    return total_remove
# Input reading
if __name__ == "__main__":
    n = int(input())
    marks = list(map(int, input().split()))
    print(remove_students_for_toppers(n, marks))

```

8. (Hirepro -On Campus))



Solutions:

C++

```

#include <iostream>
using namespace std;
int main() {
    int a, b;
    cin >> a >> b;
    if (a >= -10000 && a <= 10000 && b >= -10000 && b <= 10000)
        cout << a + b << endl;
    else
        cout << -1 << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;
public class AddInRange {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();
        if (a >= -10000 && a <= 10000 && b >= -10000 && b <= 10000) {
            System.out.println(a + b);
        } else {
            System.out.println(-1);
        }
    }
}

```

Python

```

a = int(input())
b = int(input())
if -10000 <= a <= 10000 and -10000 <= b <= 10000:
    print(a + b)
else:
    print(-1)

```

9.

Problem Statement: Climbing Stairs

Alice climbs a staircase and takes N steps to reach the top. In each turn, Alice can climb either 1 step or M steps.

Your task is to determine the minimum number of climbs required to reach the top (i.e., reach the Nth stair exactly).

Input Format:

The input consists of a single line:

- Two space-separated integers N and M

The input will be read from STDIN by the candidate.

Output Format:

Print the minimum number of climbs required to reach the top.

The output will be matched to the candidate's output printed on the STDOUT

Constraints:

- $1 \leq N \leq 10^9$ $1 \leq N \leq 10^9$
- $1 \leq M \leq 10^9$ $1 \leq M \leq 10^9$

Testcase 1:

I/p : 5 1

O/p : 5

Explanation:

Alice can only climb 1 step at a time. So she needs 5 steps:

0 -> 1 -> 2 -> 3 -> 4 -> 5 (5 climbs)

Testcase 2:

I/p : 4 3

O/p : 2

Explanation:

Alice can do:

0 -> 1 -> 4 (1 climb of 1, then a climb of 3)

or

0 -> 3 -> 4 (1 climb of 3, then a climb of 1)

Minimum number of moves = 2

C++

```
#include <iostream>
```

```

#include <climits>
using namespace std;
int main() {
    long long n, m;
    cin >> n >> m;
    long long min_moves = LLONG_MAX;
    for (long long b = 0; b * m <= n; ++b) {
        long long a = n - b * m;
        long long total = a + b;
        min_moves = min(min_moves, total);
    }
    cout << min_moves << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;
public class ClimbingStairs {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long n = sc.nextLong();
        long m = sc.nextLong();
        long minMoves = Long.MAX_VALUE;
        for (long b = 0; b * m <= n; b++) {
            long a = n - b * m;
            long total = a + b;
            minMoves = Math.min(minMoves, total);
        }
        System.out.println(minMoves);
    }
}

```

Python

```

n, m = map(int, input().split())

```

```

min_moves = float('inf')
b = 0
while b * m <= n:
    a = n - b * m
    total = a + b
    min_moves = min(min_moves, total)
    b += 1
print(min_moves)

```

10.

Problem Statement: Minimum Cost String

You are given a string `str` consisting of lowercase Latin letters ('a'-'z'). Your task is to transform the string so that every character becomes the same vowel. The acceptable vowels are 'a', 'e', 'i', 'o', and 'u'.

There are two types of transformation costs:

- **Consonant to vowel:** Converting any consonant into any vowel costs **10 dollars**.
- **Vowel to vowel:** Converting one vowel into another costs **the absolute difference between their ASCII codes** (which in this challenge represents the "difference" of their positions as used in the sample explanation).

If the string is already composed of a single vowel repeated, the output should be `-1`. Otherwise, you need to print the minimum cost required to convert the string into one entirely made up of a single vowel.

For example, consider the input:

`greiay`

If we choose to convert every character to 'e', then the cost calculation is:

- For 'g': consonant → cost **10**
- For 'r': consonant → cost **10**
- For 'e': already 'e' → cost **0**
- For 'i': vowel → cost $| \text{'e'} - \text{'i'} | = | 101 - 105 | = 4$
- For 'a': vowel → cost $| \text{'e'} - \text{'a'} | = | 101 - 97 | = 4$
- For 'y': consonant → cost **10**

Total cost = $10 + 10 + 0 + 4 + 4 + 10 = 38$

Solutions

C++

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>
#include <cstdlib>
using namespace std;

int minCostToVowel(const string &str) {
    // Only lowercase vowels to consider
    vector<char> vowels = {'a', 'e', 'i', 'o', 'u'};
    int minCost = INT_MAX;
    // Try each vowel as a target transformation
    for (char target : vowels) {
        int cost = 0;
        for (char c : str) {
            // If the character is already the target vowel, no cost.
            if (c == target)
                continue;
            // If c is a vowel (but not target), add ASCII absolute difference.
            if (find(vowels.begin(), vowels.end(), c) != vowels.end())
                cost += abs(target - c);
            else // c is a consonant: fixed cost 10.
                cost += 10;
        }
        minCost = min(minCost, cost);
    }
    // If no transformation is necessary, return -1.
    return minCost == 0 ? -1 : minCost;
}

int main() {
    string str;
    cin >> str;

```

```

    cout << minCostToVowel(str) << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class MinimumCostString {
    public static int minCostToVowel(String str) {
        char[] vowels = {'a', 'e', 'i', 'o', 'u'};
        int minCost = Integer.MAX_VALUE;
        // Evaluate each vowel as the target
        for (char target : vowels) {
            int cost = 0;
            for (char c : str.toCharArray()) {
                // No cost if already the target vowel.
                if (c == target) continue;
                // Check if the character is a vowel.
                if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u') {
                    cost += Math.abs(target - c);
                } else { // Otherwise, it's a consonant.
                    cost += 10;
                }
            }
            minCost = Math.min(minCost, cost);
        }
        // If the minimal cost is zero then the string is already uniform with one vowel.
        return minCost == 0 ? -1 : minCost;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
    }
}

```



```

        String str = sc.next();
        System.out.println(minCostToVowel(str));
        sc.close();
    }
}

```

Python

```

def min_cost_to_vowel(s):
    vowels = 'aeiou'
    min_cost = float('inf')
    # Check cost for converting entire string to each vowel.
    for target in vowels:
        cost = 0
        for c in s:
            # No cost if already equal to the target vowel.
            if c == target:
                continue
            # If c is a vowel, cost is the absolute difference of ASCII.
            if c in vowels:
                cost += abs(ord(target) - ord(c))
            else:
                # c is a consonant—cost is fixed.
                cost += 10
        min_cost = min(min_cost, cost)
    # When no cost is involved (string is already uniform and a vowel), return -1.
    return -1 if min_cost == 0 else min_cost

if __name__ == "__main__":
    s = input().strip()
    print(min_cost_to_vowel(s))

```

11.

Problem statement Hide problem statement <<

You are given a function,
`int FindLargestPairSum(int* arr, int n);`

The function accepts an integer array 'arr' of length 'n'. Implement the function to find the largest pair sum in 'arr'. Here, you need to find two elements whose sum is the largest of all the other pairs sum and print the result.

Assumption: Array index starts from 0 and $n > 1$.

Example:
Input:
arr: 3 9 2 30 12 17
Output:
47
Explanation:
The largest pair sum will be $30 + 17$, i.e. 47.

Sample Input
arr: 20 14 7 32 2

Sample Output
52

Solutions

C++

```
#include <iostream>
#include <climits>
using namespace std;
int FindLargestPairSum(int* arr, int n) {
    int firstMax = INT_MIN, secondMax = INT_MIN;
    for (int i = 0; i < n; i++) {
        if (arr[i] > firstMax) {
            secondMax = firstMax;
            firstMax = arr[i];
        } else if (arr[i] > secondMax) {
            secondMax = arr[i];
        }
    }
}
```

```

    }
    return firstMax + secondMax;
}

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int* arr = new int[n];
    cout << "Enter the array elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    cout << "Largest Pair Sum: " << FindLargestPairSum(arr, n) << endl;
    delete[] arr;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {

    public static int FindLargestPairSum(int[] arr, int n) {
        int firstMax = Integer.MIN_VALUE;
        int secondMax = Integer.MIN_VALUE;
        for (int i = 0; i < n; i++) {
            if (arr[i] > firstMax) {
                secondMax = firstMax;
                firstMax = arr[i];
            } else if (arr[i] > secondMax) {
                secondMax = arr[i];
            }
        }

        return firstMax + secondMax;
    }
}

```

```

    }
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = scanner.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter array elements: ");
        for (int i = 0; i < n; i++) {
            arr[i] = scanner.nextInt();
        }
        System.out.println("Largest Pair Sum: " + FindLargestPairSum(arr, n));
        scanner.close();
    }
}

```

Python

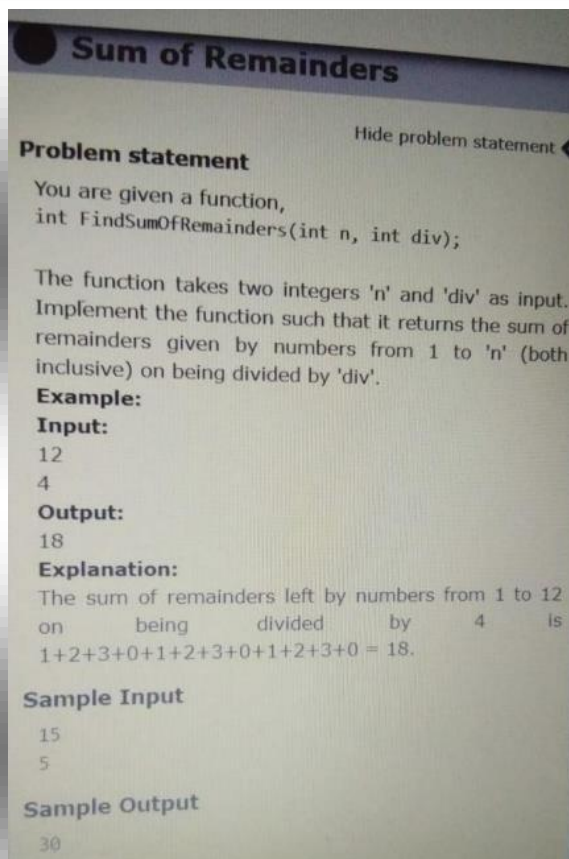
```

def FindLargestPairSum(arr):
    first_max = second_max = float('-inf')
    for num in arr:
        if num > first_max:
            second_max = first_max
            first_max = num
        elif num > second_max:
            second_max = num
    return first_max + second_max

# Taking input from the user
n = int(input("Enter number of elements: "))
arr = list(map(int, input("Enter the array elements: ").split()))
if len(arr) != n:
    print("Error: Number of elements does not match the input size.")
else:
    result = FindLargestPairSum(arr)
    print("Largest Pair Sum:", result)

```

12.



Solutions

C++

```
#include <iostream>
using namespace std;
// Function to calculate sum of remainders from 1 to n divided by div
int FindSumOfRemainders(int n, int div) {
    int sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i % div;
    }
    return sum;
}
int main() {
    int n, div;
    cout << "Enter the value of n: ";
    cin >> n;
```

```

cout << "Enter the divisor: ";
cin >> div;
if (div <= 0) {
    cout << "Divisor must be greater than 0." << endl;
    return 1;
}
int result = FindSumOfRemainders(n, div);
cout << "Output:\n" << result << endl;

return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {

    public static int FindSumOfRemainders(int n, int div) {
        int sum = 0;
        for (int i = 1; i <= n; i++) {
            sum += i % div;
        }
        return sum;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the value of n: ");
        int n = sc.nextInt();
        System.out.print("Enter the divisor: ");
        int div = sc.nextInt();
        if (div <= 0) {
            System.out.println("Divisor must be greater than 0.");
            return;
        }
    }
}

```

```
        int result = FindSumOfRemainders(n, div);
        System.out.println("Output:\n" + result);
        sc.close();
    }
}
```

Python

```
def FindSumOfRemainders(n, div):
    return sum(i % div for i in range(1, n + 1))

# User input
n = int(input("Enter the value of n: "))
div = int(input("Enter the divisor: "))

if div <= 0:
    print("Divisor must be greater than 0.")
else:
    result = FindSumOfRemainders(n, div)
    print("Output:")
    print(result)
```

Hide problem statement

Problem statement

Implement the following function:

```
int* NextSmallerNumber(int a[], int n);
```

The function accepts an integer array 'a' of size n. Replace each number of array with nearest smaller number on its right in the array.

Assumption: All integers are > 0.

Note:

- Return null if array is null.
- Null refers to None in case of Python.
- Replace the number with '-1', if no smaller number is present on its right.

Example:

Input:

a: 3 2 11 7 6 5 6 1

Output:

2 1 7 6 5 1 1 -1

Explanation:

Every number is replaced with the 1st smaller number on its right, ('3' -> '2', '2' -> '1', '11' -> '7', '7' -> '6', '6' -> '5', '5' -> '1', '6' -> '1' and '1' -> '-1').

Sample Input

a: 10 5 4 3 2 1

Sample Output

5 4 3 2 1 -1

Solutions

C++

```
#include <iostream>
```

```
using namespace std;
```

```
int* NextSmallerNumber(int a[], int m) {
```

```
    if (a == nullptr) return nullptr;
```

```
    int* result = new int[m];
```

```
    for (int i = 0; i < m; i++) {
```

```
        result[i] = -1;
```

```
        for (int j = i + 1; j < m; j++) {
```

```
            if (a[j] < a[i]) {
```

```
                result[i] = a[j];
```

```
                break;
```

```
            }
```

```
        }
```

```
    }
```

```
    return result;
```



```

}
int main() {
    int m;
    cout << "Enter size of array: ";
    cin >> m;
    int* a = new int[m];
    cout << "Enter array elements: ";
    for (int i = 0; i < m; i++) {
        cin >> a[i];
    }
    int* result = NextSmallerNumber(a, m);
    cout << "Output:\n";
    for (int i = 0; i < m; i++) {
        cout << result[i] << " ";
    }
    delete[] a;
    delete[] result;
    return 0;
}

```

Java

```

import java.util.Scanner;
public class Main {
    public static int[] NextSmallerNumber(int[] a, int m) {
        if (a == null) return null;
        int[] result = new int[m];
        for (int i = 0; i < m; i++) {
            result[i] = -1;
            for (int j = i + 1; j < m; j++) {
                if (a[j] < a[i]) {
                    result[i] = a[j];
                    break;
                }
            }
        }
    }
}

```

```

    }
}
return result;
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter size of array: ");
    int m = sc.nextInt();
    int[] a = new int[m];
    System.out.println("Enter array elements:");
    for (int i = 0; i < m; i++) {
        a[i] = sc.nextInt();
    }
    int[] result = NextSmallerNumber(a, m);
    System.out.println("Output:");
    for (int val : result) {
        System.out.print(val + " ");
    }
    sc.close();
}
}

```

Python

```

def NextSmallerNumber(a):
    if a is None:
        return None
    m = len(a)
    result = [-1] * m
    for i in range(m):
        for j in range(i + 1, m):
            if a[j] < a[i]:
                result[i] = a[j]
                break

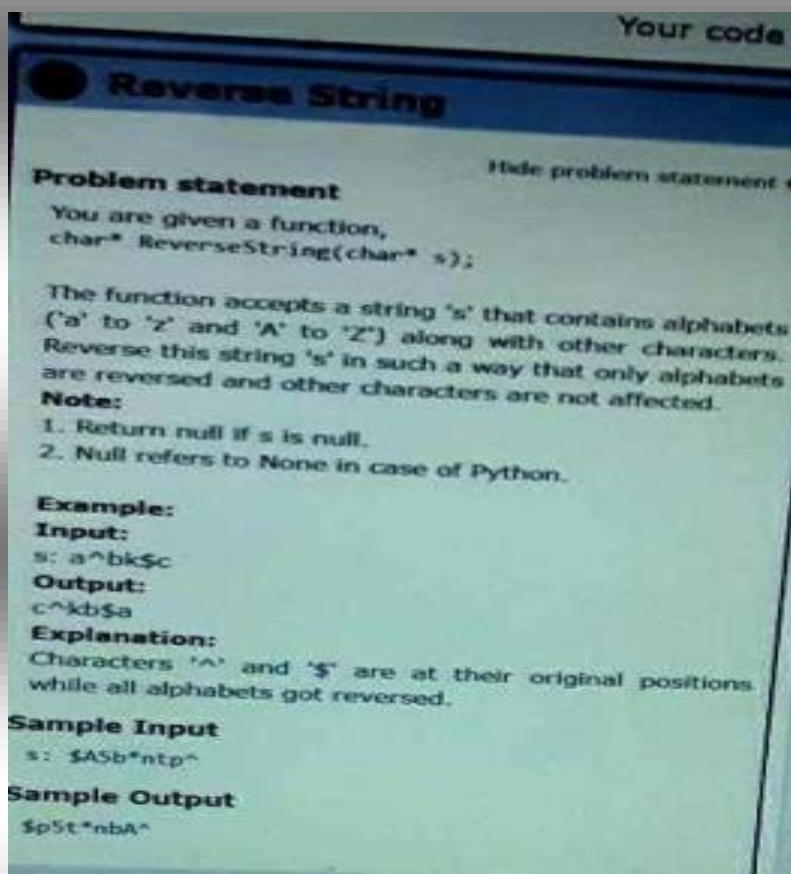
```

```

        return result
# User input
m = int(input("Enter size of array: "))
a = list(map(int, input("Enter array elements: ").split()))
if len(a) != m:
    print("Error: Size mismatch.")
else:
    result = NextSmallerNumber(a)
    print("Output:")
    print(" ".join(map(str, result)))

```

14.



Solution

C++

```

#include <iostream>
#include <cstring>

```

```

#include <cctype>
using namespace std;
char* ReverseString(char* s) {
    if (s == nullptr) return nullptr;
    int left = 0;
    int right = strlen(s) - 1;
    while (left < right) {
        if (!isalpha(s[left])) {
            left++;
        } else if (!isalpha(s[right])) {
            right--;
        } else {
            swap(s[left], s[right]);
            left++;
            right--;
        }
    }
    return s;
}

int main() {
    string input;
    cout << "Enter the string: ";
    getline(cin, input);

    char* str = new char[input.length() + 1];
    strcpy(str, input.c_str());
    char* result = ReverseString(str);
    cout << "Output:\n" << result << endl;
    delete[] str;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {

    public static String ReverseString(String s) {
        if (s == null) return null;
        char[] chars = s.toCharArray();
        int left = 0, right = chars.length - 1;
        while (left < right) {
            if (!Character.isLetter(chars[left])) {
                left++;
            } else if (!Character.isLetter(chars[right])) {
                right--;
            } else {
                char temp = chars[left];
                chars[left] = chars[right];
                chars[right] = temp;
                left++;
                right--;
            }
        }
        return new String(chars);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the string: ");
        String input = sc.nextLine();
        String result = ReverseString(input);
        System.out.println("Output:\n" + result);
        sc.close();
    }
}

```

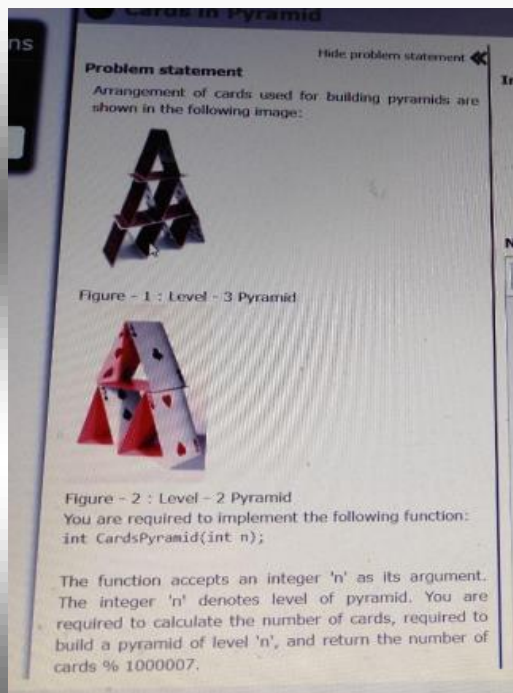
Python

```

def ReverseString(s):

```

if s is None:



User input

```
s = input("Enter the string: ")
```

```
result = ReverseString(s)
```

```
print("Output:\n" + result)
```

```
return None
```

```
s = list(s)
```

```
left, right = 0, len(s) - 1
```

```
while left < right:
```

```
    if not s[left].isalpha():
```

```
        left += 1
```

```
    elif not s[right].isalpha():
```

```
        right -= 1
```

```
    else:
```

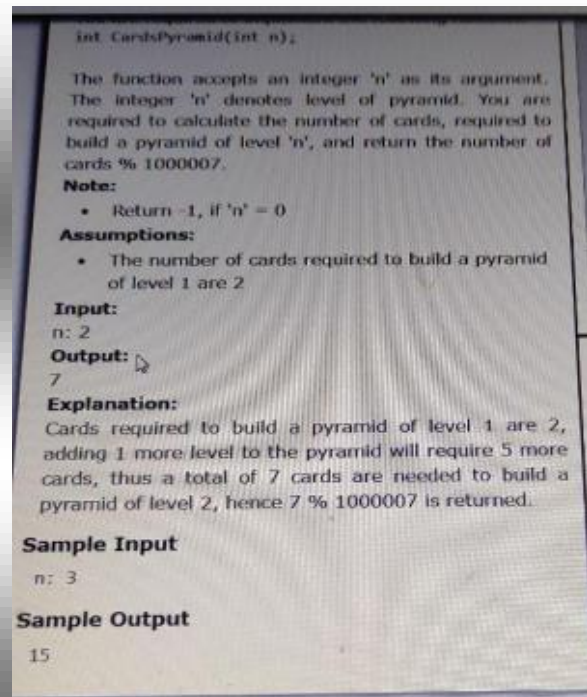
```
        s[left], s[right] = s[right], s[left]
```

```
        left += 1
```

```
        right -= 1
```

```
return "".join(s)
```

15.



Solutions:

C++

```
#include <iostream>
using namespace std;
int CardsPyramid(int n) {
    if (n == 0) return -1;
    const int MOD = 1000007;
    long long result = (1LL * n * (3 * n + 1)) / 2;
    return result % MOD;
}
int main() {
    int n;
    cout << "Enter the level of pyramid: ";
    cin >> n;

    int cards = CardsPyramid(n);
    cout << "Output:\n" << cards << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;

public class Main {
    public static int CardsPyramid(int n) {
        if (n == 0) return -1;
        final int MOD = 1000007;
        long result = ((long) n * (3 * n + 1)) / 2;
        return (int)(result % MOD);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the level of pyramid: ");
        int n = sc.nextInt();
        int cards = CardsPyramid(n);
        System.out.println("Output:\n" + cards);
        sc.close();
    }
}
```

Python

```
def CardsPyramid(n):
    if n == 0:
        return -1
    MOD = 1000007
    result = (n * (3 * n + 1)) // 2
    return result % MOD

# User input
n = int(input("Enter the level of pyramid: "))
print("Output:")
print(CardsPyramid(n))

16.
```


The function accepts a positive integer 'n' as its argument. A number 'n' can always be represented as a sum of squares of other integers. Implement the function to find the count of minimum number of squares that adds up to 'n' and return the count.

Example:

Input:

n: 13

Output:

2

Explanation:

Squares of integers are:
 $1^2 = 1$, $2^2 = 4$, $3^2 = 9$, $4^2 = 16$, and so on...

Possible combinations for $(n = 13)$ are:

n	Possible combinations for (n = 13) are:	Count
13	Sum of Squares 1 + 1 + 1 + 1 + 1 + 1 + 1	13
13	1 + 1 + 1 + 1 + 1 + 1 + 1	10
13	1 + 1 + 1 + 1 + 1 + 1 + 1	7
13	1 + 1 + 1 + 1 + 1 + 1 + 1	4
13	1 + 1 + 1 + 1 + 1 + 1 + 1	5
13	1 + 1 + 1 + 1 + 1 + 1 + 1	2
13	1 + 1 + 1 + 1 + 1 + 1 + 1	

is 2. Thus, output is 2.

Least count is 2. Thus, output is 2.

Sample Input

n = 6

Sample Output

Solutions

C++

```
#include <iostream>
```

```
#include <vector>
```

```
#include <cmath>
```

```
using namespace std;
```

```
int MinSquares(int n) {
```

```
vector<int> dp(n + 1, n); // Worst case: all 1's
```

```
dp[0] = 0;
```

```
for (int i = 1; i <= n; i++) {
```

```
for (int j = 1; j * j <= i; j++) {
```

```
dp[i] = min(dp[i], dp[i - j * j] + 1);
```

}

}

```
return dp[n];
```

}

```
int main() {
```

```
int n;
```

```

cout << "Enter n: ";
cin >> n;
cout << "Output:\n" << MinSquares(n) << endl;
return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {

    public static int MinSquares(int n) {
        int[] dp = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            dp[i] = i; // Worst case: all 1^2
            for (int j = 1; j * j <= i; j++) {
                dp[i] = Math.min(dp[i], dp[i - j * j] + 1);
            }
        }
        return dp[n];
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = sc.nextInt();
        System.out.println("Output:\n" + MinSquares(n));
        sc.close();
    }
}

```

Python

```

def MinSquares(n):
    dp = [0] + [n] * n # dp[0] = 0, rest initialized to max (n)
    for i in range(1, n + 1):
        for j in range(1, int(i**0.5) + 1):
            dp[i] = min(dp[i], dp[i - j*j] + 1)

```

```

        return dp[n]
# User input
n = int(input("Enter n: "))
print("Output:")
print(MinSquares(n))

```

17. Hirepro (2023)

Sum of Binary Digits

Problem Statement: You are given a number N. Convert the number to its binary form and return the sum of its binary digits (i.e., the number of 1's in the binary representation).

Example:

Input:

15 Binary of 15 is 1111, so the sum is 4.

Solutions

C++

```

#include <iostream>
using namespace std;
int SumOfBinaryDigits(int n) {
    int count = 0;
    while (n > 0) {
        count += (n & 1); // Add 1 if last bit is 1
        n >>= 1;          // Shift right
    }
    return count;
}
int main() {
    int n;
    cout << "Enter number: ";
    cin >> n;
    cout << "Output:\n" << SumOfBinaryDigits(n) << endl;
    return 0;
}

```

Java

```
import java.util.Scanner;

public class Main {
    public static int SumOfBinaryDigits(int n) {
        int count = 0;
        while (n > 0) {
            count += (n & 1); // Check last bit
            n >>= 1;         // Shift right
        }
        return count;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number: ");
        int n = sc.nextInt();
        System.out.println("Output:\n" + SumOfBinaryDigits(n));
        sc.close();
    }
}
```

Python

```
def SumOfBinaryDigits(n):
    return bin(n).count('1')

# User input
n = int(input("Enter number: "))
print("Output:")
print(SumOfBinaryDigits(n))
```

18.Hirpro (2023)

3) Shorten Word with Middle Character Count

Problem Statement:

Given a word, return a string that contains the first letter, the count of the middle characters, and the last letter.

Example:

Input: examination

Output: e9n

Solutions

C++

```
#include <iostream>
#include <string>
using namespace std;
string AbbreviateWord(const string& word) {
    int len = word.length();
    if (len <= 2) return word;
    return word[0] + to_string(len - 2) + word[len - 1];
}
int main() {
    string word;
    cout << "Enter word: ";
    cin >> word;
    cout << "Output:\n" << AbbreviateWord(word) << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;

public class Main {
    public static String AbbreviateWord(String word) {
        int len = word.length();
        if (len <= 2) return word;
        return word.charAt(0) + String.valueOf(len - 2) + word.charAt(len - 1);
    }
}
```

```

    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter word: ");
        String word = sc.next();
        System.out.println("Output:\n" + AbbreviateWord(word));
        sc.close();
    }
}

```

Python

```

def AbbreviateWord(word):
    if len(word) <= 2:
        return word
    return word[0] + str(len(word) - 2) + word[-1]

# User input
word = input("Enter word: ")
print("Output:")
print(AbbreviateWord(word))

```

19.

Balance fruits (Category: Medium)

Implement the following function:

```
int BalanceFruits(int a, int m, int rs);
```

You have a basket full of apples and mangoes, your job is to make the number of apples and given a function that accepts three integers 'a', 'm' and 'rs' as its argument where 'a' and 'm' are the number of apples and mangoes respectively and 'rs' is the rupees that you have. Implement the function to balance the basket.

- If 'a' > 'm', then buy (a - m) mangoes at the rate of Rs 1 per mango.
- If 'a' < 'm', then sell (m - a) mangoes at the rate of Rs 1 per mango.

Return the total rupees left with you after balancing the fruits.

Assumption:

- a >= 0, m >= 0 and rs >= 0
- rs >= (a - m)

Note: If a = m, return rs unchanged

Example**Sample Input**

a : 8

m : 4

Rs : 6

Sample Output

2

Explanation: Since $a > m$, $(a - m)$ mangoes are bought at Rs 1 per mango, so $rs = 6 - 4 = 2$. Thus, output is 2.

Case 1**Input**

8 4 6

Output

2

Case 2**Input**

7 4 6

Output

3

Solutions

C++

```
#include <iostream>
using namespace std;
int BalanceFruits(int a, int m, int rs) {
    if (a > m) rs -= (a - m);
    else if (m > a) rs += (m - a);
    return rs;
}
int main() {
    int a, m, rs;
    cout << "Enter apples, mangoes, rupees: ";
    cin >> a >> m >> rs;
    cout << "Output:\n" << BalanceFruits(a, m, rs) << endl;
    return 0;
}
```

Java

```

import java.util.Scanner;

public class Main {

    public static int BalanceFruits(int a, int m, int rs) {
        if (a > m) rs -= (a - m);
        else if (m > a) rs += (m - a);
        return rs;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter apples, mangoes, rupees: ");
        int a = sc.nextInt();
        int m = sc.nextInt();
        int rs = sc.nextInt();
        System.out.println("Output:\n" + BalanceFruits(a, m, rs));
        sc.close();
    }
}

```

Python

```

def BalanceFruits(a, m, rs):
    if a > m:
        rs -= (a - m)
    elif m > a:
        rs += (m - a)
    return rs

# User input
a = int(input("Enter number of apples: "))
m = int(input("Enter number of mangoes: "))
rs = int(input("Enter rupees you have: "))
print("Output:")
print(BalanceFruits(a, m, rs))
20.

```

Tallest Tree (Category: Medium)

A TreeHeight is represented by the following structure:

```
struct TreeHeight
{
    int feets;
    int inches;
};
```

You are given a function,

```
int TallestTree(struct TreeHeight trees[], int n);
```

The function accepts an array 'trees' of type 'TreeHeight' consisting of 'n' elements as its argument. 'TreeHeight' consists of two integers 'feets' and 'inches' which represents height of a tree. Implement the function to find the tallest tree among the 'trees' and return its height (in inches).

Height of a tree = (12 * feets) + inches

Assumption:

- feets > 0
- 0 >= inches >= 11

Note:

- Computed value lies within integer range.
- Return -1 if trees is null (None, in case of Python).
- trees(0) is the first element.

Example:

Sample Input:

Feets	Inches
10	4
23	5
21	2
27	7

Sample Output:

331

Explanation:

Height of trees (in inches):

- trees[0] = (10 * 12) + 4 = 124
- trees[1] = (23 * 12) + 5 = 281
- trees[2] = (21 * 12) + 2 = 254
- trees[3] = (27 * 12) + 7 = 331

Maximum of {124, 281, 254, 331} = 331, thus output is 331.

Solutions

C++

```
#include <iostream>
```

```
using namespace std;
```

```
struct TreeHeight {
```

```
    int feets;
```

```
    int inches;
```

```
};
```

```
int TallestTree(TreeHeight trees[], int n) {
```

```
    if (trees == NULL || n <= 0) return -1;
```

```
    int maxHeight = 0;
```

```
    for (int i = 0; i < n; ++i) {
```

```
        int heightInInches = trees[i].feets * 12 + trees[i].inches;
```

```
        if (heightInInches > maxHeight) {
```

```
            maxHeight = heightInInches;
```

```
        }
```

```

    }
    return maxHeight;
}
int main() {
    int n;
    cout << "Enter number of trees: ";
    cin >> n;
    TreeHeight* trees = new TreeHeight[n];
    for (int i = 0; i < n; i++) {
        cout << "Enter feets and inches for tree " << i + 1 << ": ";
        cin >> trees[i].feets >> trees[i].inches;
    }
    cout << "Tallest Tree Height (in inches):\n" << TallestTree(trees, n) << endl;
    delete[] trees;
    return 0;
}

```

Java

```

import java.util.Scanner;
class TreeHeight {
    int feets;
    int inches;
    TreeHeight(int f, int i) {
        this.feets = f;
        this.inches = i;
    }
}
public class Main {
    public static int TallestTree(TreeHeight[] trees, int n) {
        if (trees == null || n <= 0) return -1;
        int maxHeight = 0;
        for (int i = 0; i < n; i++) {

```

```

        int height = trees[i].feets * 12 + trees[i].inches;
        if (height > maxHeight) {
            maxHeight = height;
        }
    }
    return maxHeight;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter number of trees: ");
    int n = sc.nextInt();
    TreeHeight[] trees = new TreeHeight[n];
    for (int i = 0; i < n; i++) {
        System.out.print("Enter feets and inches for tree " + (i + 1) + ": ");
        int f = sc.nextInt();
        int in = sc.nextInt();
        trees[i] = new TreeHeight(f, in);
    }
    System.out.println("Tallest Tree Height (in inches):");
    System.out.println(TallestTree(trees, n));
    sc.close();
}
}

```

-----Capgemini-----

1. Reverse ASCII Sum (2024)

Given a lowercase English string, calculate its "Reverse ASCII Sum". The "Reverse ASCII Sum" is computed by assigning a reverse alphabetical value to each character

(where 'a' corresponds to 26, 'b' to 25, ..., 'z' to 1). Then, multiply this reverse alphabetical value by its 1-based position in the string and sum up all the results.

Input: A string consisting of lowercase English letters.

Output: An integer representing the "Reverse ASCII Sum".

Constraints:

- The input string will only contain lowercase English alphabets ('a' through 'z').
- The string length will be between 1 and 1000 characters (inclusive).

Test Case 1:

- Input: abc
- Expected Output: 148
- Explanation:

For 'a' (position 1): Reverse ASCII value of 'a' is 26. Contribution:
 $1 \times 26 = 26$

For 'b' (position 2): Reverse ASCII value of 'b' is 25. Contribution:
 $2 \times 25 = 50$

For 'c' (position 3): Reverse ASCII value of 'c' is 24. Contribution:
 $3 \times 24 = 72$

Total Sum: $26 + 50 + 72 = 148$

Test Case 2:

- Input: z
- Expected Output: 1
- Explanation:

For 'z' (position 1): Reverse ASCII value of 'z' is 1. Contribution: $1 \times 1 = 1$

Total Sum: 1

Solutions

C++

```
#include <iostream>
```

```

#include <string>
using namespace std;
int main() {
    string s;
    cin >> s;
    int sum = 0;
    for (int i = 0; i < s.length(); i++) {
        int reverseVal = 'z' - s[i] + 1;
        sum += (i + 1) * reverseVal;
    }
    cout << sum << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;
public class ReverseAsciiSum {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int sum = 0;
        for (int i = 0; i < s.length(); i++) {
            int reverseVal = 'z' - s.charAt(i) + 1;
            sum += (i + 1) * reverseVal;
        }
        System.out.println(sum);
        sc.close();
    }
}

```

Python

```

s = input()

```

```
total = 0
for i in range(len(s)):
    reverse_val = ord('z') - ord(s[i]) + 1
    total += (i + 1) * reverse_val
print(total)
```

2. Lexicographically Smallest String from Permutations (2024)

Given a string, rearrange its characters to form the lexicographically smallest possible string.

Input: A string consisting of lowercase English letters.

Output: A new string containing the same characters as the input string, but sorted alphabetically.

Constraints:

- The input string will only contain lowercase English alphabets ('a' through 'z').
- The string length will be between 1 and 1000 characters (inclusive).

Test Case 1:

- Input: banana
- Expected Output: aaabnn
- Explanation:

The characters in "banana" are: 'b', 'a', 'n', 'a', 'n', 'a'.

When sorted alphabetically: 'a', 'a', 'a', 'b', 'n', 'n'.

Combining these sorted characters gives "aaabnn".

Test Case 2:

- Input: apple
- Expected Output: aelpp
- Explanation:

The characters in "apple" are: 'a', 'p', 'p', 'l', 'e'.

When sorted alphabetically: 'a', 'e', 'l', 'p', 'p'.

Combining these sorted characters gives "aelpp".

Solutions

C++

```
#include <iostream>
#include <algorithm>
#include <string>
using namespace std;
int main() {
    string s;
    cin >> s;
    sort(s.begin(), s.end());
    cout << s << endl;
    return 0;
}
```

Java

```
import java.util.Arrays;
import java.util.Scanner;
public class LexicoSmallestString {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        char[] arr = s.toCharArray();
        Arrays.sort(arr);
        String result = new String(arr);
        System.out.println(result);
        sc.close();
    }
}
```

Python

```
s = input()
sorted_str = "".join(sorted(s))
print(sorted_str)
```

3. Segment Coverage Sum (2024)

You are given a list of N 2D integer segments, where each segment is represented by a pair (L,R) indicating the range $[L,R]$ (inclusive). You are also given a query range $[X,Y]$.

Your task is to calculate the total number of times points within the query range $[X,Y]$ are covered by the given segments. Specifically, for each integer point P from X to Y (inclusive), count how many of the N segments cover that point P . The final answer is the sum of these individual counts for all P in the range $[X,Y]$.

Input:

- An integer N , representing the number of segments.
- A list of N pairs (L_i, R_i) representing the segments.
- Two integers X and Y , representing the start and end of the query point range.

Output: An integer representing the total sum of segment coverages for points in $[X,Y]$.

Test Case

- **Input:**

$N=5$

Segments: $[(2,5), (1,4), (2,8), (3,6), (5,7)]$

Query Range: $X=1, Y=3$

- **Expected Output:** 8
- **Explanation:**

Point $P = 1$:

- $(1,4)$ covers 1. Count = 1.

Point $P = 2$:

- $(2,5)$ covers 2.
- $(1,4)$ covers 2.
- $(2,8)$ covers 2. Count = 3.

Point $P = 3$:

- $(2,5)$ covers 3.
- $(1,4)$ covers 3.
- $(2,8)$ covers 3.

- (3,6) covers 3. Count = 4.

Total Sum = Count(P=1) + Count(P=2) + Count(P=3) = 1+3+4=8.

Solutions

C++

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int N, X, Y;
    cin >> N;
    vector<pair<int, int>> segments(N);
    for (int i = 0; i < N; i++) {
        cin >> segments[i].first >> segments[i].second;
    }
    cin >> X >> Y;
    int total = 0;
    for (int p = X; p <= Y; p++) {
        for (auto seg : segments) {
            if (p >= seg.first && p <= seg.second)
                total++;
        }
    }
    cout << total << endl;
    return 0;
}
```

Java

```
import java.util.*;
public class SegmentCoverageSum {
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);
int N = sc.nextInt();
int[][] segments = new int[N][2];
for (int i = 0; i < N; i++) {
    segments[i][0] = sc.nextInt();
    segments[i][1] = sc.nextInt();
}
int X = sc.nextInt();
int Y = sc.nextInt();
int total = 0;
for (int p = X; p <= Y; p++) {
    for (int[] seg : segments) {
        if (p >= seg[0] && p <= seg[1])
            total++;
    }
}

System.out.println(total);
sc.close();
}

```

Python

```

N = int(input())
segments = [tuple(map(int, input().split())) for _ in range(N)]
X, Y = map(int, input().split())
total = 0
for p in range(X, Y + 1):
    for l, r in segments:
        if l <= p <= r:
            total += 1

```

```
print(total)
```

4.

John works as a technical expert for the mobile company where he analyses if mobiles are infected by viruses. a mobile is infected by virus if str is a special string . A string is called special string if all the characters at the alternative position starting from index 0 are same

write a program to print 1 if particular mobile contains virus else 0

Input Format:

The first line contains an integer N, representing the number of mobile strings.

The next N lines each contain a string.

Output Format:

N lines of output, each containing 1 or 0 for the corresponding input string.

Sample Input 1:

3

ADAOAS

MADAM

LFLRL

Sample Output 1:

1

0

1

Solutions

C++

```
#include <iostream>
```

```
using namespace std;
```

```
int isSpecialString(string s) {
```

```
    char expected = s[0];
```

```
    for (int i = 0; i < s.length(); i += 2) {
```

```
        if (s[i] != expected)
```

```

        return 0;
    }
    return 1;
}
int main() {
    int N;
    cin >> N;
    while (N--) {
        string s;
        cin >> s;
        cout << isSpecialString(s) << endl;
    }
    return 0;
}

```

Java

```

import java.util.Scanner;

public class VirusDetection {

    public static int isSpecialString(String s) {
        char expected = s.charAt(0);
        for (int i = 0; i < s.length(); i += 2) {
            if (s.charAt(i) != expected) return 0;
        }
        return 1;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        sc.nextLine(); // Consume newline
        for (int i = 0; i < N; i++) {
            String s = sc.nextLine();

```

```

        System.out.println(isSpecialString(s));
    }
    sc.close();
}
}

```

Python

```

def is_special_string(s):
    expected = s[0]
    for i in range(0, len(s), 2):
        if s[i] != expected:
            return 0
    return 1
N = int(input())
for _ in range(N):
    s = input().strip()
    print(is_special_string(s))

```

5. Calculate Initial Velocity (2024)

Given the final velocity (v), acceleration (a), and time interval (t) from the first equation of motion ($v=u+at$), calculate the initial velocity (u).

Input Format:

The input consists of a single line containing three integers: v , a , and t respectively, separated by spaces.

Output Format:

Print the initial velocity (u).

Test Case 1

Input: 1 1 1

Expected Output: 0

Explanation:

The equation is $v=u+at$.

We need to find u , so rearrange: $u=v-at$.

Given: $v=1$, $a=1$, $t=1$.

$u=1-(1\times 1)=1-1=0$.

Test Case 2:

Input: 10 2 3

Expected Output: 4

Solutions

C++

```
#include <iostream>
using namespace std;
int main() {
    int v, a, t;
    cin >> v >> a >> t;
    int u = v - a * t;
    cout << u << endl;
    return 0;}
```

Java

```
import java.util.Scanner;
public class InitialVelocity {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int v = sc.nextInt();
        int a = sc.nextInt();
        int t = sc.nextInt();
        int u = v - a * t;
        System.out.println(u);
        sc.close();
    }
}
```

```
}  
}
```

Python

```
v, a, t = map(int, input().split())  
u = v - a * t  
print(u)
```

6. Sum of Digits in an Interval

Given two integers A and B, write a program to find S, which is the sum of digits of all integers in the interval [A,B] (inclusive).

Input Format:

The input contains two integers, A and B, separated by a single white space, where A and B give the interval boundaries.

Output Format:

The output contains an integer, S, which is the sum of digits of all integers in the interval [A,B].

Constraints:

- $0 \leq A \leq B \leq 1018$

Test Case 1:

Input: 50 55

Expected Output: 45

Explanation:

Numbers in interval [50,55] are: 50, 51, 52, 53, 54, 55.

$\text{sum_digits}(50) = 5 + 0 = 5$

$\text{sum_digits}(51) = 5 + 1 = 6$

$\text{sum_digits}(52) = 5 + 2 = 7$

$\text{sum_digits}(53) = 5 + 3 = 8$

$\text{sum_digits}(54) = 5 + 4 = 9$

$\text{sum_digits}(55) = 5 + 5 = 10$

Total Sum $S = 5 + 6 + 7 + 8 + 9 + 10 = 45$.

Test Case 2:

Input: 28 31

Expected Output: 28

Solutions

C++

```
#include <iostream>
using namespace std;
int sumDigits(long long n) {
    int sum = 0;
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}
int main() {
    long long A, B;
    cin >> A >> B;
    long long total = 0;
    for (long long i = A; i <= B; ++i) {
        total += sumDigits(i);
    }
    cout << total << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;
public class SumOfDigitsInInterval {
    public static int sumDigits(long n) {
        int sum = 0;
        while (n > 0) {
```



```

        sum += (int)(n % 10);
        n /= 10;
    }
    return sum;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    long A = sc.nextLong();
    long B = sc.nextLong();
    long total = 0;
    for (long i = A; i <= B; i++) {
        total += sumDigits(i);
    }
    System.out.println(total);
    sc.close();
}
}

```

Python

```

def sum_digits(n):
    return sum(int(d) for d in str(n))

A, B = map(int, input().split())
total = 0
for num in range(A, B + 1):
    total += sum_digits(num)
print(total)

```

7. Array Rotation (2021)

Write a program with a function `rotate(arr[], d, n)` that rotates an array `arr[]` of size `n` by `d` elements. The rotation should be a *left rotation*, meaning the first `d` elements are moved to the end of the array.

Input Format: The input will be provided in three lines:

1. An integer n, representing the size of the array.
2. A space-separated list of n integers, representing the elements of the array arr[].
3. An integer d, representing the number of elements to rotate.

Output Format: The rotated array arr[], with elements separated by spaces.

Test Case 1:

- **Input:**

5
1 3 5 7 9
2

- **Expected Output:**

5 7 9 1 3

- **Explanation:**

Array arr = [1, 3, 5, 7, 9], n = 5, d = 2.

Left rotation by 2 means the first 2 elements (1 and 3) move to the end.

The remaining elements [5, 7, 9] come first.

Then, [1, 3] are appended.

Resulting array: [5, 7, 9, 1, 3].

Test Case 2:

- **Input:**

4
10 20 30 40
0

- **Expected Output:**

10 20 30 40

Solutions

C++

```
#include <iostream>
#include <vector>
using namespace std;
```

```

void rotate(vector<int>& arr, int d, int n) {
    d = d % n;
    vector<int> rotated;
    for (int i = d; i < n; ++i)
        rotated.push_back(arr[i]);
    for (int i = 0; i < d; ++i)
        rotated.push_back(arr[i]);
    for (int i = 0; i < n; ++i)
        cout << rotated[i] << " ";
    cout << endl;
}

```

```

int main() {
    int n, d;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; ++i)
        cin >> arr[i];
    cin >> d;
    rotate(arr, d, n);
    return 0;
}

```

Java

```

import java.util.*;

public class Main {
    public static void rotate(int[] arr, int d, int n) {
        d = d % n;
        int[] result = new int[n];
        int idx = 0;
        for (int i = d; i < n; i++)

```

```

        result[idx++] = arr[i];
    for (int i = 0; i < d; i++)
        result[idx++] = arr[i];
    for (int i = 0; i < n; i++)
        System.out.print(result[i] + " ");
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int[] arr = new int[n];
    for (int i = 0; i < n; i++)
        arr[i] = sc.nextInt();
    int d = sc.nextInt();
    rotate(arr, d, n);
}
}

```

Python

```

n = int(input())
arr = list(map(int, input().split()))
d = int(input())
# Normalize rotation count
d = d % n
rotated = arr[d:] + arr[:d]
print(" ".join(map(str, rotated)))

```

8.

You have write a function that accepts, a string which length is “len”, the string has some “#”, in it you have to move all the hashes to the front of the string and return the whole string back and print it.

Sample Test Case

Input:

Move#Hash#to#Front

Output:

###MoveHasToFront

Solutions

C++

```
#include <iostream>
#include <string>
using namespace std;
string moveHashToFront(string s) {
    int hashCount = 0;
    string result = "";
    for (char ch : s) {
        if (ch == '#') hashCount++;
        else result += ch;
    }
    return string(hashCount, '#') + result;
}
int main() {
    string input;
    getline(cin, input);
    cout << moveHashToFront(input) << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;
public class Main {
    public static String moveHashToFront(String s) {
        int hashCount = 0;
        StringBuilder result = new StringBuilder();
```

```

    for (char ch : s.toCharArray()) {
        if (ch == '#') hashCount++;
        else result.append(ch);
    }
    StringBuilder finalResult = new StringBuilder();
    for (int i = 0; i < hashCount; i++) finalResult.append('#');
    finalResult.append(result);
    return finalResult.toString();
}

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String input = sc.nextLine();
    System.out.println(moveHashToFront(input));
}
}

```

Python

```

def move_hash_to_front(s):
    hash_count = s.count('#')
    without_hash = s.replace('#', '')
    return '#' * hash_count + without_hash

# Read input
input_str = input()
print(move_hash_to_front(input_str))

```

9.

The students are given a string with multiple characters that are repeated consecutively. You're supposed to reduce the size of this string using mathematical logic given as in the example below :

Input :

aabbbbbeeefggg

Output:

a2b4e4f2g3

Input :

abbccccc

Output:

ab2c5

Solutions:

C++

```
#include <iostream>
#include <string>
using namespace std;
string compressString(string s) {
    if (s.empty()) return "";
    string result = "";
    int count = 1;
    for (size_t i = 1; i < s.length(); ++i) {
        if (s[i] == s[i-1]) {
            count++;
        } else {
            result += s[i-1];
            if (count > 1)
                result += to_string(count);
            count = 1;
        }
    }
    result += s.back();
    if (count > 1)
        result += to_string(count);
    return result;
}
```

```

}
int main() {
    string input;
    cin >> input;
    cout << compressString(input) << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {

    public static String compressString(String s) {
        if (s.isEmpty()) return "";
        StringBuilder result = new StringBuilder();
        int count = 1;
        for (int i = 1; i < s.length(); i++) {
            if (s.charAt(i) == s.charAt(i - 1)) {
                count++;
            } else {
                result.append(s.charAt(i - 1));
                if (count > 1) result.append(count);
                count = 1;
            }
        }
        result.append(s.charAt(s.length() - 1));
        if (count > 1) result.append(count);
        return result.toString();
    }

    public static void main(String[] args) {

```



```

        Scanner sc = new Scanner(System.in);
        String input = sc.nextLine();
        System.out.println(compressString(input));
    }
}

```

Python

```

def compress_string(s):
    if not s:
        return ""
    result = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i-1]:
            count += 1
        else:
            result.append(s[i-1])
            if count > 1:
                result.append(str(count))
            count = 1
    # Handle the last character(s)
    result.append(s[-1])
    if count > 1:
        result.append(str(count))
    return "".join(result)

# Input and Output
input_str = input()
print(compress_string(input_str))

```

10.

You're given an array of integers, print the number of times each integer has occurred in the array.

Example

Input :

10

1 2 3 3 4 1 4 5 1 2

Output :

1 occurs 3 times

2 occurs 2 times

3 occurs 2 times

4 occurs 2 times

5 occurs 1 times

Solutions

C++

```
#include <iostream>
#include <unordered_map>
using namespace std;
int main() {
    int n, x;
    cin >> n;
    unordered_map<int, int> freq;
    for (int i = 0; i < n; i++) {
        cin >> x;
        freq[x]++;
    }
    for (auto &p : freq)
        cout << p.first << " occurs " << p.second << " times\n";
    return 0;
}
```

Java

```
import java.util.*;
```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        Map<Integer, Integer> freq = new HashMap<>();
        for (int i = 0; i < n; i++) {
            int num = sc.nextInt();
            freq.put(num, freq.getDefault(num, 0) + 1);
        }
        for (Map.Entry<Integer, Integer> entry : freq.entrySet()) {
            System.out.println(entry.getKey() + " occurs " + entry.getValue() + " times");
        }
    }
}

```

Python

```

from collections import Counter
n = int(input())
arr = list(map(int, input().split()))
for num, count in Counter(arr).items():
    print(f"{num} occurs {count} times")

```

11.

A function is there which tells how many dealerships there are and the total number of cars in each dealership.

Your job is to calculate how many tyres would be there in each dealership.

Input

3

4 2

4 0

1 2

Output

20

16

8

There are total 3 dealerships

dealerships1 contains 4 cars and 2 bikes

dealerships2 contains 4 cars and 0 bikes

dealerships3 contains 1 cars and 2 bikes

Total number of tyres in dealerships1 is $(4 \times 4) + (2 \times 2) = 20$

Total number of tyres in dealerships2 is $(4 \times 4) + (0 \times 2) = 16$

Total number of tyres in dealerships3 is $(1 \times 4) + (2 \times 2) = 8$

Solutions

C++

```
#include <iostream>
using namespace std;
int main() {
    int n, cars, bikes;
    cin >> n;
    while (n-- > 0) {
        cin >> cars >> bikes;
        cout << cars * 4 + bikes * 2 << endl;
    }
    return 0;
}
```

Java

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
```

```

while (n-- > 0) {
    int cars = sc.nextInt();
    int bikes = sc.nextInt();
    System.out.println(cars * 4 + bikes * 2);
}
}
}

```

Python

```

n = int(input())
for _ in range(n):
    cars, bikes = map(int, input().split())
    print(cars * 4 + bikes * 2)

```

12. Check for Balanced Parentheses

Given a string *s* containing just the characters (,), {, }, [, and], determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Input Format: A single line containing the string *s*.

Output Format: Print true if the input string is valid, false otherwise.

Test Case 1:

Input: {{()}}

Expected Output: true

Explanation:

1. {: Push {. Stack: [{]}
2. {: Push {. Stack: [{, {}]}
3. (: Push (. Stack: [{, {, ()}]
4.): Match with top (. Pop. Stack: [{, {}]}

5. }: Match with top {. Pop. Stack: [{}]

6. }: Match with top {. Pop. Stack: [{}]

End of string. Stack is empty. Therefore, true.

Test Case 2:

Input: ()

Expected Output: true

Solutions

C++

```
#include <iostream>
#include <stack>
#include <unordered_map>
using namespace std;
bool isValid(string s) {
    stack<char> st;
    unordered_map<char, char> match = {{'(',')', {'[',']'}, {'{','}'}};
    for (char ch : s) {
        if (match.count(ch)) {
            if (st.empty() || st.top() != match[ch])
                return false;
            st.pop();
        } else {
            st.push(ch);
        }
    }
    return st.empty();
}

int main() {
    string s;
    cin >> s;
```

```
cout << (isValid(s) ? "true" : "false") << endl;
return 0;}
```

Java

```
import java.util.*;

public class Main {

    public static boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();
        Map<Character, Character> map = Map.of(')', '(', '}', '{', ']', '[');
        for (char ch : s.toCharArray()) {
            if (map.containsKey(ch)) {
                if (stack.isEmpty() || stack.peek() != map.get(ch))
                    return false;
                stack.pop();
            } else {
                stack.push(ch);
            }
        }
        return stack.isEmpty();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        System.out.println(isValid(s));
    }
}
```

Python

```
def is_valid_parentheses(s):
    stack = []
    mapping = {')': '(', '}': '{', ']': '['}
    for ch in s:
```

```

    if ch in mapping.values():
        stack.append(ch)
    elif ch in mapping:
        if not stack or stack[-1] != mapping[ch]:
            return False
        stack.pop()
    return not stack
# Input reading
s = input()
print(str(is_valid_parentheses(s)).lower())

```

13. Spiral Matrix Traversal

Write code to traverse a given matrix in a spiral format and print its elements in the order of traversal. The traversal should start from the top-left corner and proceed clockwise.

Input Format:

1. The first line contains two integers, rows and cols, representing the number of rows and columns of the matrix, respectively.
2. The subsequent rows lines each contain cols space-separated integers, representing the elements of the matrix.

Output Format: Print the elements of the matrix in a single line, separated by spaces, in spiral order.

Test Case 1:

- **Input:**

```

5 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20

```

- **Expected Output:**

```

1 2 3 4 8 12 16 20 19 18 17 13 9 5 6 7 11 15 14 10

```


- **Explanation:**

Right: 1, 2, 3, 4

Down: 8, 12, 16, 20

Left: 19, 18, 17

Up: 13, 9, 5

Right (Inner): 6, 7

Down (Inner): 11, 15

Left (Inner): 14

Up (Inner): 10

Combined: 1 2 3 4 8 12 16 20 19 18 17 13 9 5 6 7 11 15 14 10

Test Case 2:

Input:

3 3

1 2 3

4 5 6

7 8 9

Expected Output:

1 2 3 6 9 8 7 4 5

Solutions

C++

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int rows, cols;
```

```
    cin >> rows >> cols;
```

```
    int a[100][100];
```

```
    for (int i = 0; i < rows; ++i)
```

```
        for (int j = 0; j < cols; ++j)
```

```
            cin >> a[i][j];
```

```

int top = 0, bottom = rows - 1, left = 0, right = cols - 1;
while (top <= bottom && left <= right) {
    for (int i = left; i <= right; ++i) cout << a[top][i] << " ";
    top++;
    for (int i = top; i <= bottom; ++i) cout << a[i][right] << " ";
    right--;
    if (top <= bottom) {
        for (int i = right; i >= left; --i) cout << a[bottom][i] << " ";
        bottom--;
    }
    if (left <= right) {
        for (int i = bottom; i >= top; --i) cout << a[i][left] << " ";
        left++;
    }
}
return 0;
}

```

Java

```

import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int rows = sc.nextInt(), cols = sc.nextInt();
        int[][] a = new int[rows][cols];

        for (int i = 0; i < rows; ++i)
            for (int j = 0; j < cols; ++j)
                a[i][j] = sc.nextInt();

        int top = 0, bottom = rows - 1, left = 0, right = cols - 1;
    }
}

```

```

while (top <= bottom && left <= right) {
    for (int i = left; i <= right; ++i) System.out.print(a[top][i] + " ");
    top++;
    for (int i = top; i <= bottom; ++i) System.out.print(a[i][right] + " ");
    right--;
    if (top <= bottom) {
        for (int i = right; i >= left; --i) System.out.print(a[bottom][i] + " ");
        bottom--;
    }
    if (left <= right) {
        for (int i = bottom; i >= top; --i) System.out.print(a[i][left] + " ");
        left++;
    }
}
}
}

```

Python

```

rows, cols = map(int, input().split())
matrix = [list(map(int, input().split())) for _ in range(rows)]
top, bottom, left, right = 0, rows - 1, 0, cols - 1
res = []
while top <= bottom and left <= right:
    for i in range(left, right + 1): res.append(matrix[top][i])
    top += 1
    for i in range(top, bottom + 1): res.append(matrix[i][right])
    right -= 1
    if top <= bottom:
        for i in range(right, left - 1, -1): res.append(matrix[bottom][i])
        bottom -= 1
    if left <= right:

```

```
        for i in range(bottom, top - 1, -1): res.append(matrix[i][left])
        left += 1
print(*res)
```

14. Find Factors of a Number

Bela teaches her daughter to find the factors of a given number. When she provides a number to her daughter, she should tell the factors of that number. Help her to do this by writing a program. You need to write a class FindFactor.java and write the main method in it.

Note:

- If the input provided is negative, ignore the sign and provide the output (i.e., find factors of its absolute value).
- If the input is zero, the output should be "No Factors".

Input Format: A single integer representing the number.

Output Format: Print the factors of the number, separated by commas and spaces.

Test Case 1:

- Input: 54
- Expected Output: 1, 2, 3, 6, 9, 18, 27, 54
- Explanation:

The factors of 54 are the numbers that divide 54 evenly.

$54 \div 1 = 54$

$54 \div 2 = 27$

$54 \div 3 = 18$

$54 \div 6 = 9$

The pairs are (1, 54), (2, 27), (3, 18), (6, 9).

Sorted, these are: 1, 2, 3, 6, 9, 18, 27, 54.

Test Case 2:

Input: 0

Expected Output: No Factors

Solutions

C++

```
#include <iostream>
```

```

#include <set>
#include <cmath>
using namespace std;
int main() {
    int num;
    cin >> num;
    if (num == 0) {
        cout << "No Factors";
        return 0;
    }
    num = abs(num);
    set<int> factors;
    for (int i = 1; i * i <= num; i++) {
        if (num % i == 0) {
            factors.insert(i);
            factors.insert(num / i);
        }
    }
    bool first = true;
    for (int f : factors) {
        if (!first) cout << ", ";
        cout << f;
        first = false;
    }
    return 0;
}

```

Java

```

import java.util.*;
public class FindFactor {
    public static void main(String[] args) {

```

```

Scanner sc = new Scanner(System.in);
int num = sc.nextInt();
if (num == 0) {
    System.out.println("No Factors");
    return;
}
num = Math.abs(num);
TreeSet<Integer> factors = new TreeSet<>();
for (int i = 1; i * i <= num; i++) {
    if (num % i == 0) {
        factors.add(i);
        factors.add(num / i);
    }
}
StringJoiner sj = new StringJoiner(", ");
for (int f : factors)
    sj.add(String.valueOf(f));
System.out.println(sj.toString());
}
}

```

Python

```

num = int(input())
if num == 0:
    print("No Factors")
else:
    num = abs(num)
    factors = set()
    for i in range(1, int(num**0.5) + 1):
        if num % i == 0:
            factors.add(i)

```

```
factors.add(num // i)

print(' '.join(map(str, sorted(factors))))
```

15. Minimum Discount Item

Mayuri buys "N" number of products from a shop. The shop offers a different percentage of discount on each item. She wants to know the item that has the minimum discount offer (in absolute monetary value), so that she can avoid buying that and save money.

Input Format:

1. The first line refers to the number of items, N.
2. The subsequent N lines each contain the item name, its original price, and the discount percentage, separated by commas (e.g., item_name,price,discount_percentage).

Output Format: Print the name of the item that has the minimum discount offer.

Note: If there can be more than one product with a minimum discount, print the name of the first such product encountered in the input.

Test Case 1:

- **Input:**

```
4
mobile,10000,20
shoe,5000,10
watch,6000,15
laptop,35000,5
```

- **Expected Output:**

```
shoe
```

- **Explanation:**

mobile: Discount = $(10000 \times 20) / 100 = 2000$

shoe: Discount = $(5000 \times 10) / 100 = 500$

watch: Discount = $(6000 \times 15) / 100 = 900$

laptop: Discount = $(35000 \times 5) / 100 = 1750$

Comparing the calculated discounts: 2000,500,900,1750.

The minimum discount is 500, which corresponds to the item shoe.

Test Case 2:

- **Input:**

3

itemA,100,0

itemB,200,0

itemC,300,0

- **Expected Output:**

itemA

Solutions

C++

```
#include <iostream>
#include <climits>
using namespace std;
int main() {
    int n;
    cin >> n;
    cin.ignore();
    string item, minItem;
    int minDisc = INT_MAX;
    while (n--) {
        string line;
        getline(cin, line);
        size_t p1 = line.find(','), p2 = line.rfind(',');
        item = line.substr(0, p1);
        int price = stoi(line.substr(p1 + 1, p2 - p1 - 1));
        int disc = stoi(line.substr(p2 + 1));
        int discount = price * disc / 100;
        if (discount < minDisc) {
            minDisc = discount;
            minItem = item;
        }
    }
}
```



```

    }
}
cout << minItem;
return 0;
}
Java
import java.util.*;
public class MinimumDiscountItem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine());
        int minDiscount = Integer.MAX_VALUE;
        String minItem = "";
        for (int i = 0; i < n; i++) {
            String[] parts = sc.nextLine().split(",");
            String item = parts[0];
            int price = Integer.parseInt(parts[1]);
            int discount = Integer.parseInt(parts[2]);
            int discAmount = price * discount / 100;
            if (discAmount < minDiscount) {
                minDiscount = discAmount;
                minItem = item;
            }
        }
        System.out.println(minItem);
    }
}

```

Python

```

n = int(input())
min_discount = float('inf')
min_item = ""

```

```

for _ in range(n):
    item, price, disc = input().split(',')
    discount = int(price) * int(disc) // 100
    if discount < min_discount:
        min_discount = discount
        min_item = item
print(min_item)

```

16. Pythagorean Triplets

Generate all Pythagorean triplets (a,b,c) with values smaller than a given limit. A Pythagorean triplet satisfies the condition $a^2+b^2=c^2$, where a, b, and c are positive integers.

Input Format: The input consists of a single line: limit = [integer_value].

Output Format: Print each Pythagorean triplet on a new line, with the values separated by spaces (e.g., 3 4 5). The order of triplets and the order of a and b within a triplet (e.g., 3 4 5 vs 4 3 5) is typically determined by sorting rules (e.g., sort by 'a', then 'b', then 'c', and ensure $a < b$). The sample output shows $a < b$.

Test Case 1

- Input: limit = 20
- Expected Output:

3 4 5

5 12 13

8 15 17

12 16 20

- Explanation:

For each triplet (a,b,c), all three values must be less than 20.

$3^2 + 4^2 = 9 + 16 = 25 = 5^2$. a=3,b=4,c=5. All are <20.

$5^2 + 12^2 = 25 + 144 = 169 = 13^2$. a=5, b=12, c=13. All are < 20.

$8^2 + 15^2 = 64 + 225 = 289 = 17^2$. a=8,b=15,c=17. All are <20.

$12^2 + 16^2 = 144 + 256 = 400 = 20^2$. a=12, b=16, c=20.

Test Case 2:

- Input: limit = 10

- Expected Output:

3 4 5

6 8 10

Solutions

C++

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    string s;
    getline(cin, s);
    int limit = stoi(s.substr(s.find('=') + 1));
    for (int a = 1; a < limit; a++)
        for (int b = a + 1; b < limit; b++) {
            int c2 = a*a + b*b, c = sqrt(c2);
            if (c*c == c2 && c <= limit)
                cout << a << " " << b << " " << c << "\n";
        }
    return 0;
}
```

Java

```
import java.util.*;
public class PythagoreanTriplets {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int limit = Integer.parseInt(sc.nextLine().split("=")[1].trim());
        for (int a = 1; a < limit; a++)
            for (int b = a + 1; b < limit; b++) {
                int c2 = a*a + b*b, c = (int)Math.sqrt(c2);
```

```

        if (c*c == c2 && c <= limit)
            System.out.println(a + " " + b + " " + c);
    }
}

```

Python

```

import re

limit = int(re.findall(r'\d+', input())[0])

for a in range(1, limit):
    for b in range(a+1, limit):
        c = (a**2 + b**2)**0.5
        if c.is_integer() and c <= limit:
            print(f"{a} {b} {int(c)}")

```

17. Reverse a Linked List

Write a function to reverse a singly linked list.

Input Format: The input will be a representation of a singly linked list, typically shown as a sequence of numbers connected by arrows .

Output Format: The output should be the reversed linked list, represented in the same format

Test Case 1:

- Input: 1 -> 2 -> 3 -> 4 -> 5
- Expected Output: 5 -> 4 -> 3 -> 2 -> 1
- Explanation: The original list is traversed, and each node's next pointer is flipped to point backward, resulting in the head becoming the tail and vice-versa.

Test Case 2:

- Input: (Empty list, or null head node)
- Expected Output: (Empty list, or null head node)

Solutions

C++

```

#include <iostream>
using namespace std;
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* reverseList(ListNode* head) {
    ListNode *prev = nullptr, *next = nullptr;
    while (head) {
        next = head->next;
        head->next = prev;
        prev = head;
        head = next;
    }
    return prev;
}

void printList(ListNode* head) {
    while (head) {
        cout << head->val;
        if (head->next) cout << " -> ";
        head = head->next;
    }
    cout << endl;
}

int main() {
    int n, val;
    cout << "Enter number of nodes: ";
    cin >> n;

```

```

if (n == 0) {
    cout << "List is empty.\n";
    return 0;
}
cout << "Enter " << n << " elements:\n";
cin >> val;
ListNode* head = new ListNode(val);
ListNode* current = head;
for (int i = 1; i < n; ++i) {
    cin >> val;
    current->next = new ListNode(val);
    current = current->next;
}
cout << "Original List: ";
printList(head);
head = reverseList(head);
cout << "Reversed List: ";
printList(head);

return 0;}

```

Java

```

import java.util.Scanner;
class ListNode {
    int val;
    ListNode next;
    ListNode(int val) {
        this.val = val;
        this.next = null;
    }
}

```

```

public class ReverseLinkedList {
    public static ListNode reverseList(ListNode head) {
        ListNode prev = null, next = null;
        while (head != null) {
            next = head.next;
            head.next = prev;
            prev = head;
            head = next;
        }
        return prev;
    }

    public static void printList(ListNode head) {
        while (head != null) {
            System.out.print(head.val);
            if (head.next != null) System.out.print(" -> ");
            head = head.next;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();
        if (n == 0) {
            System.out.println("List is empty.");
            return;
        }
        System.out.println("Enter " + n + " elements:");
        int val = sc.nextInt();
        ListNode head = new ListNode(val);
    }
}

```

```

        ListNode current = head;
        for (int i = 1; i < n; i++) {
            val = sc.nextInt();
            current.next = new ListNode(val);
            current = current.next;
        }
        System.out.print("Original List: ");
        printList(head);

        head = reverseList(head);
        System.out.print("Reversed List: ");
        printList(head);
    }
}

```

Python

```

class ListNode:
    def __init__(self, val):
        self.val = val
        self.next = None

def reverse_list(head):
    prev = None
    while head:
        next_node = head.next
        head.next = prev
        prev = head
        head = next_node
    return prev

def print_list(head):
    while head:
        print(head.val, end=" -> " if head.next else "\n")

```



```

        head = head.next
if __name__ == "__main__":
    n = int(input("Enter number of nodes: "))
    if n == 0:
        print("List is empty.")
    else:
        print(f"Enter {n} elements:")
        values = list(map(int, input().split()))
        head = ListNode(values[0])
        current = head
        for val in values[1:]:
            current.next = ListNode(val)
            current = current.next
        print("Original List:", end=" ")
        print_list(head)
        head = reverse_list(head)
        print("Reversed List:", end=" ")
        print_list(head)

```

18. Longest Substring Without Repeating Characters:

- Problem: Given a string, find the length of the longest substring without repeating characters.
- Input:
 - String: "abcabcbb"
- Output: 3
- Explanation: The longest substring is "abc", which has a length of 3.

Solutions

C++

```

#include <iostream>
#include <unordered_set>
#include <string>

```

```

using namespace std;
int lengthOfLongestSubstring(string s) {
    unordered_set<char> seen;
    int left = 0, maxLen = 0;
    for (int right = 0; right < s.length(); right++) {
        while (seen.count(s[right])) {
            seen.erase(s[left]);
            left++;
        }
        seen.insert(s[right]);
        maxLen = max(maxLen, right - left + 1);
    }
    return maxLen;
}

int main() {
    string s;
    cout << "Enter a string: ";
    cin >> s;
    cout << "Output: " << lengthOfLongestSubstring(s) << endl;
    return 0;
}

```

Java

```

import java.util.*;
public class LongestUniqueSubstring {
    public static int lengthOfLongestSubstring(String s) {
        Set<Character> seen = new HashSet<>();
        int left = 0, maxLen = 0;
        for (int right = 0; right < s.length(); right++) {
            while (seen.contains(s.charAt(right))) {

```

```

        seen.remove(s.charAt(left));
        left++;
    }
    seen.add(s.charAt(right));
    maxLen = Math.max(maxLen, right - left + 1);
}
return maxLen;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter a string: ");
    String input = sc.nextLine();
    System.out.println("Output: " + lengthOfLongestSubstring(input));
}

```

Python

```

def length_of_longest_substring(s):
    seen = set()
    left = 0
    max_len = 0
    for right in range(len(s)):
        while s[right] in seen:
            seen.remove(s[left])
            left += 1
        seen.add(s[right])
        max_len = max(max_len, right - left + 1)
    return max_len

# Input
s = input("Enter a string: ")
print("Output:", length_of_longest_substring(s))

```

19. Matrix Rotation by 90 Degrees:

Problem: Rotate a 2D matrix by 90 degrees clockwise.

Input:

Matrix: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

Output:

[[7, 4, 1], [8, 5, 2], [9, 6, 3]]

Explanation: The matrix is rotated 90 degrees clockwise.

Solutions

C++

```
#include <iostream>
#include <vector>
#include <algorithm> // Needed for reverse
using namespace std;

void rotate(vector<vector<int>>& matrix) {
    int n = matrix.size();
    // Transpose the matrix
    for(int i = 0; i < n; ++i)
        for(int j = i + 1; j < n; ++j)
            swap(matrix[i][j], matrix[j][i]);
    // Reverse each row
    for(auto& row : matrix)
        reverse(row.begin(), row.end());
}

int main() {
    int n;
    cout << "Enter matrix size (n x n): ";
    cin >> n;
    vector<vector<int>> mat(n, vector<int>(n));
    cout << "Enter matrix elements:\n";
```

```

for(int i = 0; i < n; ++i)
    for(int j = 0; j < n; ++j)
        cin >> mat[i][j];
rotate(mat);
cout << "Rotated matrix:\n";
for(auto& row : mat) {
    for(int val : row) cout << val << " ";
    cout << "\n";
}
return 0;
}

```

Java

```

import java.util.*;

public class RotateMatrix {
    public static void rotate(int[][] mat) {
        int n = mat.length;
        // Transpose
        for (int i = 0; i < n; ++i)
            for (int j = i + 1; j < n; ++j) {
                int tmp = mat[i][j];
                mat[i][j] = mat[j][i];
                mat[j][i] = tmp;
            }
        // Reverse rows
        for (int[] row : mat)
            for (int i = 0, j = n - 1; i < j; ++i, --j) {
                int tmp = row[i];
                row[i] = row[j];
                row[j] = tmp;
            }
    }
}

```

```

    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter matrix size (n x n): ");
        int n = sc.nextInt();
        int[][] mat = new int[n][n];
        System.out.println("Enter matrix elements:");
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                mat[i][j] = sc.nextInt();
        rotate(mat);
        System.out.println("Rotated matrix:");
        for (int[] row : mat) {
            for (int val : row) System.out.print(val + " ");
            System.out.println();
        }
    }
}

```

Python

```

def rotate(matrix):
    # Transpose
    matrix[:] = [list(row) for row in zip(*matrix)]

    # Reverse rows
    for row in matrix:
        row.reverse()

# Input
n = int(input("Enter matrix size (n x n): "))
matrix = []
print("Enter matrix elements row by row:")
for _ in range(n):

```

```

row = list(map(int, input().split()))
matrix.append(row)
rotate(matrix)
print("Rotated matrix:")
for row in matrix:
    print(*row)

```

20. Largest Sum Contiguous Subarray (Kadane's Algorithm):

Problem: Find the maximum sum of a contiguous subarray.

Input:

- Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

Output:

- 6

Explanation: The subarray [4, -1, 2, 1] has the largest sum = 6.

Solutions

C++

```

#include <iostream>
#include <vector>
#include <climits>
using namespace std;
int main() {
    int n; cin >> n;
    vector<int> arr(n);
    for(int i = 0; i < n; ++i) cin >> arr[i];
    int max_sum = arr[0], curr_sum = arr[0];
    for(int i = 1; i < n; ++i) {
        curr_sum = max(arr[i], curr_sum + arr[i]);
        max_sum = max(max_sum, curr_sum);
    }
    cout << max_sum << endl;
}

```

```

        return 0;
    }
Java
import java.util.*;
public class Kadane {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i = 0; i < n; i++) arr[i] = sc.nextInt();
        int maxSum = arr[0], currSum = arr[0];
        for(int i = 1; i < n; i++) {
            currSum = Math.max(arr[i], currSum + arr[i]);
            maxSum = Math.max(maxSum, currSum);
        }
        System.out.println(maxSum);
    }
}

```

Python

```

arr = list(map(int, input().split()))
max_sum = curr_sum = arr[0]
for num in arr[1:]:
    curr_sum = max(num, curr_sum + num)
    max_sum = max(max_sum, curr_sum)
print(max_sum)

```


-----Cognizant-----

1. Conditional Subarrays

You have a sorted box of numbers stored in an integer array A, and you want to find all the "mini boxes" (subarrays) that meet a special rule. This rule says that the sum of the smallest and biggest number in each mini box must be less than or equal to a given number X. Since the original box is already sorted, the smallest number is always at the beginning, and the biggest one is at the end of each mini box you create. Your task is to find and return an integer value representing the number of mini boxes that can be made while following the rule.

Input Specification:

- input1: An integer N, representing the size of the array.
- input2: An integer X, representing the comparison value.
- input3: An integer array A, representing the sorted box of numbers.

Output Specification:

Return an integer value representing the number of mini boxes that can be made which follow the rule.

Example:

Input:

input1: 4

input2: 5

input3: {1,3,5,8}

Output: 2

Solutions

C++

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int countMiniBoxes(int N, int X, vector<int>& A) {
    int left = 0, right = N - 1, count = 0;
    while (left <= right) {
        if (A[left] + A[right] <= X) {
            count += (right - left + 1);
            left++;
        } else {
            right--;
        }
    }
    return count;
}

int main() {
    int input1, input2;
    cout << "input1: ";
    cin >> input1;
    cout << "input2: ";
    cin >> input2;
    vector<int> input3(input1);
    cout << "input3: ";
    for (int i = 0; i < input1; i++) {
        cin >> input3[i];
    }
    sort(input3.begin(), input3.end()); // Optional if already sorted
    cout << countMiniBoxes(input1, input2, input3) << endl;
    return 0;
}

```

```
}
```

Java

```
import java.util.*;
```

```
public class MiniBoxCounter {
```

```
    public static int countMiniBoxes(int N, int X, int[] A) {
```

```
        int left = 0, right = N - 1, count = 0;
```

```
        while (left <= right) {
```

```
            if (A[left] + A[right] <= X) {
```

```
                count += (right - left + 1);
```

```
                left++;
```

```
            } else {
```

```
                right--;
```

```
            }
```

```
        }
```

```
        return count;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("input1: ");
```

```
        int input1 = sc.nextInt();
```

```
        System.out.print("input2: ");
```

```
        int input2 = sc.nextInt();
```

```
        int[] input3 = new int[input1];
```

```
        System.out.print("input3: ");
```

```
        for (int i = 0; i < input1; i++) {
```

```
            input3[i] = sc.nextInt();
```

```
        }
```

```
        Arrays.sort(input3); // Optional if already sorted
```

```
        System.out.println(countMiniBoxes(input1, input2, input3));
```

```
    }}
```

Python

```
def count_mini_boxes(N, X, A):
    left = 0
    right = N - 1
    count = 0
    while left <= right:
        if A[left] + A[right] <= X:
            count += (right - left + 1)
            left += 1
        else:
            right -= 1
    return count

# Matching the input format
input1 = int(input("input1: ")) # N
input2 = int(input("input2: ")) # X
input3 = list(map(int, input("input3: ").strip().split())) # A
# Optional: sort input if not guaranteed to be sorted
input3.sort()
# Output the result
print(count_mini_boxes(input1, input2, input3))
```

2. Palindromic String Count

Jack broke a toy which consisted of a string S. The toy was broken in such a way that some letters were still present, and some were lost. Jack decided to replace the missing letters with '?'. He can generate palindromic string by replacing '?' with any letter from 'a' to 'z'. Jack has a favorite number M. Your task is to find and return an integer value representing the total number of palindromic strings modulo M that he can generate by replacing '?' with any letter from 'a' to 'z'.

Note: String S contains lowercase alphabets only.

Input Specification:

- input1 : An integer value M
- input2 : A string S representing a word with missing characters.

Output Specification:

Return an integer value representing the total number of palindromic words Jack can form after performing modulo with a given integer M.

Example 1:

- input1 : 3
- input2 : ca?d

Output: 0

Solutions

C++

```
#include <iostream>
#include <string>
using namespace std;
int countPalindromicStrings(int M, string S) {
    int n = S.size();
    long long result = 1;

    for (int i = 0; i < n / 2; i++) {
        char a = S[i];
        char b = S[n - 1 - i];
        if (a == '?' && b == '?') {
            result = (result * 26) % M;
        } else if (a == '?' || b == '?') {
            continue;
        } else if (a != b) {
            return 0;
        }
    }

    if (n % 2 == 1 && S[n / 2] == '?') {
        result = (result * 26) % M;
    }
}
```

```

        return result;
    }

    int main() {
        int input1;
        string input2;
        cout << "input1: ";
        cin >> input1;
        cout << "input2: ";
        cin >> input2;
        cout << countPalindromicStrings(input1, input2) << endl;
        return 0;
    }

```

Java

```

import java.util.*;

public class PalindromeCount {

    public static int countPalindromicStrings(int M, String S) {
        int n = S.length();
        long result = 1;
        for (int i = 0; i < n / 2; i++) {
            char a = S.charAt(i);
            char b = S.charAt(n - 1 - i);
            if (a == '?' && b == '?') {
                result = (result * 26) % M;
            } else if (a == '?' || b == '?') {
                continue;
            } else if (a != b) {
                return 0;
            }
        }
        if (n % 2 == 1 && S.charAt(n / 2) == '?') {

```

```

        result = (result * 26) % M;
    }
    return (int) result;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("input1: ");
    int input1 = sc.nextInt();
    sc.nextLine(); // clear newline
    System.out.print("input2: ");
    String input2 = sc.nextLine();
    System.out.println(countPalindromicStrings(input1, input2));}

```

Python

```

def count_palindromic_strings(M, S):
    n = len(S)
    result = 1
    for i in range(n // 2):
        a = S[i]
        b = S[n - 1 - i]
        if a == '?' and b == '?':
            result = (result * 26) % M
        elif a == '?' or b == '?':
            continue # only 1 valid choice, so no change in result
        elif a != b:
            return 0
    if n % 2 == 1 and S[n // 2] == '?':
        result = (result * 26) % M
    return result

```

```
# Matching input format
input1 = int(input("input1: "))
input2 = input("input2: ")
print(count_palindromic_strings(input1, input2))
```

3.

Maria is a math's teacher. She has an array A containing N integers, starting from 1 up till N. She gives her students a problem, wherein they are required to create an array B where each element is sum of adjacent elements of array A i.e., $B[i] = A[i] + A[i+1]$. The last element of B should be sum of last and the first element.

She also gives an integer X; the students are required to return the sum of only those elements of array B which are completely divisible by X.

Example 1:

- input1 : 10
- input2 : 2

Output: 0

Explanation:

The value N is 10, array A = {1,2,3,4,5,6,7,8,9,10} and X = 2. Array B will be {3,5,7,9,11,13,15,17,19,11}

There are no elements of array B which are divisible by 2, hence 0 is returned as the output.

Solutions

C++

```
#include <iostream>
#include <vector>
using namespace std;
int specialSum(int N, int X) {
    vector<int> A(N);
    for (int i = 0; i < N; i++) {
        A[i] = i + 1;
    }
    vector<int> B;
```



```

    for (int i = 0; i < N - 1; i++) {
        B.push_back(A[i] + A[i + 1]);
    }
    B.push_back(A[N - 1] + A[0]); // last + first
    int total = 0;
    for (int val : B) {
        if (val % X == 0) {
            total += val;
        }
    }

    return total;
}

int main() {
    int input1, input2;
    cout << "input1: ";
    cin >> input1;
    cout << "input2: ";
    cin >> input2;
    cout << specialSum(input1, input2) << endl;
    return 0;
}

```

Java

```

import java.util.*;

public class SpecialSum {
    public static int specialSum(int N, int X) {
        int[] A = new int[N];
        for (int i = 0; i < N; i++) {
            A[i] = i + 1;
        }
    }
}

```

```

int[] B = new int[N];
for (int i = 0; i < N - 1; i++) {
    B[i] = A[i] + A[i + 1];
}
B[N - 1] = A[N - 1] + A[0]; // last + first
int total = 0;
for (int val : B) {
    if (val % X == 0) {
        total += val;
    }
}
return total;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("input1: ");
    int input1 = sc.nextInt();
    System.out.print("input2: ");
    int input2 = sc.nextInt();
    System.out.println(specialSum(input1, input2));
}
}

```

Python

```

def special_sum(N, X):
    A = list(range(1, N + 1))
    B = []
    for i in range(N - 1):
        B.append(A[i] + A[i + 1])
    B.append(A[-1] + A[0]) # last + first

```

```
total = sum(b for b in B if b % X == 0)
```

```
return total
```

```
# Input format
```

```
input1 = int(input("input1: "))
```

```
input2 = int(input("input2: "))
```

```
print(special_sum(input1, input2))
```

4.

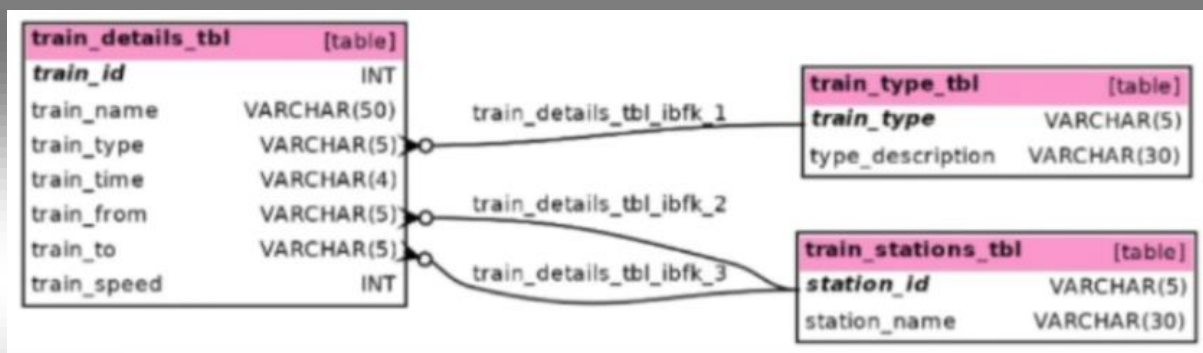
Write an SQL query to find:

The train ID and name of all trains that have a name starting with the alphabet 'M' and that go to the station with name "PUNE".

Your output should contain 2 columns in the below-mentioned order.

train_id	train_name
----------	------------

Schema:



SQL Query:

```
-- Selects the train_id and train_name columns.
SELECT
    td.train_id,
    td.train_name
FROM
    -- Specifies the main table, train_details_tbl, and assigns it an alias 'td'.
    train_details_tbl AS td
-- Joins train_details_tbl with train_stations_tbl using the train_to and station_id columns.
-- This links trains to their destination stations.
JOIN
    train_stations_tbl AS ts ON td.train_to = ts.station_id
WHERE
    -- Filters the results:
    -- 1. train_name starts with 'M' (case-sensitive, depending on database collation).
    -- 2. station_name is exactly 'PUNE'.
    td.train_name LIKE 'M%' AND ts.station_name = 'PUNE';
```

5. Schema:

Qn:

The employee ID, Type of leave, and Total number of leaves for employees who have taken more than 10 leaves, where the leave type is either 'Casual Leave'(CL) or 'Medical Leave'(ML).

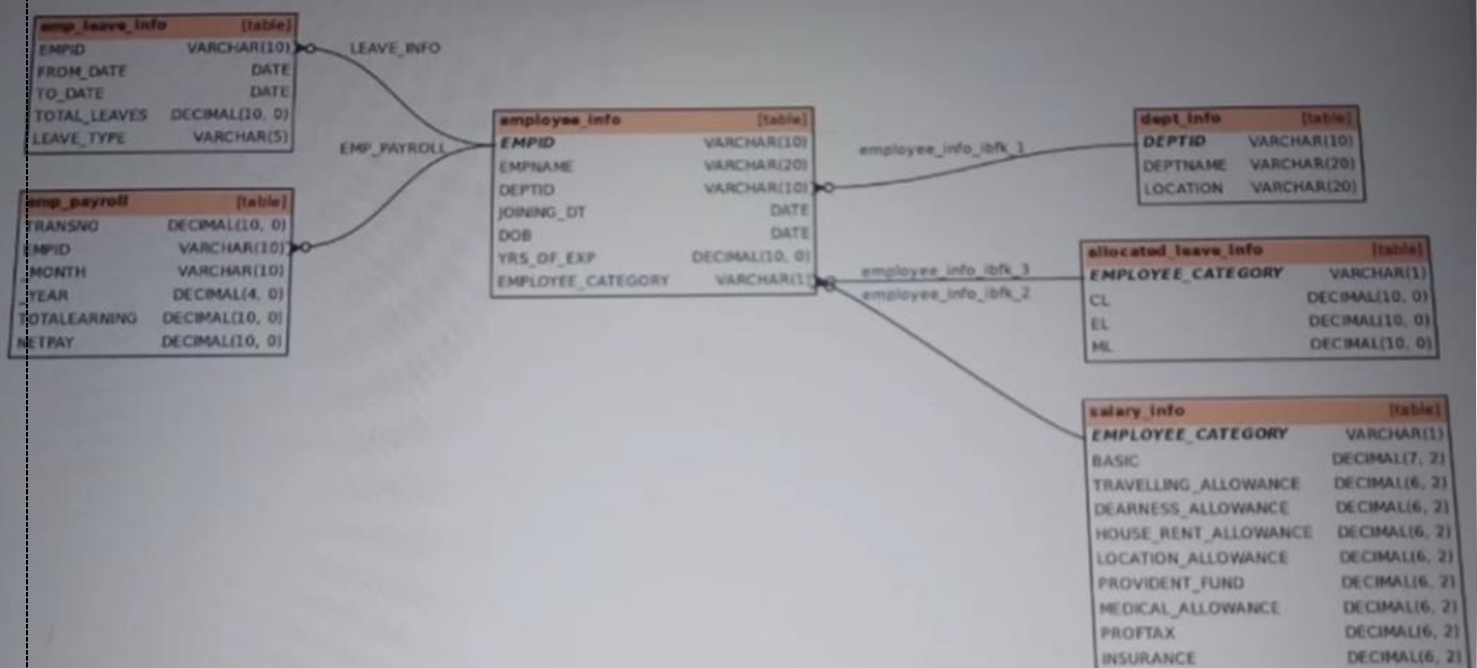
Your output should have 3 columns as given below:

EMPID	LEAVE_TYPE	TOTAL_LEAVES
-------	------------	--------------

SQL Query

```
-- Selects the EMPID, LEAVE_TYPE, and TOTAL_LEAVES columns.
SELECT
    EMPID,
    LEAVE_TYPE,
    TOTAL_LEAVES
FROM
    -- Specifies the table containing leave information.
    emp_leave_info
WHERE
    -- Filters the results based on two conditions:
    -- 1. TOTAL_LEAVES is greater than 10.
    -- 2. LEAVE_TYPE is either 'CL' (Casual Leave) or 'ML' (Medical Leave).
    TOTAL_LEAVES > 10
    AND LEAVE_TYPE IN ('CL', 'ML');
```

Qn:



Write an SQL query to display:

The ID, Name, Department name, and Basic salary of employees working in the 'HR' department.

Your output should have 4 columns as given below:

EMPID	EMPNAME	DEPTNAME	BASIC
-------	---------	----------	-------

SQL Query

```
-- Selects the employee ID, employee name, department name, and basic salary.
SELECT
    ei.EMPID,          -- Employee ID from employee_info table
    ei.EMPNAME,        -- Employee Name from employee_info table
    di.DEPTNAME,       -- Department Name from dept_info table
    si.BASIC           -- Basic Salary from salary_info table
FROM
    -- Starts with the employee_info table, aliased as 'ei'.
    employee_info AS ei
-- Joins employee_info with dept_info on the DEPTID to get department details.
JOIN
    dept_info AS di ON ei.DEPTID = di.DEPTID
-- Joins employee_info with salary_info on EMPLOYEE_CATEGORY to get salary details.
JOIN
    salary_info AS si ON ei.EMPLOYEE_CATEGORY = si.EMPLOYEE_CATEGORY
WHERE
    -- Filters the results to include only employees from the 'HR' department.
    di.DEPTNAME = 'HR';
```

6.

You need to arrange books in a library with the help of students. Each student has a collection of books. Bookshelves can hold a maximum of K books and must be fully filled. If a student has more books than shelf capacity, the excess books carry over to the next student's shelf. If they have fewer books, those books also carry over to the next student's shelf. Your task is to calculate and return the total number of fully decked bookshelves.

If the last student has any extra books, those books are discarded.

If the last student has fewer books than that of shelf capacity, then those books are to be discarded.

Input Specification:

- input1: An integer representing the number of students
- input2: An integer K, representing the maximum number of books a shelf can hold.
- input3: An integer array representing the number of books each student has.

Output Specification:

Return an integer value representing the total number of bookshelves that are completely decked by the books.

Example:

input1: 4 input2: 4 input3: [5, 7, 8, 10] Output: 7

Solutions

C++

```
#include <iostream>
#include <vector>
using namespace std;
int countFilledShelves(int N, int K, vector<int>& books) {
    int shelves = 0, carry = 0;
    for (int i = 0; i < N; i++) {
        int total = books[i] + carry;
        shelves += total / K;
        carry = total % K;
    }
    return shelves;
}
int main() {
    int input1, input2;
    cout << "input1: ";
    cin >> input1;
    cout << "input2: ";
    cin >> input2;
    cout << "input3: ";
    vector<int> input3(input1);
    for (int i = 0; i < input1; i++) {
        cin >> input3[i];
    }
    cout << countFilledShelves(input1, input2, input3) << endl;
    return 0;
}
```

Java

```
import java.util.*;

public class BookshelfCounter {

    public static int countFilledShelves(int N, int K, int[] books) {
        int shelves = 0;
        int carry = 0;
        for (int i = 0; i < N; i++) {
            int total = books[i] + carry;
            shelves += total / K;
            carry = total % K;
        }
        return shelves;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("input1: ");
        int input1 = sc.nextInt();
        System.out.print("input2: ");
        int input2 = sc.nextInt();
        System.out.print("input3: ");
        int[] input3 = new int[input1];
        for (int i = 0; i < input1; i++) {
            input3[i] = sc.nextInt();
        }
        System.out.println(countFilledShelves(input1, input2, input3));
    }
}
```

Python

```
def count_filled_shelves(N, K, books):
    carry = 0
```

```

shelves = 0
for i in range(N):
    total = books[i] + carry
    shelves += total // K
    carry = total % K
return shelves

# Input
input1 = int(input("input1: "))
input2 = int(input("input2: "))
input3 = list(map(int, input("input3: ").strip("[]").split(',')))
print(count_filled_shelves(input1, input2, input3))

```

7.

You have a lift in your society that can accommodate a maximum weight of X units. The weights of the people planning to use the lift are given in an integer array A of size N. Your task is to find and return an integer value representing the maximum number of people that can use the lift together without exceeding its weight capacity.

Input Specification:

- input1: An integer N representing the number of people planning to use the lift
- input2: An integer X representing the maximum weight capacity of the lift
- input3: An integer array A containing the weights of the people planning to use the lift

Output Specification:

Return an integer value representing the maximum number of people that can use the lift together without exceeding its weight capacity.

Example:

input1: 3

input2: 9

input3: [5,1,5]

#Output: 2

Solutions

C++


```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int maxPeople(int N, int X, vector<int>& A) {
    sort(A.begin(), A.end());
    int total = 0, count = 0;
    for (int w : A) {
        if (total + w <= X) {
            total += w;
            count++;
        } else {
            break;
        }
    }
    return count;
}
int main() {
    int input1, input2;
    cout << "input1: ";
    cin >> input1;
    cout << "input2: ";
    cin >> input2;
    cout << "input3: ";
    vector<int> input3(input1);
    for (int i = 0; i < input1; i++) {
        cin >> input3[i];
    }
    cout << maxPeople(input1, input2, input3) << endl;
    return 0;
}

```

```
}
```

Java

```
import java.util.*;
```

```
public class LiftCapacity {
```

```
    public static int maxPeople(int N, int X, int[] A) {
```

```
        Arrays.sort(A);
```

```
        int total = 0, count = 0;
```

```
        for (int weight : A) {
```

```
            if (total + weight <= X) {
```

```
                total += weight;
```

```
                count++;
```

```
            } else {
```

```
                break;
```

```
            }
```

```
        }
```

```
        return count;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("input1: ");
```

```
        int input1 = sc.nextInt();
```

```
        System.out.print("input2: ");
```

```
        int input2 = sc.nextInt();
```

```
        System.out.print("input3: ");
```

```
        int[] input3 = new int[input1];
```

```
        for (int i = 0; i < input1; i++) {
```

```
            input3[i] = sc.nextInt();
```

```
        }
```

```
        System.out.println(maxPeople(input1, input2, input3));
```

```
    }
```

```
}
```

Python

```
def max_people(N, X, A):  
    A.sort()  
    total_weight = 0  
    count = 0  
    for weight in A:  
        if total_weight + weight <= X:  
            total_weight += weight  
            count += 1  
        else:  
            break  
    return count  
  
# Input  
input1 = int(input("input1: "))  
input2 = int(input("input2: "))  
input3 = list(map(int, input("input3: ").strip("[]").split(',')))  
print(max_people(input1, input2, input3))
```

8. Unique airport codes

Mike is travelling on a flight and he was curious to see the 3-character codes on the ticket. He has a string **S** of length **N** (which is always a multiple of 3), and he has to split the string in such a way that each partition is 3 characters long, which may resemble any airport code.

The subset should not overlap the partitioning, which means the partition should be like:

- from position 1–3,
- then 4–6,
- then 7–9, and so on.

For Mike, **ABC** is the same as **BAC** or **CAB**, as they contain the same letters. So he is interested in knowing how many **unique codes** exist in the given string **S**.

Your task is to find and return an **integer** value representing the **unique number of 3-character codes** present in the string if it is split into groups of 3.

Input1: 12

Input2: ABCZABCADDAC

Output: 3

Explanation : Since ABC , ZAB ,CAD are **3** unique sets

Solution

C++

```
#include <iostream>
#include <set>
#include <algorithm>
using namespace std;
int uniqueCodes(int N, string S) {
    set<string> seen;
    for (int i = 0; i < N; i += 3) {
        string code = S.substr(i, 3);
        sort(code.begin(), code.end());
        seen.insert(code);
    }
    return seen.size();
}
int main() {
    int input1;
    string input2;
    cout << "input1: ";
    cin >> input1;
    cout << "input2: ";
    cin >> input2;
    cout << uniqueCodes(input1, input2) << endl;
    return 0;
}
```

Java

```
import java.util.*;
```

```

public class UniqueAirportCodes {
    public static int uniqueCodes(int N, String S) {
        Set<String> seen = new HashSet<>();
        for (int i = 0; i < N; i += 3) {
            char[] chars = S.substring(i, i + 3).toCharArray();
            Arrays.sort(chars);
            seen.add(new String(chars));
        }
        return seen.size();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("input1: ");
        int input1 = sc.nextInt();
        System.out.print("input2: ");
        String input2 = sc.next();
        System.out.println(uniqueCodes(input1, input2));
    }
}

```

Python

```

def unique_codes(N, S):
    seen = set()
    for i in range(0, N, 3):
        code = ".".join(sorted(S[i:i+3]))
        seen.add(code)
    return len(seen)

# Input
input1 = int(input("input1: "))
input2 = input("input2: ")
print(unique_codes(input1, input2))

```

9.

There are three stones in different positions on the X-axis. You are given three integers a , b , and c , the positions of the stones.

In one move, you pick up a stone at an endpoint (i.e., either the lowest or highest position stone), and move it to an unoccupied position between those endpoints. Formally, let's say the stones are currently at positions x , y , and z with $x < y < z$. You pick up the stone at either position x or position z , and move that stone to an integer position k , with $x < k < z$ and $k \neq y$.

The game ends when you cannot make any more moves (i.e., the stones are in three consecutive positions).

Return an integer array `answer` of length 2 where:

`answer[0]` is the minimum number of moves you can play, and

`answer[1]` is the maximum number of moves you can play.

Example 1:

Input: $a = 1, b = 2, c = 5$

Output: `[1,2]`

Explanation: Move the stone from 5 to 3, or move the stone from 5 to 4 to 3.

Example 2:

Input: $a = 4, b = 3, c = 2$

Output: `[0,0]`

Solution

C++

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```

using namespace std;
vector<int> numMovesStones(int a, int b, int c) {
    vector<int> result;

    // Sort the values
    vector<int> stones = {a, b, c};
    sort(stones.begin(), stones.end());
    int x = stones[0], y = stones[1], z = stones[2];
    int max_moves = (y - x - 1) + (z - y - 1);
    int min_moves = 0;
    if (y - x == 1 && z - y == 1) {
        min_moves = 0;
    } else if (y - x <= 2 || z - y <= 2) {
        min_moves = 1;
    } else {
        min_moves = 2;
    }
    result.push_back(min_moves);
    result.push_back(max_moves);
    return result;
}

int main() {
    int a, b, c;
    // Take input from user
    cout << "Enter positions of the three stones (a b c): ";
    cin >> a >> b >> c;
    vector<int> res = numMovesStones(a, b, c);
    // Output result
    cout << "[" << res[0] << ", " << res[1] << "]\n";
    return 0;}

```

Java

```
import java.util.*;

public class ThreeStonesGame {

    public static int[] numMovesStones(int a, int b, int c) {

        int[] stones = {a, b, c};
        Arrays.sort(stones);
        int x = stones[0], y = stones[1], z = stones[2];

        if (z - x == 2)
            return new int[]{0, 0};
        else if (y - x <= 2 || z - y <= 2)
            return new int[]{1, z - x - 2};
        else
            return new int[]{2, z - x - 2};
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("input1: ");
        int a = sc.nextInt();
        System.out.print("input2: ");
        int b = sc.nextInt();
        System.out.print("input3: ");
        int c = sc.nextInt();
        int[] res = numMovesStones(a, b, c);
        System.out.println "[" + res[0] + ", " + res[1] + "]");
    }
}
```

Python

```
def numMovesStones(a, b, c):
    x, y, z = sorted([a, b, c])
```



```
if z - x == 2:
    return [0, 0]
elif y - x <= 2 or z - y <= 2:
    return [1, z - x - 2]
else:
    return [2, z - x - 2]

# Input
a = int(input("input1: "))
b = int(input("input2: "))
c = int(input("input3: "))
print(numMovesStones(a, b, c))
```

10. Write the SQL Query based on the schema below the question.

The names of employees who have more than 5 years of experience and joined after January 1, 2001. Use alias "Employee ID", "Employee Name"

Your output should have 2 columns as given below:

| Employee ID | Employee Name |

employee_info EMPID VARCHAR(10) EMPNAME VARCHAR(20) DEPTID VARCHAR(10) JOINING_DT DATE DOB DATE YRS_OF_EXP DECIMAL(10,0) EMPLOYEE_CATEGORY(1)	dept_info DEPTID VARCHAR(10) DEPTNAME VARCHAR(20) LOCATION VARCHAR(20)
emp_leave_info EMPID VARCHAR(10) FROM_DATE DATE TO_DATE DATE TOTAL_LEAVES DECIMAL(10,0) LEAVE_TYPE VARCHAR(5)	emp_payroll TRANSNO DECIMAL(100,0) EMPID VARCHAR(10) MONTH VARCHAR(10) YEAR DECIMAL(4,0) TOTALEARN DECIMAL(10,0) NETPAY DECIMAL(10,0)
salary_info EMPLOYEE_CATEGORY BASIC DECIMAL(7,2)	allocated_leave_info EMPLOYEE_CATEGORY CL DECIMAL(1) EL DECIMAL(10,0) ML DECIMAL(10,0)
	salary_info EMPLOYEE_CATEGORY BASIC DECIMAL(72) TRAVELLING_ALLOWANCE(6,2) DEARNESS_ALLOWANCE(6,2) HOUSE_RENT_ALLOWANCE(6,2) LOCATION_ALLOWANCE(6,2) PROVIDENT_FUND (6,2) MEDICAL_ALLOWANCE(6,2) PROETAX DECIMAL(6,2)

SQL Query:

```
-- Selects the Employee ID and Employee Name with specified aliases.
SELECT
    EMPID AS "Employee ID",          -- Employee ID column, aliased as "Employee ID"
    EMPNAME AS "Employee Name"      -- Employee Name column, aliased as "Employee Name"
FROM
    -- Specifies the table containing employee information.
    employee_info
WHERE
    -- Filters the results based on two conditions:
    -- 1. YRS_OF_EXP (Years of Experience) is greater than 5.
    YRS_OF_EXP > 5
    -- 2. JOINING_DT (Joining Date) is after '2001-01-01'.
    -- Date literals should typically be in 'YYYY-MM-DD' format.
    AND JOINING_DT > '2001-01-01';
```

11. Write an SQL Query to display

PATIENTID, PAYMENTSTATUS, OUTSTANDING AMOUNT

The **PatientID**, **PaymentStatus**, and **Outstanding Amount** (use alias "Outstanding Amount") for patients whose bill has not been fully paid (i.e., **PaidAmount** is less than **TotalAmount**).

Note: Outstanding Amount = **TotalAmount - PaidAmount**

Your output should have three columns as given below:

Query:

```
SELECT PatientID, PaymentStatus,  
(TotalAmount - PaidAmount) AS Outstanding  
Amount FROM Patients  
WHERE PaidAmount < TotalAmount;
```

12.

Write a program that checks whether the sum of the first half of an array is less than the sum of the second half. If this condition is met, the program should reverse the entire array. Finally, the program should print the resulting array."

Solution

C++

```
#include <iostream>  
#include <vector>  
#include <algorithm>  
using namespace std;  
int main() {  
    int n;  
    cin >> n;  
    vector<int> arr(n);  
    for (int i = 0; i < n; i++) cin >> arr[i];  
    int mid = n / 2;  
    int sum1 = 0, sum2 = 0;  
    for (int i = 0; i < mid; i++) sum1 += arr[i];  
    for (int i = mid; i < n; i++) sum2 += arr[i];  
    if (sum1 < sum2) reverse(arr.begin(), arr.end());  
    for (int i = 0; i < n; i++) cout << arr[i] << " ";  
    return 0;  
}
```

Java

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n];

        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        int mid = n / 2;

        int sum1 = 0, sum2 = 0;

        for (int i = 0; i < mid; i++) sum1 += arr[i];

        for (int i = mid; i < n; i++) sum2 += arr[i];

        if (sum1 < sum2) {

            for (int i = 0; i < n / 2; i++) {

                int temp = arr[i];

                arr[i] = arr[n - 1 - i];

                arr[n - 1 - i] = temp;

            }

        }

        for (int i = 0; i < n; i++) System.out.print(arr[i] + " ");

    }

}
```

Python

```
n = int(input())

arr = list(map(int, input().split()))

mid = n // 2

sum1 = sum(arr[:mid])

sum2 = sum(arr[mid:])

if sum1 < sum2:

    arr.reverse()
```

```
print(*arr)
```

13.

Write a program to calculate the fuel consumption of your truck. The program should ask the user to enter the quantity of diesel to fill up the tank and the distance covered till the tank goes dry. Calculate the fuel consumption and display it in the format (liters per 100 kilometers).

Convert the same result to the U.S. style of miles per gallon and display the result. If the quantity or distance is zero or negative display " is an Invalid Input".

[Note: The US approach of fuel consumption calculation (distance / fuel) is the inverse of the European approach (fuel / distance). Also note that 1 kilometer is 0.6214 miles, and 1 liter is 0.2642 gallons.]

The result should be with two decimal place . To get two decimal place refer the below-mentioned print statement :

```
float cost=670.23;
```

```
System.out.printf("You need a sum of Rs.%.2f to cover the trip",cost);
```

Sample Input 1:

- Enter the no of liters to fill the tank
20
- Enter the distance covered
150

Sample Output 1:

- Liters/100KM
13.33
- Miles/gallons
17.64

Explanation:

- For 150 KM fuel consumption is 20 liters,
- Then for 100 KM fuel consumption would be $(20/150)*100=13.33$,
- Distance is given in KM, we have to convert it to miles $(150*0.6214)=93.21$,
- Fuel consumption is given in liters, we have to convert it to gallons $(20*0.2642)=5.284$,
- Then find $(\text{miles/gallons})=(93.21/5.284)=17.64$

Sample Input 2:

- Enter the no of liters to fill the tank
-5

Sample Output 2:

- -5 is an Invalid Input

Solutions

C++

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    float liters, distance;
    cout << "Enter the no of liters to fill the tank\n";
    cin >> liters;
    if (liters <= 0) {
        cout << fixed << setprecision(0) << liters << " is an Invalid Input\n";
        return 0;
    }
    cout << "Enter the distance covered\n";
    cin >> distance;
    if (distance <= 0) {
        cout << fixed << setprecision(0) << distance << " is an Invalid Input\n";
        return 0;
    }
    float litersPer100km = (liters / distance) * 100;
    float miles = distance * 0.6214;
    float gallons = liters * 0.2642;
    float milesPerGallon = miles / gallons;
    cout << "Liters/100KM\n";
```

```

cout << fixed << setprecision(2) << litersPer100km << endl;
cout << "Miles/gallons\n";
cout << fixed << setprecision(2) << milesPerGallon << endl;
return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the no of liters to fill the tank");
        float liters = sc.nextFloat();
        if (liters <= 0) {
            System.out.printf("%.0f is an Invalid Input\n", liters);
            return;
        }

        System.out.println("Enter the distance covered");
        float distance = sc.nextFloat();
        if (distance <= 0) {
            System.out.printf("%.0f is an Invalid Input\n", distance);
            return;
        }

        float litersPer100km = (liters / distance) * 100;
        float miles = distance * 0.6214f;
        float gallons = liters * 0.2642f;
        float milesPerGallon = miles / gallons;

        System.out.println("Liters/100KM");
        System.out.printf("%.2f\n", litersPer100km);
        System.out.println("Miles/gallons");
        System.out.printf("%.2f\n", milesPerGallon);
    }
}

```

```
}  
}
```

Python

```
liters = float(input("Enter the no of liters to fill the tank\n"))
```

```
if liters <= 0:
```

```
    print(f"{int(liters)} is an Invalid Input")
```

```
else:
```

```
    distance = float(input("Enter the distance covered\n"))
```

```
    if distance <= 0:
```

```
        print(f"{int(distance)} is an Invalid Input")
```

```
    else:
```

```
        liters_per_100km = (liters / distance) * 100
```

```
        miles = distance * 0.6214
```

```
        gallons = liters * 0.2642
```

```
        mpg = miles / gallons
```

```
        print("Liters/100KM")
```

```
        print(f"{liters_per_100km:.2f}")
```

```
        print("Miles/gallons")
```

```
        print(f"{mpg:.2f}")
```

14. Vohra went to a movie with his friends in a Wave theatre and during break time he bought pizzas, puffs and cool drinks. Consider the following prices :

- Rs.100/pizza
- Rs.20/puffs
- Rs.10/cooldrink

Generate a bill for What Vohra has bought.

Sample Input 1:

- Enter the no of pizzas bought :10
- Enter the no of puffs bought :12
- Enter the no of cool drinks bought :5

Sample Output 1:

Bill Details

- No of pizzas:10
- No of puffs:12
- No of cooldrinks:5
- Total price=1290

ENJOY THE SHOW!!!

Solutions

C++

```
#include <iostream>
using namespace std;
int main() {
    int pizzas, puffs, cooldrinks;
    cout << "Enter the no of pizzas bought:";
    cin >> pizzas;
    cout << "Enter the no of puffs bought:";
    cin >> puffs;
    cout << "Enter the no of cool drinks bought:";
    cin >> cooldrinks;
    int total = (pizzas * 100) + (puffs * 20) + (cooldrinks * 10);
    cout << "\nBill Details\n";
    cout << "No of pizzas:" << pizzas << endl;
    cout << "No of puffs:" << puffs << endl;
    cout << "No of cooldrinks:" << cooldrinks << endl;
    cout << "Total price=" << total << endl;
    cout << "ENJOY THE SHOW!!!\n";
    return 0;
}
```

Java

```
import java.util.Scanner;
```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the no of pizzas bought:");
        int pizzas = sc.nextInt();
        System.out.print("Enter the no of puffs bought:");
        int puffs = sc.nextInt();
        System.out.print("Enter the no of cool drinks bought:");
        int cooldrinks = sc.nextInt();
        int total = (pizzas * 100) + (puffs * 20) + (cooldrinks * 10);
        System.out.println("\nBill Details");
        System.out.println("No of pizzas:" + pizzas);
        System.out.println("No of puffs:" + puffs);
        System.out.println("No of cooldrinks:" + cooldrinks);
        System.out.println("Total price=" + total);
        System.out.println("ENJOY THE SHOW!!!");
    }
}

```

Python

```

pizzas = int(input("Enter the no of pizzas bought:"))
puffs = int(input("Enter the no of puffs bought:"))
cooldrinks = int(input("Enter the no of cool drinks bought:"))
total = (pizzas * 100) + (puffs * 20) + (cooldrinks * 10)
print("\nBill Details")
print(f"No of pizzas:{pizzas}")
print(f"No of puffs:{puffs}")
print(f"No of cooldrinks:{cooldrinks}")
print(f"Total price={total}")
print("ENJOY THE SHOW!!!")

```

15.

Ritik wants a magic board, which displays a character for a corresponding number for his science project. Help him to develop such an application. For example when the digits 65,66,67,68 are entered, the alphabet ABCD are to be displayed.

[Assume the number of inputs should be always 4]

Sample Input 1:

Enter the digits:

65

66

67

68

Sample Output 1:

65-A

66-B

67-C

68-D

Sample Input 2:

Enter the digits:

115

116

101

112

Sample Output 2:

115-s

116-t

101-e

112-p

Solutions

C++

```
#include <iostream>
using namespace std;
int main() {
    int digits[4];
    cout << "Enter the digits:\n";
    for (int i = 0; i < 4; i++) {
```

```

        cin >> digits[i];
    }
    for (int i = 0; i < 4; i++) {
        cout << digits[i] << "-" << char(digits[i]) << endl;
    }
    return 0;}

```

Java

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        int[] digits = new int[4];
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the digits:");
        for (int i = 0; i < 4; i++) {
            digits[i] = sc.nextInt();
        }
        for (int i = 0; i < 4; i++) {
            System.out.println(digits[i] + "-" + (char)digits[i]);
        }
    }
}

```

Python

```

digits = []
print("Enter the digits:")
for _ in range(4):
    num = int(input())
    digits.append(num)
for num in digits:
    print(f"{num}-{chr(num)}")

```

16.

FOE college wants to recognize the department which has succeeded in getting the maximum number of placements for this academic year. The departments that have participated in the recruitment drive are CSE,ECE, MECH. Help the college find the department getting maximum placements. Check for all the possible output given in the sample snapshot

Note : If any input is negative, the output should be "Input is Invalid". If all department has equal number of placements, the output should be "None of the department has got the highest placement".

Sample Input 1:

- Enter the no of students placed in CSE:90
- Enter the no of students placed in ECE:45
- Enter the no of students placed in MECH:70

Sample Output 1:

- Highest placement
CSE

Sample Input 2:

- Enter the no of students placed in CSE:55
- Enter the no of students placed in ECE:85
- Enter the no of students placed in MECH:85

Sample Output 2:

- Highest placement
ECE
MECH

Solutions

C++

```
#include <iostream>
using namespace std;
int main() {
    int cse, ece, mech;
    cout << "Enter the no of students placed in CSE:";
```

```

cin >> cse;
cout << "Enter the no of students placed in ECE:";
cin >> ece;
cout << "Enter the no of students placed in MECH:";
cin >> mech;
if (cse < 0 || ece < 0 || mech < 0) {
    cout << "Input is Invalid";
} else if (cse == ece && ece == mech) {
    cout << "None of the department has got the highest placement";
} else {
    cout << "Highest placement\n";
    int max_val = max(cse, max(ece, mech));
    if (cse == max_val) cout << "CSE\n";
    if (ece == max_val) cout << "ECE\n";
    if (mech == max_val) cout << "MECH\n";
}
return 0;
}

```

Java

```

import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the no of students placed in CSE:");
        int cse = sc.nextInt();
        System.out.print("Enter the no of students placed in ECE:");
        int ece = sc.nextInt();
        System.out.print("Enter the no of students placed in MECH:");
        int mech = sc.nextInt();
        if (cse < 0 || ece < 0 || mech < 0) {

```

```

        System.out.println("Input is Invalid");
    } else if (cse == ece && ece == mech) {
        System.out.println("None of the department has got the highest placement");
    } else {
        System.out.println("Highest placement");
        int max = Math.max(cse, Math.max(ece, mech));
        if (cse == max) System.out.println("CSE");
        if (ece == max) System.out.println("ECE");
        if (mech == max) System.out.println("MECH");
    }
}
}

```

Python

```

cse = int(input("Enter the no of students placed in CSE:"))
ece = int(input("Enter the no of students placed in ECE:"))
mech = int(input("Enter the no of students placed in MECH:"))
if cse < 0 or ece < 0 or mech < 0:
    print("Input is Invalid")
elif cse == ece == mech:
    print("None of the department has got the highest placement")
else:
    print("Highest placement")
    max_val = max(cse, ece, mech)
    if cse == max_val:
        print("CSE")
    if ece == max_val:
        print("ECE")
    if mech == max_val:
        print("MECH")

```

17.

To speed up his composition of generating unpredictable rhythms, Blue Bandit wants the list of prime numbers available in a range of numbers. Can you help him out?

Write a program to print all prime numbers in the interval [a,b] (a and b, both inclusive).

Note

- Input 1 should be lesser than Input 2. Both the inputs should be positive.
- Range must always be greater than zero.
- If any of the condition mentioned above fails, then display "Provide valid input"
- Use a minimum of one for loop and one while loop

Sample Input 1:

2

15

Sample Output 1:

2 3 5 7 11 13

Sample Input 2:

8

5

Sample Output 2:

- Provide valid input

Solutions

C++

```
#include <iostream>
using namespace std;
bool isPrime(int num) {
    if (num < 2) return false;
    int i = 2;
    while (i * i <= num) {
```



```

        if (num % i == 0)
            return false;
        i++;
    }
    return true;
}

int main() {
    int a, b;
    cin >> a >> b;
    if (a <= 0 || b <= 0 || a >= b) {
        cout << "Provide valid input";
        return 0;
    }
    for (int i = a; i <= b; i++) {
        if (isPrime(i)) {
            cout << i << " ";
        }
    }
    return 0;}

```

Java

```

import java.util.Scanner;

public class Main {

    public static boolean isPrime(int num) {
        if (num < 2) return false;
        int i = 2;
        while (i * i <= num) {
            if (num % i == 0)
                return false;
            i++;
        }
        return true;
    }
}

```

```

}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int a = sc.nextInt();
    int b = sc.nextInt();
    if (a <= 0 || b <= 0 || a >= b) {
        System.out.println("Provide valid input");
        return;
    }
    for (int i = a; i <= b; i++) {
        if (isPrime(i)) {
            System.out.print(i + " ");
        }
    }
}

```

Python

```

a = int(input())
b = int(input())
if a <= 0 or b <= 0 or a >= b:
    print("Provide valid input")
else:
    for num in range(a, b + 1):
        if num < 2:
            continue
        i = 2
        is_prime = True
        while i * i <= num:
            if num % i == 0:
                is_prime = False
                break
            i += 1
        if is_prime:
            print(num, end=" ")

```

18.

Raj wants to know the maximum marks scored by him in each semester. The mark should be between 0 to 100 ,if goes beyond the range display "You have entered invalid mark."

Sample Input 1:

- Enter no of semester:
3
- Enter no of subjects in 1 semester:
3
- Enter no of subjects in 2 semester:
4
- Enter no of subjects in 3 semester:
2
- Marks obtained in semester 1:
50
60
70
- Marks obtained in semester 2:
90
98
76
67
- Marks obtained in semester 3:
89
76

Sample Output 1:

- Maximum mark in 1 semester:70
- Maximum mark in 2 semester:98
- Maximum mark in 3 semester:89

Solutions

C++

```
#include <iostream>
```

```

#include <vector>
using namespace std;
int main() {
    int semCount;
    cout << "Enter no of semester:\n";
    cin >> semCount;
    vector<int> maxMarks(semCount);
    for (int i = 0; i < semCount; i++) {
        int subCount;
        cout << "Enter no of subjects in " << i + 1 << " semester:\n";
        cin >> subCount;
        cout << "Marks obtained in semester " << i + 1 << ":\n";
        int mark, maxMark = -1;
        for (int j = 0; j < subCount; j++) {
            cin >> mark;
            if (mark < 0 || mark > 100) {
                cout << "You have entered invalid mark." << endl;
                return 0;
            }
            if (mark > maxMark)
                maxMark = mark;
        }
        maxMarks[i] = maxMark;
    }
    for (int i = 0; i < semCount; i++) {
        cout << "Maximum mark in " << i + 1 << " semester:" << maxMarks[i] << endl;
    }
    return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter no of semester:");

        int semCount = sc.nextInt();

        int[] maxMarks = new int[semCount];

        for (int i = 0; i < semCount; i++) {

            System.out.println("Enter no of subjects in " + (i + 1) + " semester:");

            int subCount = sc.nextInt();

            System.out.println("Marks obtained in semester " + (i + 1) + ":");

            int maxMark = -1;

            for (int j = 0; j < subCount; j++) {

                int mark = sc.nextInt();

                if (mark < 0 || mark > 100) {

                    System.out.println("You have entered invalid mark.");

                    return;

                }

                if (mark > maxMark)

                    maxMark = mark;

            }

            maxMarks[i] = maxMark;

        }

        for (int i = 0; i < semCount; i++) {

            System.out.println("Maximum mark in " + (i + 1) + " semester:" +

maxMarks[i]);

        }

    }

}

```

Python

```
sem_count = int(input("Enter no of semester:\n"))
```

```

max_marks = []
for i in range(sem_count):
    sub_count = int(input(f"Enter no of subjects in {i + 1} semester:\n"))
    print(f"Marks obtained in semester {i + 1}:")
    max_mark = -1
    for _ in range(sub_count):
        mark = int(input())
        if mark < 0 or mark > 100:
            print("You have entered invalid mark.")
            exit()
        if mark > max_mark:
            max_mark = mark
    max_marks.append(max_mark)
for i, m in enumerate(max_marks):
    print(f"Maximum mark in {i + 1} semester:{m}")

```

19.

Chaman planned to choose a four digit lucky number for his car. His lucky numbers are 3,5 and 7. Help him find the number, whose sum is divisible by 3 or 5 or 7. Provide a valid car number, Fails to provide a valid input then display that number is not a valid car number.

Note : The input other than 4 digit positive number[includes negative and 0] is considered as invalid.

Sample Input 1:

- Enter the car no:1234

Sample Output 1:

- Lucky Number

Sample Input 2:

- Enter the car no:1214

Sample Output 2:

- Sorry its not my lucky number

Solution

C++

```
#include <iostream>
using namespace std;
int main() {
    int num;
    cout << "Enter the car no:";
    cin >> num;
    if (num < 1000 || num > 9999) {
        cout << num << " is not a valid car number" << endl;
        return 0;
    }
    int sum = 0, temp = num;
    while (temp > 0) {
        sum += temp % 10;
        temp /= 10;
    }
    if (sum % 3 == 0 || sum % 5 == 0 || sum % 7 == 0)
        cout << "Lucky Number" << endl;
    else
        cout << "Sorry its not my lucky number" << endl;

    return 0;
}
```

Java

```
import java.util.Scanner;
public class LuckyCar {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```

System.out.print("Enter the car no:");
int num = sc.nextInt();
if (num < 1000 || num > 9999) {
    System.out.println(num + " is not a valid car number");
    return;
}
int sum = 0, temp = num;
while (temp > 0) {
    sum += temp % 10;
    temp /= 10;
}
if (sum % 3 == 0 || sum % 5 == 0 || sum % 7 == 0)
    System.out.println("Lucky Number");
else
    System.out.println("Sorry its not my lucky number");
}
}

```

Python

```

num = int(input("Enter the car no:"))
if num < 1000 or num > 9999:
    print(f"{num} is not a valid car number")
else:
    digit_sum = sum(int(d) for d in str(num))
    if digit_sum % 3 == 0 or digit_sum % 5 == 0 or digit_sum % 7 == 0:
        print("Lucky Number")
    else:
        print("Sorry its not my lucky number")

```

20.

XYZ Technologies is in the process of increment the salary of the employees. This increment is done based on their salary and their performance appraisal rating.

1. If the appraisal rating is between 1 and 3, the increment is 10% of the salary.
2. If the appraisal rating is between 3.1 and 4, the increment is 25% of the salary.
3. If the appraisal rating is between 4.1 and 5, the increment is 30% of the salary.

Note : If either the salary is 0 or negative (or) if the appraisal rating is not in the range 1 to 5 (inclusive), then the output should be "Invalid Input".

Sample Input 1 :

- Enter the salary
8000
- Enter the Performance appraisal rating
3

Sample Output 1 :

8800

Sample Input 2 :

- Enter the salary
7500
- Enter the Performance appraisal rating
4.3

Sample Output 2 :

9750

Solutions

C++

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    double salary, rating;
    cout << "Enter the salary" << endl;
    cin >> salary;
```

```

cout << "Enter the Performance appraisal rating" << endl;
cin >> rating;
if (salary <= 0 || rating < 1 || rating > 5) {
    cout << "Invalid Input" << endl;
} else {
    double increment = 0;
    if (rating >= 1 && rating <= 3) {
        increment = 0.10 * salary;
    } else if (rating > 3 && rating <= 4) {
        increment = 0.25 * salary;
    } else if (rating > 4 && rating <= 5) {
        increment = 0.30 * salary;
    }
    double newSalary = salary + increment;
    cout << fixed << setprecision(0) << newSalary << endl;
}
return 0;
}

```

Java

```

import java.util.Scanner;

public class IncrementCalculation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Taking salary input
        System.out.println("Enter the salary");
        double salary = sc.nextDouble();
        // Taking performance appraisal rating input
        System.out.println("Enter the Performance appraisal rating");
        double rating = sc.nextDouble();
        // Check for valid inputs
    }
}

```

```

if (salary <= 0 || rating < 1 || rating > 5) {
    System.out.println("Invalid Input");
} else {
    double increment = 0;
    // Determine the increment percentage
    if (rating >= 1 && rating <= 3) {
        increment = salary * 0.10;
    } else if (rating > 3 && rating <= 4) {
        increment = salary * 0.25;
    } else if (rating > 4 && rating <= 5) {
        increment = salary * 0.30;
    }
    // Display incremented salary
    double newSalary = salary + increment;
    System.out.printf("%.0f\n", newSalary);
}
sc.close();
}
}

```

Python

```

salary = float(input("Enter the salary\n"))
rating = float(input("Enter the Performance appraisal rating\n"))
if salary <= 0 or rating < 1 or rating > 5:
    print("Invalid Input")
else:
    if 1 <= rating <= 3:
        increment = 0.10 * salary
    elif 3 < rating <= 4:
        increment = 0.25 * salary
    else:

```

```
increment = 0.30 * salary
new_salary = salary + increment
print(f"{new_salary:.0f}")
```

-----Tech Mahindra-----

1.

A geometric progression (G.P.) is a sequence of numbers where each element after the first is found by multiplying the previous one by a fixed, non-zero number called the common ratio.

Example: 1, 3, 9, 27, 81 is a G.P. with a ratio of 3.

Task: Determine the nth term, given that the second and third terms of the G.P. are provided.

Input:

1. Second term of the G.P. (in double)
2. Third term of the G.P. (in double)
3. nth term (in integer)

Output: The nth term of the series.

Solution

C++

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    double second, third;
    int n;
    cout << "Enter the second term of the G.P." << endl;
    cin >> second;
    cout << "Enter the third term of the G.P." << endl;
    cin >> third;
    cout << "Enter the value of n" << endl;
```

```

cin >> n;
double r = third / second;
double a = second / r;
double nth_term = a * pow(r, n - 1);
cout << "The " << n << "th term of the G.P. is: " << nth_term << endl;
return 0;
}

```

Java

```

import java.util.Scanner;

public class GP_TermFinder {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the second term of the G.P.:");
        double second = sc.nextDouble();

        System.out.println("Enter the third term of the G.P.:");
        double third = sc.nextDouble();

        System.out.println("Enter the value of n:");
        int n = sc.nextInt();

        double r = third / second;    // common ratio
        double a = second / r;        // first term
        double nthTerm = a * Math.pow(r, n - 1); // nth term formula
        System.out.printf("The %dth term of the G.P. is: %.2f\n", n, nthTerm);

    }

}

```

Python

```

second = float(input("Enter the second term of the G.P.\n"))
third = float(input("Enter the third term of the G.P.\n"))
n = int(input("Enter the value of n\n"))

# Calculate the common ratio
r = third / second

```

```
# Calculate the first term
a = second / r
# Calculate the nth term
nth_term = a * (r ** (n - 1))
print(f"The {n}th term of the G.P. is: {nth_term}")
```

2.

A company sent its message in encrypted form. Then it is decrypted on the other side. The encryption is done by replacing each letter with the letter at 3 positions to the left. Like 'a' is replaced with 'x', 'b' with 'y', 'd' with 'a'.

You have been provided an encrypted ciphertext as input. Determine the corresponding original text. The text is always in lowercase and consists of alphabets only.

Input: encrypted text Output: original text

Example:

Input1: nrfzh

Output: quick

Explanation:

Since encryption is done by replacing each letter with the letter at 3 positions to the left, therefore to decrypt, we need to find letters at 3 positions to the right.

Solution

C++

```
#include <iostream>
using namespace std;
int main() {
    string encrypted, decrypted = "";
    cout << "Enter encrypted text: ";
    cin >> encrypted;
    for (char ch : encrypted) {
        char original = (ch - 'a' + 3) % 26 + 'a';
```

```

        decrypted += original;
    }
    cout << "Decrypted text: " << decrypted << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class DecryptMessage {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter encrypted text:");
        String encrypted = sc.nextLine();
        StringBuilder decrypted = new StringBuilder();
        for (int i = 0; i < encrypted.length(); i++) {
            char ch = encrypted.charAt(i);
            char original = (char)((ch - 'a' + 3) % 26 + 'a');
            decrypted.append(original);
        }
        System.out.println("Decrypted text: " + decrypted);
    }
}

```

Python

```

encrypted = input("Enter encrypted text: ")
decrypted = ""
for ch in encrypted:
    original = chr((ord(ch) - ord('a') + 3) % 26 + ord('a'))
    decrypted += original
print("Decrypted text:", decrypted)

```

3.

Write a program to return the difference between the count of odd numbers and even numbers.

Note: You are expected to write code in the countOddEvenDifference function only, which will receive the first parameter as the number of items in the array and the second parameter as the array itself. You are not required to take input from the console.

Example:

Finding the difference between the count of odd and even numbers from a list of 5 numbers

Input: input 1: 8 input 2: 10 20 30 40 55 66 77 83

Output: -2

Solutions

C++

```
#include <iostream>
using namespace std;
int countOddEvenDifference(int n, int arr[]) {
    int odd = 0, even = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] % 2 == 0)
            even++;
        else
            odd++;
    }
    return odd - even;
}
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter the elements: ";
```



```

    for (int i = 0; i < n; i++)
        cin >> arr[i];
    int result = countOddEvenDifference(n, arr);
    cout << result << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {

    public static int countOddEvenDifference(int n, int[] arr) {
        int oddCount = 0, evenCount = 0;
        for (int num : arr) {
            if (num % 2 == 0)
                evenCount++;
            else
                oddCount++;
        }
        return oddCount - evenCount;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of elements:");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter the elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        int result = countOddEvenDifference(n, arr);
        System.out.println(result);
    }
}

```

```
}}
```

Python

```
def countOddEvenDifference(n, arr):  
    odd_count = sum(1 for x in arr if x % 2 != 0)  
    even_count = sum(1 for x in arr if x % 2 == 0)  
    return odd_count - even_count  
  
# Input from user  
n = int(input("Enter the number of elements: "))  
arr = list(map(int, input("Enter the elements separated by space: ").split()))  
  
# Check input length matches n  
if len(arr) != n:  
    print("Invalid input count")  
else:  
    result = countOddEvenDifference(n, arr)  
    print(result)
```

4.

A company is transmitting data to another server. The data is in the form of numbers. To secure the data during transmission, they plan to obtain a security key that will be sent along with the data. The security key is identified as the count of the unique repeating digits in the data.

Write an algorithm to find the security key for the data. The input consists of an integer, representing the data to be transmitted.

Print an integer representing the security key for the given data.

Example 1:

Input: 12345

Output: 0

Example 2:

Input: 121342

Output: 2

Solutions:

C++

```
#include <iostream>
#include <string>
using namespace std;
int findSecurityKey(int num) {
    int freq[10] = {0};
    string data = to_string(num);
    for (char ch : data) {
        freq[ch - '0']++;
    }
    int count = 0;
    for (int i = 0; i < 10; ++i) {
        if (freq[i] > 1)
            count++;
    }
    return count;
}
int main() {
    int num;
    cout << "Enter the data to be transmitted: ";
    cin >> num;
    cout << findSecurityKey(num);
    return 0;
}
```

Java

```
import java.util.Scanner;
public class SecurityKey {
    public static int findSecurityKey(int num) {
        int[] freq = new int[10];
        String str = String.valueOf(num);
```

```

        for (char c : str.toCharArray()) {
            freq[c - '0']++;
        }
        int count = 0;
        for (int f : freq) {
            if (f > 1)
                count++;
        }
        return count;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the data to be transmitted: ");
        int num = sc.nextInt();
        System.out.println(findSecurityKey(num));
    }
}

```

Python

```

def find_security_key(num):
    freq = [0] * 10 # For digits 0 to 9
    for digit in str(num):
        freq[int(digit)] += 1
    count = 0
    for f in freq:
        if f > 1:
            count += 1
    return count

# --- Get input and print result ---
num = int(input("Enter the data to be transmitted: "))
print(find_security_key(num))

```

5.

Build a program for calculating and returning the sums of absolute differences between adjacent numbers in arrays of positive integers. This must be calculated from the position determined by the current user.

Example:

Input 2:

7

1, 2, 3, 6, 4, 6, 3 (The elements of the array)

3 (The starting position)

Output: 10

Solutions

C++

```
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;
int sumAbsoluteDifferences(const vector<int>& arr, int pos) {
    int sum = 0;
    for (int i = pos - 1; i < arr.size() - 1; ++i) {
        sum += abs(arr[i + 1] - arr[i]);
    }
    return sum;
}
int main() {
    int n, pos;
    cout << "Enter number of elements: ";
    cin >> n;
    vector<int> arr(n);
    cout << "Enter the elements:\n";
    for (int i = 0; i < n; i++) {
```

```

        cin >> arr[i];
    }
    cout << "Enter the starting position (1-based index): ";
    cin >> pos;
    if (pos < 1 || pos > n) {
        cout << "Invalid input\n";
        return 0;
    }
    int result = sumAbsoluteDifferences(arr, pos);
    cout << "Output: " << result << endl;
    return 0;}

```

Java

```

import java.util.Scanner;

public class AbsoluteDifferenceSum {

    public static int sumAbsoluteDifferences(int[] arr, int pos) {
        int sum = 0;
        for (int i = pos - 1; i < arr.length - 1; i++) {
            sum += Math.abs(arr[i + 1] - arr[i]);
        }
        return sum;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter the elements:");
        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
    }
}

```

```

        System.out.print("Enter the starting position (1-based index): ");
        int pos = sc.nextInt();
        if(pos < 1 || pos > n) {
            System.out.println("Invalid input");
            return;
        }
        int result = sumAbsoluteDifferences(arr, pos);
        System.out.println("Output: " + result);
    }
}

```

Python

```

def sum_absolute_differences(arr, pos):
    n = len(arr)
    if pos < 1 or pos >= n:
        return 0
    total = 0
    for i in range(pos - 1, n - 1):
        total += abs(arr[i + 1] - arr[i])
    return total

# User input
n = int(input("Enter number of elements: "))
arr = list(map(int, input("Enter the elements separated by space: ").split()))
pos = int(input("Enter the starting position (1-based index): "))
if len(arr) != n or pos <= 0 or pos > n:
    print("Invalid input")
else:
    result = sum_absolute_differences(arr, pos)
    print("Output:", result)

```

6.

Input

The first line of the input consists of two space-separated integers- days (M) and products (N), respectively.

The next M lines consist of N space-separated integers - salesRecord, representing the grid of sales revenue (can be positive or negative) received from each product each day.

Output

Print M space-separated integers representing the maximum revenue received each day.

Example

Input:

```
3 4
100 198 333 323
122 232 221 111
223 565 245 764
```

Output:

```
333 232 764
```

Solution

C++

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <climits> // Required for INT_MIN
using namespace std;
int main() {
    int M, N;
    cin >> M >> N;
    for (int i = 0; i < M; i++) {
        int max_val = INT_MIN;
        for (int j = 0; j < N; j++) {
            int sale;
```



```

        cin >> sale;
        if (sale > max_val)
            max_val = sale;
    }
    cout << max_val << " ";
}

return 0;}

```

Java

```

import java.util.Scanner;

public class MaxSalesPerDay {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int M = sc.nextInt();
        int N = sc.nextInt();
        for (int i = 0; i < M; i++) {
            int max = Integer.MIN_VALUE;
            for (int j = 0; j < N; j++) {
                int val = sc.nextInt();
                if (val > max) max = val;
            }
            System.out.print(max + " ");
        }
        sc.close();
    }
}

```

Python

```

# Read M and N
M, N = map(int, input().split())
max_sales = []
for _ in range(M):
    row = list(map(int, input().split()))

```

```
max_sales.append(max(row))  
# Print results  
print(" ".join(map(str, max_sales)))
```

7.

Write a program to calculate and return the sum of distances between the adjacent number in an array of positive integers. Note: You are expected to write code in the findTotalDistance function only which receive the first parameter as the numbers of items in the array and second parameter as the array itself. You are not required to take the input from the console.

Example Finding the total distance between adjacent items of a list of 5 numbers.

Input 1 : 5 input 2 : 10 11 7 12 14

Output 12

Solutions

C++

```
#include <iostream>  
#include <cmath>  
using namespace std;  
int findTotalDistance(int n, int arr[]) {  
    int total = 0;  
    for (int i = 0; i < n - 1; ++i) {  
        total += abs(arr[i+1] - arr[i]);  
    }  
    return total;  
}  
// Sample test in main  
int main() {  
    int arr[] = {10, 11, 7, 12, 14};  
    int n = sizeof(arr) / sizeof(arr[0]);  
    cout << findTotalDistance(n, arr); // Output: 12  
    return 0;  
}
```

Java

```
public class DistanceCalculator {  
    public static int findTotalDistance(int n, int[] arr) {  
        int total = 0;  
        for (int i = 0; i < n - 1; i++) {  
            total += Math.abs(arr[i+1] - arr[i]);  
        }  
        return total;  
    }  
    public static void main(String[] args) {  
        int[] arr = {10, 11, 7, 12, 14};  
        int n = arr.length;  
        System.out.println(findTotalDistance(n, arr)); // Output: 12  
    }  
}
```

Python

```
def findTotalDistance(n, arr):  
    total = 0  
    for i in range(n - 1):  
        total += abs(arr[i+1] - arr[i])  
    return total  
  
# Sample Test  
arr = [10, 11, 7, 12, 14]  
n = len(arr)  
print(findTotalDistance(n, arr)) # Output: 12
```

8.

Write a program that takes an integer as input and calculates the product of its digits.

Sample Input:

345

Sample Output:

60

Explanation:

The problem asks to calculate the product of the digits of the input number. For the input 345, the digits are 3, 4, and 5.

To get the output, we multiply these digits together: $3 \times 4 \times 5 = 12 \times 5 = 60$

Solution

C++

```
#include <iostream>
using namespace std;
int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;
    int product = 1;
    if (num == 0) {
        product = 0;
    } else {
        while (num > 0) {
            int digit = num % 10;
            product *= digit;
            num /= 10;
        }
    }
    cout << product << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;
public class ProductOfDigits {
```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter a number: ");
    int num = sc.nextInt();
    int product = 1;
    if (num == 0) {
        product = 0;
    } else {
        while (num > 0) {
            int digit = num % 10;
            product *= digit;
            num /= 10;
        }
    }
    System.out.println(product);
    sc.close();
}
}

```

Python

```

num = int(input("Enter a number: "))
product = 1
if num == 0:
    product = 0
else:
    while num > 0:
        digit = num % 10
        product *= digit
        num //= 10
print(product)

```

9.

A Cloth merchant has some pieces of cloth of different lengths. He has an order of curtains of length of 12 feet. He has to find how many curtains can be made from these pieces. Length of pieces of cloth is recorded in feet. Note : You are expected to write code in the findTotalCurtains function only which receive the first parameter as the number of items in the array and second parameter as the array itself. You are not required to take the input from the console.

Example

Finding the total curtains from a list of 5 cloth pieces.

Input

input 1 : 5

input 2 : 3 42 60 6 14

Output 9

Explanation

The first parameter 5 is the size of the array. Next is an array of measurements in feet. The total number of curtains is 5 which is calculated as under

3 -> 0

42 -> 3

60 -> 5

6 -> 0

14 -> 1 total = 9

Solutions

C++

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter number of cloth pieces: ";
    cin >> n;
    int length, totalCurtains = 0;
    cout << "Enter lengths in feet:\n";
    for (int i = 0; i < n; i++) {
```

```

        cin >> length;
        totalCurtains += length / 12;
    }
    cout << "Total curtains: " << totalCurtains << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class Curtains {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of cloth pieces: ");
        int n = sc.nextInt();
        int totalCurtains = 0;
        System.out.println("Enter lengths in feet:");
        for (int i = 0; i < n; i++) {
            int length = sc.nextInt();
            totalCurtains += length / 12;
        }
        System.out.println("Total curtains: " + totalCurtains);
        sc.close();
    }
}

```

Python

```

n = int(input("Enter number of cloth pieces: "))
lengths = list(map(int, input("Enter lengths in feet: ").split()))
total_curtains = 0
for length in lengths:
    total_curtains += length // 12
print("Total curtains:", total_curtains)

```

10.

Write a program to count the number of vowels and consonants in a given string.

Example:

Input

Hello World

Output

Vowels: 3

Consonants: 7

Solutions

C++

```
#include <iostream>
#include <cctype>
using namespace std;
int main() {
    string str;
    cout << "Enter a string: ";
    getline(cin, str);
    int vowels = 0, consonants = 0;
    for (char ch : str) {
        ch = tolower(ch);
        if (isalpha(ch)) {
            if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
                vowels++;
            else
                consonants++;
        }
    }
    cout << "Vowels: " << vowels << endl;
```



```
    cout << "Consonants: " << consonants << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;

public class VowelConsonantCounter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String input = sc.nextLine().toLowerCase();
        int vowels = 0, consonants = 0;
        for (char ch : input.toCharArray()) {
            if (Character.isLetter(ch)) {
                if ("aeiou".indexOf(ch) != -1)
                    vowels++;
                else
                    consonants++;
            }
        }
        System.out.println("Vowels: " + vowels);
        System.out.println("Consonants: " + consonants);
        sc.close();
    }
}
```

Python

```
string = input("Enter a string: ").lower()
vowels = consonants = 0
for char in string:
    if char.isalpha():
        if char in 'aeiou':
```

```
        vowels += 1
    else:
        consonants += 1
print("Vowels:", vowels)
print("Consonants:", consonants)
```

11.

Write a program to print the Fibonacci series up to a given number of terms.

Solutions

C++

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter number of terms: ";
    cin >> n;
    int a = 0, b = 1, next;
    for (int i = 1; i <= n; ++i) {
        cout << a << " ";
        next = a + b;
        a = b;
        b = next;
    }
    return 0;
}
```

Java

```
import java.util.Scanner;

public class FibonacciSeries {
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);
System.out.print("Enter number of terms: ");
int n = sc.nextInt();
int a = 0, b = 1, next;
for (int i = 1; i <= n; i++) {
    System.out.print(a + " ");
    next = a + b;
    a = b;
    b = next;}}}

```

Python

```

n = int(input("Enter number of terms: "))
a, b = 0, 1
for _ in range(n):
    print(a, end=' ')
    a, b = b, a + b

```

12.

Write a program to count occurrences of a specific character in a string.

Solutions

C++

```

#include <iostream>
#include <string>
using namespace std;
int main() {
    string str;
    char ch;
    cout << "Enter a string: ";
    getline(cin, str);
    cout << "Enter a character to count: ";
    cin >> ch;

```

```

int count = 0;
for (char c : str) {
    if (c == ch) {
        count++;
    }
}

cout << "Character '" << ch << "' occurs " << count << " times." << endl;
return 0;
}

```

Java

```

import java.util.Scanner;

public class CharCount {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");

        String str = sc.nextLine();

        System.out.print("Enter a character to count: ");

        char ch = sc.next().charAt(0);

        int count = 0;

        for (int i = 0; i < str.length(); i++) {

            if (str.charAt(i) == ch) {

                count++;

            }

        }

        System.out.println("Character '" + ch + "' occurs " + count + " times.");

    }

}

```

Python

```

string = input("Enter a string: ")
ch = input("Enter a character to count: ")

```

```

count = 0
for c in string:
    if c == ch:
        count += 1
print(f"Character '{ch}' occurs {count} times.")

```

13.

Calculate the total interest on loans for an array of amounts. However, till the amount of 2000, there is no interest applicable, but, there is a 20% interest applicable for the remaining amounts in the array.

Example:

If there are 5 amounts with the first input as 5 and the amounts being 2000, 4000, 6000, 8000 and 10000.

The total sum would be = 20% of 4000 + 20% of 6000 + 20% of 8000 + 20% of 10000 = 5600 (as the 2000 will not be considered for interest)

Solutions

C++

```

#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n;
    cout << "Enter number of loan amounts: ";
    cin >> n;
    vector<int> loans(n);
    cout << "Enter the loan amounts:\n";
    for (int i = 0; i < n; ++i) {
        cin >> loans[i];
    }
    double totalInterest = 0.0;
    for (int amount : loans) {

```

```

        if (amount > 2000) {
            totalInterest += (amount ) * 0.20;
        }
    }
    cout << "Total interest: ₹" << totalInterest << endl;
    return 0;}

```

Java

```

import java.util.Scanner;

public class LoanInterest {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of loan amounts: ");

        int n = sc.nextInt();

        int[] loans = new int[n];

        System.out.println("Enter the loan amounts:");

        for (int i = 0; i < n; i++) {

            loans[i] = sc.nextInt();

        }

        double totalInterest = 0.0;

        for (int amount : loans) {

            if (amount > 2000) {

                totalInterest += (amount) * 0.20;

            }

        }

        System.out.println("Total interest: ₹" + totalInterest);

    }
}

```

Python

```

n = int(input("Enter number of loan amounts: "))

loans = []

print("Enter the loan amounts:")

```

```

for _ in range(n):
    loans.append(int(input()))
total_interest = 0.0
for amount in loans:
    if amount > 2000:
        total_interest += (amount) * 0.20
print(f"Total interest: ₹{total_interest}")

```

14.

You are given a list of daily prices of a stock. You can buy a stock on one day and sell it later on another day after the day you bought the stock. You can perform the above operation only once. What is the maximum loss possible?

Example 1

Prices=[10,4,2,9]

The greatest loss is incurred when you buy at a price of 10 and sell at a price of 2. Return the difference:8.

Example 2

Price=[1,2,3,4]

The Price went up every day. Return 0.

Sample Input for Custom Testing

7 → Prices [] size n=7

1 → prices =[1,8,4,2,10,3,2]

8

4

2

10

3

2

Sample Output

8

Explanation

Using zero-based index notation, the correct answer is $a[4]-a[6]=10-2=8$. There is a greater difference between 10 and 1, short selling is not allowed in this problem.

Solutions

C++

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n;
    cout << "Enter number of days: ";
    cin >> n;
    vector<int> prices(n);
    cout << "Enter daily stock prices:\n";
    for (int i = 0; i < n; ++i) {
        cin >> prices[i];
    }
    int maxPrice = prices[0];
    int maxLoss = 0;
    for (int i = 1; i < n; ++i) {
        if (prices[i] < maxPrice) {
            maxLoss = max(maxLoss, maxPrice - prices[i]);
        } else {
            maxPrice = prices[i];
        }
    }
    cout << "Maximum possible loss: " << maxLoss << endl;
    return 0;
}
```

Java


```

import java.util.Scanner;

public class MaxLoss {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of days: ");

        int n = sc.nextInt();

        int[] prices = new int[n];

        System.out.println("Enter daily stock prices:");

        for (int i = 0; i < n; i++) {

            prices[i] = sc.nextInt();

        }

        int maxPrice = prices[0];

        int maxLoss = 0;

        for (int i = 1; i < n; i++) {

            if (prices[i] < maxPrice) {

                maxLoss = Math.max(maxLoss, maxPrice - prices[i]);

            } else {

                maxPrice = prices[i];

            }

        }

        System.out.println("Maximum possible loss: " + maxLoss);

    }

}

```

Python

```

n = int(input("Enter number of days: "))

prices = []

print("Enter daily stock prices:")

for _ in range(n):

    prices.append(int(input()))

max_price = prices[0]

```

```

max_loss = 0
for price in prices[1:]:
    if price < max_price:
        max_loss = max(max_loss, max_price - price)
    else:
        max_price = price
print("Maximum possible loss:", max_loss)

```

15.

A positive integer d is said to be a factor of another positive integer N if when N is divided by d , the remainder obtained is zero. For example, for number 12, there are 6 factors 1, 2, 3, 4, 6, 12. Every positive integer k has at least two factors, 1 and the number k itself. Given two positive integers N and k , write a program to print the k th largest factor of N .

Input Format: The input is a comma-separated list of positive integer pairs (N, k) .

Output Format: The k^{th} highest factor of N . If N does not have k factors, the output should be 1.

Constraints:

- $1 < N < 10000000000$
- $1 < k < 600$.

You can assume that N will have no prime factors which are larger than 13.

Example 1

- Input: 12,3
- Output: 4

Explanation: N is 12, k is 3. The factors of 12 are (1,2,3,4,6,12). The highest factor is 12 and the third largest factor is 4. The output must be 4.

Example 2

- Input: 30,9
- Output: 1

Explanation: N is 30, k is 9. The factors of 30 are (1,2,3,5,6,10,15,30). There are only 8 factors. As k is more than the number of factors, the output is 1.

Solutions

C++

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    long long N;
    int k;
    char comma;
    cout << "Enter input (N,k): ";
    cin >> N >> comma >> k;
    vector<long long> factors;
    for (long long i = 1; i * i <= N; ++i) {
        if (N % i == 0) {
            factors.push_back(i);
            if (i != N / i) {
                factors.push_back(N / i);
            }
        }
    }
    sort(factors.rbegin(), factors.rend());
    if (k <= factors.size()) {
        cout << factors[k - 1] << endl;
    } else {
        cout << 1 << endl;
    }
    return 0;
}
```

Java

```
import java.util.*;
```

```

public class KthLargestFactor {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter input (N,k): ");
        String[] input = sc.nextLine().split(",");
        long N = Long.parseLong(input[0]);
        int k = Integer.parseInt(input[1]);
        List<Long> factors = new ArrayList<>();
        for (long i = 1; i * i <= N; i++) {
            if (N % i == 0) {
                factors.add(i);
                if (i != N / i) {
                    factors.add(N / i);
                }
            }
        }
        factors.sort(Collections.reverseOrder());
        if (k <= factors.size()) {
            System.out.println(factors.get(k - 1));
        } else {
            System.out.println(1);
        }
    }
}

```

Python

```

input_str = input("Enter input (N,k): ")
N_str, k_str = input_str.split(',')
N = int(N_str)
k = int(k_str)
factors = []

```

```

i = 1
while i * i <= N:
    if N % i == 0:
        factors.append(i)
        if i != N // i:
            factors.append(N // i)
    i += 1
factors.sort(reverse=True)
if k <= len(factors):
    print(factors[k - 1])
else:
    print(1)

```

16.

Street Lights are installed at every position along a 1-D road of length n .

`Locations[]` (an array) represent the coverage limit of these lights. The i th light has a coverage limit of `locations[i]` that can range from the position $\max((i - \text{locations}[i]), 1)$ to $\min((i + \text{locations}[i]), n)$ (Closed intervals). Initially all the lights are switched off. Find the minimum number of fountains that must be switched on to cover the road.

Example

$n = 3$

`locations[] = {0, 2, 13}` then

For position 1: `locations[1] = 0`, $\max((1 - 0),$

$1)$ to $\min(1+0), 3)$ gives range = 1 to 1

For position 2: `locations[2] = 2`, $\max((2-2),$

$1)$ to $\min((2+2), 3)$ gives range = 1 to 3

For position 3: `locations[3] = 1`, $\max((3-1),$

$1)$ to $\min((3+1), 3)$ gives range = 2 to 3

For the entire length of this road to be covered, only the light at position 2 needs to be activated.

Function Description

Returns

int: the minimum number of street lights that must be activated

Sample Case 0

Sample Input For Custom Testing

3 ->locations[] size n = 3

1 ->locations[] [1, 1, 1]

1 ->Sample Output

Solutions

C++

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int minLights(int n, vector<int>& locations) {
    vector<pair<int, int>> intervals;
    for (int i = 1; i <= n; ++i) {
        int left = max(1, i - locations[i - 1]);
        int right = min(n, i + locations[i - 1]);
        intervals.push_back({left, right});
    }
    sort(intervals.begin(), intervals.end());
    int res = 0, i = 0, currentEnd = 0, nextEnd = 0;
    while (currentEnd < n) {
        while (i < intervals.size() && intervals[i].first <= currentEnd + 1) {
            nextEnd = max(nextEnd, intervals[i].second);
            ++i;
        }
        if (nextEnd == currentEnd)
            return -1;
        ++res;
        currentEnd = nextEnd;
    }
```

```

        currentEnd = nextEnd;
    }
    return res;
}

int main() {
    int n;
    cout << "Enter road length n: ";
    cin >> n;
    vector<int> locations(n);
    cout << "Enter locations[] array:\n";
    for (int i = 0; i < n; ++i) {
        cin >> locations[i];
    }
    cout << "Minimum street lights to activate: " << minLights(n, locations) << endl;
    return 0;
}

```

Java

```

import java.util.*;

public class StreetLights {
    public static int minLights(int n, int[] locations) {
        List<int[]> intervals = new ArrayList<>();
        for (int i = 1; i <= n; i++) {
            int left = Math.max(1, i - locations[i - 1]);
            int right = Math.min(n, i + locations[i - 1]);
            intervals.add(new int[]{left, right});
        }
        intervals.sort(Comparator.comparingInt(a -> a[0]));
        int res = 0, i = 0, currentEnd = 0, nextEnd = 0;
        while (currentEnd < n) {
            while (i < intervals.size() && intervals.get(i)[0] <= currentEnd + 1) {

```

```

        nextEnd = Math.max(nextEnd, intervals.get(i)[1]);
        i++;
    }
    if (nextEnd == currentEnd)
        return -1;
    res++;
    currentEnd = nextEnd;
}
return res;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter road length n: ");
    int n = sc.nextInt();
    int[] locations = new int[n];
    System.out.println("Enter locations[] array:");
    for (int i = 0; i < n; i++) {
        locations[i] = sc.nextInt();
    }
    System.out.println("Minimum street lights to activate: " + minLights(n,
locations));
}
}

```

Python

```

def min_street_lights_to_activate(n, locations):
    intervals = []
    for i in range(1, n + 1):
        left = max(1, i - locations[i - 1])
        right = min(n, i + locations[i - 1])
        intervals.append((left, right))
    # Sort intervals by start position

```



```

intervals.sort()
res = 0
i = 0
current_end = 0
next_end = 0
while current_end < n:
    # Extend coverage as far as possible with lights starting before or at
    current_end + 1
    while i < len(intervals) and intervals[i][0] <= current_end + 1:
        next_end = max(next_end, intervals[i][1])
        i += 1
    if next_end == current_end:
        # No extension possible
        return -1 # Road cannot be fully covered
    res += 1
    current_end = next_end
return res

# --- INPUT FORMAT ---
n = int(input("Enter road length n: "))
locations = []
print("Enter locations[] array:")
for _ in range(n):
    locations.append(int(input()))
print("Minimum street lights to activate:", min_street_lights_to_activate(n, locations))

```

17.

Nobel Prize-winning physicist Erwin Schrödinger has brought multiple versions of Christopher Columbus from different parallel universes. He puts them at the starting cell (1,1) of a 2D grid of size $n \times m$.

- At position (x, y) in the grid, he places people from America.
- At position (n, m), you and your friends are placed.

All Columbus versions start at (1,1) and try to reach you at (n, m), but:

- If a Columbus reaches the American cell (x, y), he thinks it's India and stops there — he will not go further.
- Only the Columbus who reach (n,m) without ever stepping on (x,y) are considered valid.

You are to determine how many different Columbus versions will reach you at (n,m) without stepping on the American cell (x,y).

Input Format

n → Number of rows in the grid

m → Number of columns in the grid

x → Row position of the American cell

y → Column position of the American cell

Output Format

Return a single integer — the number of valid unique paths from (1,1) to (n,m) that do not go through (x,y).

Explanation

(1,1) (1,2)

(2,1) (2,2)

Blocked cell: (2,1) → the American cell

You must reach (2,2) from (1,1), without going through (2,1)

Possible valid path:

(1,1) → (1,2) → (2,2)

That is 1 valid path.

Solutions

C++

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int markgame(int n, int m, int x, int y) {
```

```
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
```

```
    dp[1][1] = 1;
```

```

for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= m; ++j) {
        if (i == 1 && j == 1) continue;
        if (i == x && j == y) {
            dp[i][j] = 0;
        } else {
            if (i > 1) dp[i][j] += dp[i - 1][j];
            if (j > 1) dp[i][j] += dp[i][j - 1];
        }
    }
}
return dp[n][m];
}

int main() {
    int n, m, x, y;
    cin >> n >> m >> x >> y;
    cout << markgame(n, m, x, y) << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class ParallelColumbus {

    public static int markgame(int n, int m, int x, int y) {
        int[][] dp = new int[n + 1][m + 1];
        dp[1][1] = 1;
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (i == 1 && j == 1) continue;
                if (i == x && j == y) {
                    dp[i][j] = 0;
                }
            }
        }
    }
}

```

```

        } else {
            if (i > 1) dp[i][j] += dp[i - 1][j];
            if (j > 1) dp[i][j] += dp[i][j - 1];
        }
    }
}

return dp[n][m];
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int m = sc.nextInt();
    int x = sc.nextInt();
    int y = sc.nextInt();
    System.out.println(markgame(n, m, x, y));
}
}

```

Python

```

def markgame(n, m, x, y):
    # Initialize 2D dp array
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    # Starting cell
    dp[1][1] = 1
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if i == 1 and j == 1:
                continue
            if i == x and j == y:
                dp[i][j] = 0 # Can't step into American cell
            else:

```

```

        dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
    return dp[n][m]
# Input
n = int(input())
m = int(input())
x = int(input())
y = int(input())
# Output
print(markgame(n, m, x, y))
18.

```

There are N Homeless people in the community and N houses in the community. It will be given in the array (people), the height of the person, and in the array house capacity of the house is given.

The government decided to give homes to people on the basis of the following conditions:

- Priority is given to the people from left to right of the array
- Each person is allotted to a house if and only if the capacity of the house is greater than or equal to the person's height
- Nearby empty Houses are allotted to the person(starting from the extreme left)

You need to find the number of homeless people who have not been allotted any home if the government follows the above conditions. So that government will have an idea of how many people they need to allot homes for next time.

Input Format for Custom Testing:

- The first line contains an integer, N, denoting the number of people and number of houses.
- Each line i of the N subsequent lines (where $0 \leq i \leq N$) contains an integer describing people i.
- Each line i of the N subsequent lines (where $0 \leq i \leq N$) contains an integer describing house i.

Sample Test Cases

Sample Input 1

```

3
4
2

```

7
3
5
10

Sample Output 1

0

Explanation

people=[4,2,7]

house=[3,5,10]

People[0] has more priority , from left to right order in houses 5 is the nearest one which fits for people[0]

people[1]=2 will fit in 3 which is nearer from left

people[2]=7 will fit in remaining house of capacity of 10

So no homeless people left so return 0

Sample Input 2

3
3
8
5
1
9
4

Sample Output 2

2

Solutions

C++

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int countHomeless(int n, vector<int>& people, vector<int>& house) {
```

```
    vector<bool> assigned(n, false);
```

```
    int homeless = 0;
```

```
    for (int i = 0; i < n; ++i) {
```

```
        bool found = false;
```

```
        for (int j = 0; j < n; ++j) {
```

```
            if (!assigned[j] && house[j] >= people[i]) {
```

```
                assigned[j] = true;
```

```

        found = true;
        break;
    }
}
if (!found) homeless++;
}
return homeless;
}
int main() {
    int n;
    cin >> n;
    vector<int> people(n), house(n);
    for (int i = 0; i < n; i++) cin >> people[i];
    for (int i = 0; i < n; i++) cin >> house[i];
    cout << countHomeless(n, people, house) << endl;
    return 0;
}

```

Java

```

import java.util.*;

public class Main {

    public static int countHomeless(int n, int[] people, int[] house) {
        boolean[] assigned = new boolean[n];
        int homeless = 0;
        for (int i = 0; i < n; i++) {
            boolean found = false;
            for (int j = 0; j < n; j++) {
                if (!assigned[j] && house[j] >= people[i]) {
                    assigned[j] = true;
                    found = true;
                    break;
                }
            }
            if (!found) homeless++;
        }
        return homeless;
    }
}

```

```

        }
    }
    if (!found) homeless++;
}
return homeless;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int[] people = new int[n];
    int[] house = new int[n];
    for (int i = 0; i < n; i++) people[i] = sc.nextInt();
    for (int i = 0; i < n; i++) house[i] = sc.nextInt();
    System.out.println(countHomeless(n, people, house));
}
}

```

Python

```

def count_homeless_people(n, people, house):
    assigned = [False] * n
    homeless = 0
    for p in people:
        found = False
        for i in range(n):
            if not assigned[i] and house[i] >= p:
                assigned[i] = True
                found = True
                break
        if not found:
            homeless += 1
    return homeless

```


Sample Input Reading

```
n = int(input())
```

```
people = [int(input()) for _ in range(n)]
```

```
house = [int(input()) for _ in range(n)]
```

```
print(count_homeless_people(n, people, house))
```

19.

Problem Statement

You are given an array of integers representing the votes received by students in a university election.

You need to return a subsequence of votes that forms a majority, i.e., the sum of selected votes is strictly more than half of the total votes.

Conditions:

- Among multiple valid subsequences:
 1. Return the subsequence with the minimum size.
 2. If still multiple, choose the one with the maximum sum.
- The result must be printed in non-increasing order.

A subsequence means picking some (or all) elements in any order, not necessarily contiguous.

Input Format

First line: An integer N, size of the votes array.

Second line: N space-separated integers (votes received by each student).

Output Format

Print the chosen subsequence in non-increasing order (space-separated).

Sample Input 1

5

2 1 3 5 2

Sample Output 1

5 3

Explanation 1

- Total votes = $2 + 1 + 3 + 5 + 2 = 13$

- Majority threshold = $13 / 2 = 6.5 \rightarrow$ need > 6.5 , so need $\text{sum} \geq 7$
- Greedily picking largest values:

Start with [5] $\rightarrow$ sum = 5 (not enough)

Add [3] $\rightarrow$ sum = 8 $\rightarrow$ more than half

This is of size 2 (minimum) and has max sum

Sample Input 2

4

4 4 4 4

Sample Output 2

4 4 4

Solutions

C++

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
vector<int> findMajoritySubsequence(vector<int>& votes) {
```

```
    int total = 0;
```

```
    for (int v : votes) total += v;
```

```
    sort(votes.begin(), votes.end(), greater<int>());
```

```
    vector<int> res;
```

```
    int sum = 0;
```

```
    for (int v : votes) {
```

```
        res.push_back(v);
```

```
        sum += v;
```

```
        if (sum > total / 2.0) break;
```

```
    }
```

```
    return res;
```

```

}
int main() {
    int n;
    cin >> n;
    vector<int> votes(n);
    for (int i = 0; i < n; i++) cin >> votes[i];
    vector<int> result = findMajoritySubsequence(votes);
    for (int v : result) cout << v << " ";
    return 0;
}

```

Java

```

import java.util.*;

public class Main {

    public static List<Integer> findMajoritySubsequence(int[] votes) {
        int total = 0;
        for (int vote : votes) total += vote;
        Arrays.sort(votes);
        List<Integer> res = new ArrayList<>();
        int sum = 0;
        for (int i = votes.length - 1; i >= 0; i--) {
            res.add(votes[i]);
            sum += votes[i];
            if (sum > total / 2.0) break;
        }
        return res;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] votes = new int[n];
    }
}

```

```

        for (int i = 0; i < n; i++) votes[i] = sc.nextInt();
        List<Integer> result = findMajoritySubsequence(votes);
        for (int v : result) System.out.print(v + " ");
    }
}

```

Python

```

def find_majority_subsequence(votes):
    total_votes = sum(votes)
    half = total_votes / 2
    votes.sort(reverse=True)
    result = []
    current_sum = 0
    for vote in votes:
        result.append(vote)
        current_sum += vote
        if current_sum > half:
            break
    return result

# Input Reading
n = int(input())
votes = list(map(int, input().split()))
res = find_majority_subsequence(votes)
print(" ".join(map(str, res)))

```

20.

During the night of the street race, there are N cars on the street, each with a given speed. The police will be alerted if the total number of speed variation pairs exceeds a certain threshold, X .

A pair of cars (i, j) has a speed variation if the absolute difference in their speeds is at least K . The total speed variation count is the number of such pairs where the speed difference is at least K .

You need to determine whether the total speed variation count is greater than the given threshold, X. If it is, the police will be alerted; otherwise, they will not.

Input Format

The first line contains three space-separated integers: N, the number of cars; K, the minimum speed difference for variation; and X, the alert threshold.

The second line contains N space-separated integers representing the speeds of the cars.

Output Format

Print YES if the total speed variation count exceeds X; otherwise, print NO.

Sample Input 1

5 10 3

1 20 30 40 50

Sample Output 1

YES

Explanation 1

- Pairs with difference ≥ 10 :

(1,20), (1,30), (1,40), (1,50)

(20,30), (20,40), (20,50)

(30,40), (30,50)

(40,50)

→ Total 10 pairs → which is greater than $X = 3$

So output: YES

Solutions

C++

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
string policeAlert(int n, int k, long long x, vector<int>& speeds) {
```

```
    sort(speeds.begin(), speeds.end());
```

```

long long count = 0;
int j = 0;
for (int i = 0; i < n; ++i) {
    while (j < n && speeds[j] - speeds[i] < k)
        j++;
    count += n - j;
}
return (count > x) ? "YES" : "NO";
}

int main() {
    int n, k;
    long long x;
    cin >> n >> k >> x;
    vector<int> speeds(n);
    for (int i = 0; i < n; i++) cin >> speeds[i];
    cout << policeAlert(n, k, x, speeds) << endl;
    return 0;
}

```

Java

```

import java.util.*;

public class Main {

    public static String policeAlert(int[] speeds, int n, int k, long x) {
        Arrays.sort(speeds);
        long count = 0;
        int j = 0;
        for (int i = 0; i < n; i++) {
            while (j < n && speeds[j] - speeds[i] < k)
                j++;
            count += n - j;
        }
    }
}

```

```

        return (count > x) ? "YES" : "NO";
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), k = sc.nextInt();
        long x = sc.nextLong();
        int[] speeds = new int[n];
        for (int i = 0; i < n; i++) speeds[i] = sc.nextInt();
        System.out.println(policeAlert(speeds, n, k, x));
    }
}

```

Python

```

def will_police_be_alerted(n, k, x, speeds):
    speeds.sort()
    count = 0
    j = 0
    for i in range(n):
        while j < n and speeds[j] - speeds[i] < k:
            j += 1
        count += n - j
    return "YES" if count > x else "NO"

# Input
n, k, x = map(int, input().split())
speeds = list(map(int, input().split()))
print(will_police_be_alerted(n, k, x, speeds))

```

-----TCS-----

1. Majority Element

In a kingdom's council, each noble gives their opinion by submitting a token with their vote. After all votes are collected, the king wants to know if there's any opinion (element) that was supported by **more than $N/3$ nobles**. Help the king identify such majority elements.

Sample Input

Enter space-separated elements: 1 2 3 2 2 4 2

Sample Output

2

Explanation of Output

There are 7 elements in total.

Threshold for majority is $N / 3 = 7 / 3 = 2.33 \approx 2$ (floor value).

Element 2 appears **4 times**, which is **> 2**, so it is printed.

Solutions

C++

```
#include <iostream>
#include <unordered_map>
#include <sstream>
using namespace std;
int main() {
    string line;
    getline(cin, line);
    stringstream ss(line);
    int num, count = 0;
    unordered_map<int, int> freq;
    while (ss >> num) {
        freq[num]++;
        count++;
    }
    bool found = false;
    for (auto &p : freq) {
        if (p.second > count / 3) {
```



```

        cout << p.first << " ";
        found = true;
    }
}
if (!found) cout << "No Majority";
return 0;
}

```

Java

```

import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        String[] input = sc.nextLine().split("\\s+");
        Map<Integer, Integer> freq = new HashMap<>();
        int N = input.length;
        for (String s : input) {
            int num = Integer.parseInt(s);
            freq.put(num, freq.getDefault(num, 0) + 1);
        }
        boolean found = false;
        for (int key : freq.keySet()) {
            if (freq.get(key) > N / 3) {
                System.out.print(key + " ");
                found = true;
            }
        }
        if (!found) {
            System.out.print("No Majority");
        }
    }
}

```

```
}
```

Python

```
from collections import defaultdict
arr = list(map(int, input().split()))
N = len(arr)
freq = defaultdict(int)
for num in arr:
    freq[num] += 1
found = False
for key in freq:
    if freq[key] > N // 3:
        print(key, end=" ")
        found = True
if not found:
    print("No Majority")
```

2.

A mysterious machine processes only three numbers: 3, 6, and 7. These numbers are placed on a conveyor belt in a random order. The machine's job is to sort them using only swaps such that all 3s come first, followed by 6s, and then all 7s at the end — similar to separating colored balls. Your job is to simulate the machine's sorting process.

Sample Input

3 7 6 7 3 6 3

Sample Output

3 3 3 6 6 7 7

Explanation

We rearranged the numbers using a 3-way partitioning (like Dutch National Flag algorithm):

- All 3s are moved to the left,
- All 6s stay in the middle,
- All 7s are moved to the right.

Solutions

C++

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<char> arr;
    char x;
    while (cin >> x) {
        arr.push_back(x);
        if (cin.peek() == '\n') break;
    }
    int left = 0, mid = 0, right = arr.size() - 1;
    while (mid <= right) {
        if (arr[mid] == '3') {
            swap(arr[left++], arr[mid++]);
        } else if (arr[mid] == '6') {
            mid++;
        } else if (arr[mid] == '7') {
            swap(arr[mid], arr[right--]);
        } else {
            mid++;
        }
    }
    for (char c : arr)
        cout << c << " ";
    return 0;
}
```

Java

```

import java.util.*;

public class SortBySwap {

    public static List<Character> sortBySwap(char[] arr, int N) {

        int left = 0, mid = 0, right = N - 1;

        while (mid <= right) {

            if (arr[mid] == '3') {

                char temp = arr[left];

                arr[left] = arr[mid];

                arr[mid] = temp;

                left++;

                mid++;

            } else if (arr[mid] == '6') {

                mid++;

            } else if (arr[mid] == '7') {

                char temp = arr[right];

                arr[right] = arr[mid];

                arr[mid] = temp;

                right--;

            } else {

                mid++; // ignore any invalid chars

            }

        }

        List<Character> result = new ArrayList<>();

        for (char c : arr) {

            result.add(c);

        }

        return result;

    }

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

```

```

System.out.print("Enter space-separated numbers (3, 6, 7 only): ");
String[] input = scanner.nextLine().split("\\s+");
char[] arr = new char[input.length];
for (int i = 0; i < input.length; i++) {
    arr[i] = input[i].charAt(0);
}
List<Character> sortedArr = sortBySwap(arr, arr.length);
for (char c : sortedArr) {
    System.out.print(c + " ");
}
}

```

Python

```

arr = input("Enter space-separated numbers (3, 6, 7 only): ").split()
left, mid, right = 0, 0, len(arr) - 1
while mid <= right:
    if arr[mid] == '3':
        arr[left], arr[mid] = arr[mid], arr[left]
        left += 1
        mid += 1
    elif arr[mid] == '6':
        mid += 1
    elif arr[mid] == '7':
        arr[mid], arr[right] = arr[right], arr[mid]
        right -= 1
    else:
        mid += 1
print(" ".join(arr))

```

3.

A group of children stood in a line for a roll call. They were supposed to be numbered from 1 to N, but one of them forgot to bring their number tag. You are given the tags of the remaining N-1 children. Your task is to find the missing number so the teacher can correctly identify who's missing.

Sample Input

Enter count N: 5

Enter space-separated elements: 1 2 4 5

Sample Output

Missing number: 3

Explanation

The list should have numbers from 1 to 5.

Sum of $1+2+3+4+5 = 15$

Sum of given numbers = $1+2+4+5 = 12$

Missing number = $15 - 12 = 3$

Solutions

C++

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int N;
    cout << "Enter total count N (including missing number): ";
    cin >> N;
    cout << "Enter " << N - 1 << " space-separated numbers: ";
    vector<int> arr(N - 1);
    int sum = 0;
    for (int i = 0; i < N - 1; ++i) {
        cin >> arr[i];
        sum += arr[i];
    }
    int actualSum = (N * (N + 1)) / 2;
```

```

        cout << "Missing number: " << actualSum - sum << endl;
        return 0;
    }

```

Java

```

import java.util.*;

public class MissingNumberFinder {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        // Input total count including the missing number
        System.out.print("Enter total count N (including the missing number): ");
        int N = scanner.nextInt();
        // Input the N-1 numbers
        System.out.print("Enter " + (N - 1) + " space-separated numbers: ");
        int sum = 0;
        for (int i = 0; i < N - 1; i++) {
            int num = scanner.nextInt();
            sum += num;
        }
        // Calculate the actual sum from 1 to N
        int total = N * (N + 1) / 2;

        // Output the missing number
        System.out.println("Missing number: " + (total - sum));
    }
}

```

Python

```

N = int(input("Enter total count N (including missing number): "))
arr = list(map(int, input(f"Enter {N - 1} space-separated numbers: ").split()))
actual_sum = (N * (N + 1)) // 2
given_sum = sum(arr)

```

```
print("Missing number:", actual_sum - given_sum)
```

4.

You are tasked with finding all numbers in a given range [n, m] that are prime and have a special property: the sum of their digits is also a prime number. For example, the number 23 is prime, and its digits sum to 5 (2+3), which is also prime. Your job is to print all such numbers between the given input range.

Sample Input:

10 30

Sample Output:

11

23

29

Explanation:

- 11 is prime, and sum of digits $1 + 1 = 2$ which is prime.
- 23 is prime, and sum of digits $2 + 3 = 5$ which is prime.
- 29 is prime, and sum of digits $2 + 9 = 11$ which is prime.
- Other primes like 13 or 17 are not printed because their digit sums (4 and 8 respectively) are not prime.

Solutions

C++

```
#include <iostream>
using namespace std;
bool isPrime(int number) {
    if (number <= 1) return false;
    if (number <= 3) return true;
    if (number % 2 == 0 || number % 3 == 0) return false;
    for (int i = 5; i * i <= number; i += 6) {
        if (number % i == 0 || number % (i + 2) == 0) return false;
    }
    return true;
}
```



```

}
bool sumIsPrime(int n) {
    int sum = 0;
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return isPrime(sum);
}
int main() {
    int n, m;
    cin >> n >> m;
    for (int i = n; i <= m; i++) {
        if (isPrime(i) && sumIsPrime(i)) {
            cout << i << "\n";
        }
    }
    return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {

    public static boolean isPrime(int number) {
        if (number <= 1) return false;
        if (number <= 3) return true;
        if (number % 2 == 0 || number % 3 == 0) return false;
        int i = 5;
        while (i * i <= number) {
            if (number % i == 0 || number % (i + 2) == 0) return false;
            i += 6;
        }
        return true;
    }
}

```

```

    }
    return true;
}

public static boolean sumIsPrime(int n) {
    int sum = 0;
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return isPrime(sum);
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    int n = scanner.nextInt();
    int m = scanner.nextInt();
    for (int i = n; i <= m; i++) {
        if (isPrime(i) && sumIsPrime(i)) {
            System.out.println(i);
        }
    }
    scanner.close();
}}

```

Python

```

def is_prime(number):
    if number <= 1:
        return False
    if number <= 3:
        return True
    if number % 2 == 0 or number % 3 == 0:
        return False

```

```

i = 5
while i * i <= number:
    if number % i == 0 or number % (i + 2) == 0:
        return False
    i += 6
return True
def sum_is_prime(n):
    s = 0
    while n > 0:
        s += n % 10
        n //= 10
    return is_prime(s)
n, m = map(int, input().split())
for i in range(n, m + 1):
    if is_prime(i) and sum_is_prime(i):
        print(i)

```

5. Fizz Buzz

You have a list of integers. For each integer, you need to print a special word depending on its divisibility: print "ThreeFive" if the number is divisible by both 3 and 5, print "Five" if divisible only by 5, print "Three" if divisible only by 3, otherwise print the number itself.

Sample Input:

```

5
[1 3 5 15 16]

```

Sample Output:

```

1 Three Five ThreeFive 16

```

Solutions

C++

```

#include <iostream>

```

```

using namespace std;
int main() {
    int n; cin >> n;
    for (int i = 0; i < n; i++) {
        int num; cin >> num;
        if (num % 3 == 0 && num % 5 == 0)
            cout << "ThreeFive ";
        else if (num % 5 == 0)
            cout << "Five ";
        else if (num % 3 == 0)
            cout << "Three ";
        else
            cout << num << " ";
    }
    cout << "\n";
    return 0;}

```

Java

```

import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for(int i = 0; i < n; i++) {
            int num = sc.nextInt();
            if(num % 3 == 0 && num % 5 == 0)
                System.out.print("ThreeFive ");
            else if(num % 5 == 0)
                System.out.print("Five ");
            else if(num % 3 == 0)

```

```

        System.out.print("Three ");
    else
        System.out.print(num + " ");
    }
    sc.close();
}
}

```

Python

```

n = int(input())
arr = list(map(int, input().split()))
for num in arr:
    if num % 3 == 0 and num % 5 == 0:
        print("ThreeFive", end=" ")
    elif num % 5 == 0:
        print("Five", end=" ")
    elif num % 3 == 0:
        print("Three", end=" ")
    else:
        print(num, end=" ")
print()

```

6.

You have a list of students with their marks. Your task is to print the top **K** students ranked by their marks in descending order. If there are fewer than K students, print all.

Sample Input:

```

3 5
85 Alice
92 Bob
78 Charlie
90 David

```

88 Eva

Sample Output:

Bob: 92

David: 90

Alice: 85

Explanation:

- We want top 3 students (K=3) out of 5.
- Students sorted by marks: Bob (92), David (90), Alice (85), Eva (88), Charlie (78).
- Top 3 printed in order: Bob, David, Alice.

Solutions

C++

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int main() {
```

```
    int k, N;
```

```
    cin >> k >> N;
```

```
    vector<pair<int, string>> students;
```

```
    for (int i = 0; i < N; i++) {
```

```
        int marks; string name;
```

```
        cin >> marks >> name;
```

```
        students.push_back({marks, name});
```

```
    }
```

```
    sort(students.begin(), students.end(), [](auto &a, auto &b) {
```

```
        return a.first > b.first; // descending by marks
```

```
    });
```

```
    for (int i = 0; i < k && i < (int)students.size(); i++) {
```

```

        cout << students[i].second << ": " << students[i].first << "\n";
    }
    return 0;
}

```

Java

```

import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int k = sc.nextInt();
        int N = sc.nextInt();
        List<Student> students = new ArrayList<>();
        for (int i = 0; i < N; i++) {
            int marks = sc.nextInt();
            String name = sc.next();
            students.add(new Student(marks, name));
        }
        sc.close();
        Collections.sort(students, (a, b) -> b.marks - a.marks);
        for (int i = 0; i < k && i < students.size(); i++) {
            System.out.println(students.get(i).name + ": " + students.get(i).marks);
        }
    }
}

class Student {
    int marks;
    String name;
    Student(int marks, String name) {
        this.marks = marks;
        this.name = name;
    }
}

```

```
}  
}
```

Python

```
k, N = map(int, input().split())  
students = []  
for _ in range(N):  
    marks, name = input().split()  
    students.append((int(marks), name))  
students.sort(key=lambda x: x[0], reverse=True)  
for i in range(min(k, len(students))):  
    print(f"{students[i][1]}: {students[i][0]}")
```

7.

Given a list of numbers, find and print all Armstrong numbers from the list. An Armstrong number is a number that is equal to the sum of its own digits each raised to the power of the number of digits. For example, 153 is an Armstrong number because $1^3 + 5^3 + 3^3 = 153$.

Solution

C++

```
#include <iostream>  
#include <vector>  
#include <cmath>  
#include <sstream>  
using namespace std;  
bool isArmstrong(int num) {  
    int sum = 0, temp = num;  
    int digits = to_string(num).length();  
    while (temp > 0) {  
        int d = temp % 10;
```



```

        sum += pow(d, digits);
        temp /= 10;
    }
    return sum == num;
}

int main() {
    string line;
    getline(cin, line);
    stringstream ss(line);
    string token;
    bool found = false;
    while (ss >> token) {
        int num = stoi(token);
        if (isArmstrong(num)) {
            cout << num << " ";
            found = true;
        }
    }
    if (!found)
        cout << "No Armstrong numbers present";
    cout << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {

    public static boolean isArmstrong(int num) {
        int sum = 0, temp = num;
        int digits = String.valueOf(num).length();
        while (temp > 0) {

```

```

        int digit = temp % 10;
        sum += Math.pow(digit, digits);
        temp /= 10;
    }
    return sum == num;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String line = sc.nextLine();
    String[] numbers = line.trim().split("\\s+");
    boolean found = false;
    for (String s : numbers) {
        int num = Integer.parseInt(s);
        if (isArmstrong(num)) {
            System.out.print(num + " ");
            found = true;
        }
    }
    if (!found)
        System.out.print("No Armstrong numbers present");
    sc.close();
}
}

```

Python

```

def is_armstrong(num):
    digits = len(str(num))
    return num == sum(int(d)**digits for d in str(num))

nums = input().split()
found = False
for n in nums:

```

```

num = int(n)
if is_armstrong(num):
    print(num, end=" ")
    found = True
if not found:
    print("No Armstrong numbers present")

```

8.

You are given a list of integers. Your task is to find the sum of all elements that appear exactly once in the list (i.e., unique elements).

Sample Input:

1 2 2 3 4 4 5

Sample Output:

9

Explanation:

- Unique elements (appear only once) are: 1, 3, 5
- Their sum = 1 + 3 + 5 = 9

Solutions

C++

```

#include <iostream>
#include <unordered_map>
#include <sstream>
using namespace std;
int main() {
    unordered_map<int,int> freq;
    int num;
    string line;
    getline(cin, line);
    istringstream ss(line);
    while (ss >> num) freq[num]++;
    int sum = 0;

```

```

for (auto &p : freq) {
    if (p.second == 1) sum += p.first;
}
cout << sum << "\n";
return 0;
}

```

Java

```

import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        String[] strNums = sc.nextLine().split(" ");
        Map<Integer, Integer> freq = new HashMap<>();
        for (String s : strNums) {
            int num = Integer.parseInt(s);
            freq.put(num, freq.getDefault(num, 0) + 1);
        }
        int sum = 0;
        for (int key : freq.keySet()) {
            if (freq.get(key) == 1)
                sum += key;
        }
        System.out.println(sum);
        sc.close();
    }
}

```

Python

```

nums = list(map(int, input().split()))
freq = {}
for num in nums:

```

```
freq[num] = freq.get(num, 0) + 1
sum_unique = sum(k for k, v in freq.items() if v == 1)
print(sum_unique)
```

9.

You want to calculate the total shipping cost of a package based on its weight (in kg) and distance (in km). The cost calculation is:

- Base cost: \$5.00
- Cost per kg: \$2.00 per kg
- Cost per 10 km: \$0.50 per 10 km

Calculate and print the total cost in dollars with 2 decimal places.

Sample Input:

10 25

Sample Output:

\$26.25

Explanation:

- Weight cost = $10 * 2.00 = \$20.00$
- Distance cost = $(25 / 10) * 0.50 = 2.5 * 0.50 = \1.25
- Total = $5.00 + 20.00 + 1.25 = \$26.25$

Solutions

C++

```
#include <iostream>
#include <iomanip>
using namespace std;
double calculateTotalCost(int weight, int distance) {
    double base = 5.0;
    double costKg = 2.0;
    double costPer10Km = 0.5;
    double weightCost = weight * costKg;
    double distanceCost = (distance / 10.0) * costPer10Km;
```

```

        return base + weightCost + distanceCost;
    }

    int main() {
        int weight, distance;
        cin >> weight >> distance;
        double total = calculateTotalCost(weight, distance);
        cout << fixed << setprecision(2) << "$" << total << "\n";
        return 0;
    }

```

Java

```

import java.util.Scanner;

public class Main {

    public static double calculateTotalCost(int weight, int distance) {
        double base = 5.0;
        double costKg = 2.0;
        double costPer10Km = 0.5;
        double weightCost = weight * costKg;
        double distanceCost = (distance / 10.0) * costPer10Km;
        return base + weightCost + distanceCost;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int weight = sc.nextInt();
        int distance = sc.nextInt();
        double total = calculateTotalCost(weight, distance);
        System.out.printf("$%.2f\n", total);
        sc.close();
    }
}

```

Python

```
def calculate_total_cost(weight, distance):
    base = 5.0
    cost_kg = 2.0
    cost_per_10km = 0.5
    weight_cost = weight * cost_kg
    distance_cost = (distance / 10) * cost_per_10km
    return base + weight_cost + distance_cost

w, d = map(int, input().split())
total = calculate_total_cost(w, d)
print(f"${total:.2f}")
```

10.

Given an array of integers and a target sum, find and print all continuous subarrays whose elements sum up exactly to the target value.

Sample Input:

```
arr = [3, 4, -7, 1, 3, 3, 1, -4]
```

```
target = 7
```

Sample Output:

```
3 4
```

```
1 3 3
```

```
3 3 1
```

Explanation:

- Subarrays [3, 4], [1, 3, 3], and [3, 3, 1] all sum to 7.
- The program finds and prints all such subarrays.

Solution

C++

```
#include <iostream>
#include <unordered_map>
#include <vector>
using namespace std;
```

```

void findSubarrays(int arr[], int n, int target) {
    unordered_map<int, int> sumMap;
    int curSum = 0;
    for (int i = 0; i < n; i++) {
        curSum += arr[i];
        if (curSum == target) {
            for (int j = 0; j <= i; j++)
                cout << arr[j] << " ";
            cout << "\n";
        }
        if (sumMap.find(curSum - target) != sumMap.end()) {
            int start = sumMap[curSum - target] + 1;
            for (int k = start; k <= i; k++)
                cout << arr[k] << " ";
            cout << "\n";
        }
        sumMap[curSum] = i;
    }
}

int main() {
    int arr[] = {3, 4, -7, 1, 3, 3, 1, -4};
    int target = 7;
    int n = sizeof(arr) / sizeof(arr[0]);
    findSubarrays(arr, n, target);
    return 0;
}

```

Java

```

import java.util.HashMap;

public class Main {

    static void optimalApproach(int[] arr, int n, int target) {
        HashMap<Integer, Integer> sumMap = new HashMap<>();
    }
}

```



```

int curSum = 0;
for (int i = 0; i < n; i++) {
    curSum += arr[i];

    if (curSum == target) {
        for (int j = 0; j <= i; j++) {
            System.out.print(arr[j] + " ");
        }
        System.out.println();
    }

    if (sumMap.containsKey(curSum - target)) {
        int startIndex = sumMap.get(curSum - target) + 1;
        for (int k = startIndex; k <= i; k++) {
            System.out.print(arr[k] + " ");
        }
        System.out.println();
    }
    sumMap.put(curSum, i);
}
}

public static void main(String[] args) {
    int[] arr = {3, 4, -7, 1, 3, 3, 1, -4};
    int target = 7;
    optimalApproach(arr, arr.length, target);
}
}

```

Python

```

def find_subarrays(arr, target):
    sum_map = {}
    cur_sum = 0

```

```

for i, val in enumerate(arr):
    cur_sum += val
    if cur_sum == target:
        print(*arr[:i+1])
    if (cur_sum - target) in sum_map:
        start = sum_map[cur_sum - target] + 1
        print(*arr[start:i+1])
    sum_map[cur_sum] = i
arr = [3, 4, -7, 1, 3, 3, 1, -4]
target = 7
find_subarrays(arr, target)

```

11.

You are standing at the top-left corner of an $m \times n$ grid. You want to reach the bottom-right corner by moving only down or right at any point in time. How many unique paths are there to reach the destination?

Sample Input:

$m = 3, n = 3$

Sample Output:

6

Explanation:

For a 3×3 grid, the possible unique paths to go from top-left to bottom-right by moving only down or right are 6.

Solutions

C++

```

#include <iostream>
#include <vector>
using namespace std;
int f(int i, int j, vector<vector<int>> &dp) {

```

```

    if (i == 0 && j == 0) return 1;
    if (i < 0 || j < 0) return 0;
    if (dp[i][j] != -1) return dp[i][j];

    int up = f(i - 1, j, dp);
    int left = f(i, j - 1, dp);
    dp[i][j] = up + left;
    return dp[i][j];
}

int uniquePaths(int m, int n) {
    vector<vector<int>> dp(m, vector<int>(n, -1));
    return f(m - 1, n - 1, dp);
}

int main() {
    int m, n;
    cin >> m >> n;
    cout << uniquePaths(m, n) << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class Solution {

    public int f(int i, int j, int[][] dp) {
        if (i == 0 && j == 0) return 1;
        if (i < 0 || j < 0) return 0;
        if (dp[i][j] != -1) return dp[i][j];

        int up = f(i - 1, j, dp);
        int left = f(i, j - 1, dp);
        dp[i][j] = up + left;
        return dp[i][j];
    }
}

```

```

    }

    public int uniquePaths(int m, int n) {
        int[][] dp = new int[m][n];
        for (int i = 0; i < m; i++)
            for (int j = 0; j < n; j++)
                dp[i][j] = -1;
        return f(m - 1, n - 1, dp);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int m = sc.nextInt();
        int n = sc.nextInt();
        sc.close();
        Solution sol = new Solution();
        System.out.println(sol.uniquePaths(m, n));
    }
}

```

Python

```

def f(i, j, dp):
    if i == 0 and j == 0:
        return 1
    if i < 0 or j < 0:
        return 0
    if dp[i][j] != -1:
        return dp[i][j]
    up = f(i - 1, j, dp)
    left = f(i, j - 1, dp)
    dp[i][j] = up + left
    return dp[i][j]

def uniquePaths(m, n):

```

```

    dp = [[-1 for _ in range(n)] for _ in range(m)]
    return f(m - 1, n - 1, dp)
m, n = map(int, input().split())
print(uniquePaths(m, n))

```

12.

A teacher wants to teach kids about cube numbers. She gives them two integers and asks them to find the sum of cubes of all the numbers between those two values, inclusive. Help the kids solve this task!

Sample Input

1 3

Sample Output

36

Explanation

The cubes of numbers from 1 to 3 are:

- $1=1^3=1$
- $2=2^3=8$
- $3=3^3=27$

Sum = $1 + 8 + 27 = 36$

Solution

C++

```

#include <iostream>
using namespace std;
int main() {
    int start, end;
    cin >> start >> end;
    int sum = 0;
    for (int i = start; i <= end; i++) {
        sum += i * i * i;
    }
}

```

```

        cout << sum << endl;
        return 0;
    }
Java
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int start = scanner.nextInt();
        int end = scanner.nextInt();
        int sum = 0;
        for (int i = start; i <= end; i++) {
            sum += i * i * i;
        }
        System.out.println(sum);
        scanner.close();
    }
}

```

Python

```

start, end = map(int, input().split())
cube_sum = sum(i**3 for i in range(start, end + 1))
print(cube_sum)

```

13.

A store tracks the sales of items. For each sale, the item name, quantity, and price per unit are recorded.

Your task is to:

1. Find the most sold item (in terms of total revenue).
2. Calculate total sales revenue.
3. Calculate average revenue per entry.

Sample Input

5
apple 2 100
banana 3 50
apple 1 100
banana 1 50
cherry 5 20

Sample Output

Most Sold Item: apple
Total Revenue: 600.00
Average Revenue Per Entry: 120.00

Solutions

C++

```
#include <iostream>
#include <unordered_map>
#include <string>
using namespace std;
int main() {
    int n;
    cin >> n;
    unordered_map<string, double> sales;
    double totalRevenue = 0;
    for (int i = 0; i < n; ++i) {
        string item;
        int qty, price;
        cin >> item >> qty >> price;
        double sale = qty * price;
        sales[item] += sale;
        totalRevenue += sale;
    }
```

```

    }
    string topItem;
    double maxSale = 0;
    for (auto& pair : sales) {
        if (pair.second > maxSale) {
            maxSale = pair.second;
            topItem = pair.first;
        }
    }
    double average = totalRevenue / n;
    cout << "Most Sold Item: " << topItem << endl;
    printf("Total Revenue: %.2f\n", totalRevenue);
    printf("Average Revenue Per Entry: %.2f\n", average);
    return 0;
}

```

Java

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();
        Map<String, Double> sales = new HashMap<>();
        double totalRevenue = 0;
        for (int i = 0; i < n; i++) {
            String item = scanner.next();
            int qty = scanner.nextInt();
            int price = scanner.nextInt();
            double sale = qty * price;
            sales.put(item, sales.getOrDefault(item, 0.0) + sale);
            totalRevenue += sale;
        }
    }
}

```



```

    }
    String topItem = "";
    double maxSale = 0;
    for (Map.Entry<String, Double> entry : sales.entrySet()) {
        if (entry.getValue() > maxSale) {
            maxSale = entry.getValue();
            topItem = entry.getKey();
        }
    }
    double average = totalRevenue / n;
    System.out.println("Most Sold Item: " + topItem);
    System.out.printf("Total Revenue: %.2f\n", totalRevenue);
    System.out.printf("Average Revenue Per Entry: %.2f\n", average);
    scanner.close();
}
}

```

Python

```

n = int(input())
sales = {}
total_revenue = 0
for _ in range(n):
    item, qty, price = input().split()
    qty = int(qty)
    price = int(price)
    sale = qty * price
    sales[item] = sales.get(item, 0) + sale
    total_revenue += sale
top_item = max(sales, key=sales.get)
average = total_revenue / n
print(f"Most Sold Item: {top_item}")

```

```
print(f"Total Revenue: {total_revenue:.2f}")
print(f"Average Revenue Per Entry: {average:.2f}")
```

14.

Write a program that takes a number n as input and prints its multiplication table from 1 to 10. Additionally, calculate and print the sum of all the products in the table.

Solutions

C++

```
#include <iostream>
using namespace std;
int main() {
    int n, sum = 0;
    cin >> n;
    for (int i = 1; i <= 10; i++) {
        int product = n * i;
        cout << n << " x " << i << " = " << product << endl;
        sum += product;
    }
    cout << "Sum = " << sum << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt(); // Input number
        int sum = 0;
        for (int i = 1; i <= 10; i++) {
            int product = n * i;
            System.out.println(n + " x " + i + " = " + product);
        }
    }
}
```

```

        sum += product;
    }
    System.out.println("Sum = " + sum);
    scanner.close();
}
}

```

Python

```

n = int(input())
total = 0
for i in range(1, 11):
    product = n * i
    print(f"{n} x {i} = {product}")
    total += product
print(f"Sum = {total}")

```

15.

Given an array of integers, find how many distinct values can be obtained by computing the bitwise OR of all possible contiguous subarrays.

- The bitwise OR of a subarray is the result of ORing all elements in that subarray.
- You need to return the count of unique results.

Input Format:

- First line: an integer n – the size of the array.
- Second line: n integers – the array elements.

Output Format:

- A single integer: the number of unique bitwise OR values from all subarrays.

Sample Input:

3

1 2 4

Sample Output:

6

Explanation:

All subarrays and their ORs:

- [1] = 1
- [1,2] = 1 | 2 = 3
- [1,2,4] = 1 | 2 | 4 = 7
- [2] = 2
- [2,4] = 2 | 4 = 6
- [4] = 4

Unique OR values: 1, 2, 3, 4, 6, 7 → Count = 6

Solutions

C++

```
#include <iostream>
#include <set>
using namespace std;
int main() {
    int n;
    cin >> n;
    int arr[n];
    for(int i = 0; i < n; i++) cin >> arr[i];
    set<int> orSet;
    for(int i = 0; i < n; i++) {
        int orVal = 0;
        for(int j = i; j < n; j++) {
            orVal |= arr[j];
            orSet.insert(orVal);
        }
    }
    cout << orSet.size() << endl;
```

```

        return 0;
    }

Java

import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for(int i = 0; i < n; i++) arr[i] = sc.nextInt();
        Set<Integer> orSet = new HashSet<>();
        for (int i = 0; i < n; i++) {
            int or = 0;
            for (int j = i; j < n; j++) {
                or |= arr[j];
                orSet.add(or);
            }
        }

        System.out.println(orSet.size());
        sc.close();
    }
}

```

Python

```

n = int(input())
arr = list(map(int, input().split()))
or_set = set()
for i in range(n):
    or_val = 0
    for j in range(i, n):

```

```
    or_val |= arr[j]
    or_set.add(or_val)
print(len(or_set))
```

16. Maximum Efficiency

Given an array of integers (positive), find the maximum product of any three numbers. This helps determine maximum team efficiency when selecting 3 employees (numbers can represent productivity which is positive).

Solutions

C++

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<int> arr(n);
    bool hasNegative = false
    for (int i = 0; i < n; ++i) {
        cin >> arr[i];
        if (arr[i] < 0)
            hasNegative = true;
    }
    if (hasNegative) {
        cout << "Negative input not allowed" << endl;
        return 0;
    }
    sort(arr.begin(), arr.end());
    int result = arr[n - 1] * arr[n - 2] * arr[n - 3];
    cout << result << endl;
```

```
    return 0;
}
```

Java

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int N = scanner.nextInt();
        int[] arr = new int[N];
        boolean hasNegative = false;
        for (int i = 0; i < N; ++i) {
            arr[i] = scanner.nextInt();
            if (arr[i] < 0) {
                hasNegative = true;
            }
        }
        if (hasNegative) {
            System.out.println("Negative input not allowed");
            return;
        }
        Arrays.sort(arr);
        int result = arr[N - 1] * arr[N - 2] * arr[N - 3];
        System.out.println(result);
    }
}
```

Python

```
def max_efficiency(arr):
    if any(x < 0 for x in arr):
        print("Negative input not allowed")
    return
```

```
arr.sort()
print(arr[-1] * arr[-2] * arr[-3])
n = int(input())
arr = list(map(int, input().split()))
max_efficiency(arr)
```

17.

You are given a list of shoes where each shoe is represented by a size and a side (L or R). Count how many complete pairs of shoes you can make. A valid pair consists of one left shoe and one right shoe of the same size.

Solutions

C++

```
#include <iostream>
#include <unordered_map>
#include <algorithm>
using namespace std;
int main() {
    int n;
    cin >> n;
    unordered_map<string, int> count;

    for (int i = 0; i < n; ++i) {
        string shoe;
        cin >> shoe;
        count[shoe]++;
    }
    int pairs = 0;
    for (auto &entry : count) {
        string key = entry.first;
        if (key.back() == 'L') {
```



```

        string opposite = key.substr(0, key.size() - 1) + 'R';
        if (count.count(opposite)) {
            pairs += min(entry.second, count[opposite]);
        }
    }
}

cout << pairs << endl;
return 0;
}

```

Java

```

import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        int n = scanner.nextInt();

        Map<String, Integer> count = new HashMap<>();

        for (int i = 0; i < n; i++) {

            String shoe = scanner.next();

            count.put(shoe, count.getDefault(shoe, 0) + 1);

        }

        int pairs = 0;

        for (String key : count.keySet()) {

            if (key.endsWith("L")) {

                String opposite = key.substring(0, key.length() - 1) + "R";

                if (count.containsKey(opposite)) {

                    pairs += Math.min(count.get(key), count.get(opposite));

                }

            }

        }

    }
}

```

```

        System.out.println(pairs);
    }
}
Python
n = int(input())
shoes = input().split()
count = {}
for shoe in shoes:
    count[shoe] = count.get(shoe, 0) + 1
pairs = 0
for key in count:
    if key.endswith('L'):
        opposite = key[:-1] + 'R'
        if opposite in count:
            pairs += min(count[key], count[opposite])
print(pairs)

```

18.

Write a program that:

- Accepts a 3-digit number as input
- Checks if the number is divisible by 9
- Prints the appropriate message

Sample Input:

126

Sample Output:

Number 126 is divisible by 9

Solution

C++

```
#include <iostream>
```

```

using namespace std;
int main() {
    int number;
    cin >> number;
    if (number >= 100 && number <= 999) {
        if (number % 9 == 0) {
            cout << "Number " << number << " is divisible by 9" << endl;
        } else {
            cout << "Number " << number << " is not divisible by 9" << endl;
        }
    }
    return 0;
}

```

Java

```

import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int number = scanner.nextInt();
        if (number >= 100 && number <= 999) {
            if (number % 9 == 0) {
                System.out.println("Number " + number + " is divisible by 9");
            } else {
                System.out.println("Number " + number + " is not divisible by 9");
            }
        }
    }
}

```

Python

```

number = int(input())

```

```
if 100 <= number <= 999:
    if number % 9 == 0:
        print(f"Number {number} is divisible by 9")
    else:
        print(f"Number {number} is not divisible by 9")
```

19.

You are given a list of students. For each student, read:

- Name
- Age
- Grade (as a character like 'A', 'B', etc.)
- Gender ("Male" or "Female")

Your Task:

1. Print names of students whose age is greater than 20.
2. Calculate and print the average ASCII value of grades of all female students.

Sample Input:

4

Alice 21 A Female

Bob 19 B Male

Clara 22 B Female

David 20 C Male

Sample Output:

Alice

Clara

65.5

Explanation:

- Alice (21) and Clara (22) → age > 20 → printed
- Female students:

Alice → Grade: 'A' → ASCII = 65

Clara → Grade: 'B' → ASCII = 66

$$\rightarrow \text{Average} = (65 + 66) / 2 = 65.5$$

Solutions

C++

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<string> names;
    int totalGrade = 0, totalFemale = 0;
    for (int i = 0; i < n; ++i) {
        string name, gender;
        int age;
        char grade;
        cin >> name >> age >> grade >> gender;
        if (age > 20) {
            names.push_back(name);
        }
        if (gender == "Female") {
            totalGrade += (int)grade;
            totalFemale++;
        }
    }
    for (string s : names) {
        cout << s << endl;
    }
    double avg = totalFemale == 0 ? 0 : (double)totalGrade / totalFemale;
    cout << avg << endl;
```

```
    return 0;
}
```

Java

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();
        List<String> above20 = new ArrayList<>();
        int totalFemale = 0, totalGrade = 0;
        while (n-- > 0) {
            String name = scanner.next();
            int age = scanner.nextInt();
            char grade = scanner.next().charAt(0);
            String gender = scanner.next();
            if (age > 20) {
                above20.add(name);
            }
            if (gender.equals("Female")) {
                totalFemale++;
                totalGrade += (int) grade;
            }
        }
        for (String name : above20) {
            System.out.println(name);
        }
        double avg = totalFemale == 0 ? 0 : (double) totalGrade / totalFemale;
        System.out.println(avg);
    }
}
```

Python

```
n = int(input())
above20 = []
total_grade = 0
female_count = 0
for _ in range(n):
    parts = input().split()
    name = parts[0]
    age = int(parts[1])
    grade = parts[2]
    gender = parts[3]
    if age > 20:
        above20.append(name)
    if gender == "Female":
        total_grade += ord(grade)
        female_count += 1
for name in above20:
    print(name)
avg = 0 if female_count == 0 else total_grade / female_count
print(avg)
```

20.

Given two integers x and y, find the nearest number divisible by y that is closest to x.

Sample Input

29 6

Sample Output

30

Explanation of Output

- $29 \div 6 = 4.83\dots$
- Round $4.83 \rightarrow 5$

- Multiply: $5 \times 6 = 30$
- So, the nearest multiple of 6 to 29 is 30

Solution

C++

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    int x, y;
    cin >> x >> y;
    double division = (double)x / y;
    int rounded = (division - (int)division >= 0.5) ? (int)division + 1 : (int)division;
    cout << rounded * y << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;
public class CustomRound {
    public static int customRound(double number) {
        int integerPart = (int) number;
        double decimalPart = number - integerPart;
        if (decimalPart >= 0.5) {
            return integerPart + 1;
        } else {
            return integerPart;
        }
    }
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
```



```

        int x = scanner.nextInt();
        int y = scanner.nextInt();
        double nearest = (double) x / y;
        int roundedNumber = customRound(nearest);
        System.out.println(roundedNumber * y);
    }
}

```

Python

```

x, y = map(int, input().split())
quotient = x / y
rounded = int(quotient) + 1 if (quotient - int(quotient)) >= 0.5 else int(quotient)
print(rounded * y)

```

21.

Given a string inputStr and an integer value, transform the string using the following rules:

1. Letters (a–z, A–Z):

Shift them forward in the alphabet by value positions, wrapping around after z or Z.

2. Digits (0–9):

Shift them forward by value, wrapping around after 9.

3. Special characters:

Replace @ with #

Replace any other special character with @

Sample Input 1

Input string: Ab9@!

Value: 1

Sample Output 1

Output: Bc0#@

Solutions:

C++

```
#include <iostream>
using namespace std;
string addValueToChars(string s, int val) {
    string result = "";
    for (char ch : s) {
        if (islower(ch)) {
            result += (char)((ch - 'a' + val) % 26 + 'a');
        } else if (isupper(ch)) {
            result += (char)((ch - 'A' + val) % 26 + 'A');
        } else if (isdigit(ch)) {
            result += (char)((ch - '0' + val) % 10 + '0');
        } else {
            result += (ch == '@') ? '#' : '@';
        }
    }
    return result;
}

int main() {
    string input;
    int value;
    cin >> input >> value;
    cout << addValueToChars(input, value) << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;
public class Main {
    public static String addValueToChars(String str, int val) {
        StringBuilder sb = new StringBuilder();
```

```

for (char ch : str.toCharArray()) {
    if (Character.isLowerCase(ch)) {
        sb.append((char) ((ch - 'a' + val) % 26 + 'a'));
    } else if (Character.isUpperCase(ch)) {
        sb.append((char) ((ch - 'A' + val) % 26 + 'A'));
    } else if (Character.isDigit(ch)) {
        sb.append((char) ((ch - '0' + val) % 10 + '0'));
    } else {
        sb.append(ch == '@' ? '#' : '@');
    }
}
return sb.toString();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String inputStr = sc.next();
    int value = sc.nextInt();
    System.out.println(addValueToChars(inputStr, value));
}
}

```

Python

```

def add_value_to_chars(s, val):
    result = ""
    for ch in s:
        if ch.islower():
            result += chr((ord(ch) - ord('a') + val) % 26 + ord('a'))
        elif ch.isupper():
            result += chr((ord(ch) - ord('A') + val) % 26 + ord('A'))
        elif ch.isdigit():
            result += chr((ord(ch) - ord('0') + val) % 10 + ord('0'))

```

```

        else:
            result += '#' if ch == '@' else '@'
    return result

# Input
s = input()
val = int(input())
print(add_value_to_chars(s, val))

```

22. You are given a number n followed by n characters as input. The program prints all contiguous combinations (substrings) of the characters in left-to-right order.

Test Case 1

Input:

3 x y z

Output:

x, x y, x y z, y, y z, z

Solutions

C++

```

#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<string> chars(n);
    for (int i = 0; i < n; i++) {
        cin >> chars[i];
    }
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            for (int k = i; k <= j; k++) {

```

```

        cout << chars[k];
        if (k < j) cout << " ";
    }
    if (!(i == n-1 && j == n-1)) cout << ", ";
}
}
return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] arr = sc.nextLine().split(" ");
        int n = Integer.parseInt(arr[0]);
        String[] chars = new String[n];
        for (int i = 0; i < n; i++) {
            chars[i] = arr[i + 1];
        }
        for (int i = 0; i < n; i++) {
            for (int j = i; j < n; j++) {
                for (int k = i; k <= j; k++) {
                    System.out.print(chars[k]);
                    if (k < j) System.out.print(" ");
                }
                if (!(i == n - 1 && j == n - 1)) {
                    System.out.print(", ");
                }
            }
        }
    }
}

```

```
}  
}
```

Python

```
arr = input().split()  
n = int(arr[0])  
chars = arr[1:]  
for i in range(n):  
    for j in range(i, n):  
        print(" ".join(chars[i:j+1]), end=" ", " if not (i == n-1 and j == n-1) else "")
```

23.

You are given a line of strings (e.g., fruits or names). The program counts how many times each word appears and prints the unique keys (i.e., words) sorted:

- First in ascending (A–Z)
- Then in descending (Z–A)

It does not print the frequencies, just the sorted keys.

Sample Input

apple banana mango banana orange apple

Sample Output

apple banana mango orange
orange mango banana apple

Solutions

C++

```
#include <iostream>  
#include <map>  
#include <vector>  
#include <algorithm>  
using namespace std;  
int main() {  
    string word;
```

```

map<string, int> freq;
while (cin >> word) {
    freq[word]++;
    if (cin.peek() == '\n') break;
}

vector<string> keys;
for (auto &p : freq) keys.push_back(p.first);
sort(keys.begin(), keys.end());
for (string s : keys) cout << s << " ";
cout << endl;
sort(keys.rbegin(), keys.rend());
for (string s : keys) cout << s << " ";
cout << endl;
return 0;
}

```

Java

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String[] arr = scanner.nextLine().split(" ");
        // Sort in ascending order
        String[] asc = arr.clone();
        Arrays.sort(asc);
        System.out.println("Ascending:");
        for (String s : asc) {
            System.out.print(s + " ");
        }
        System.out.println();
    }
}

```

```

// Sort in descending order
String[] desc = arr.clone();
Arrays.sort(desc, Collections.reverseOrder());
System.out.println("Descending:");
for (String s : desc) {
    System.out.print(s + " ");
}
}
}

```

Python

```

from collections import Counter
arr = input().split()
count = Counter(arr)
# Ascending
for key in sorted(count.keys()):
    print(key, end=' ')
print()
# Descending
for key in sorted(count.keys(), reverse=True):
    print(key, end=' ')
print()

```

24.

Given a number N, toggle all bits in its binary representation (i.e., change all 1s to 0s and all 0s to 1s), and print the decimal number after toggling.

Let N = 5

Binary: 101

Toggled: 010 → Decimal: 2

Solutions

C++


```

#include <iostream>
using namespace std;
int main() {
    int N;
    cin >> N;
    if (N <= 0) {
        cout << "N must be greater than 0";
        return 0;
    }
    int numBits = 0, temp = N;
    while (temp > 0) {
        numBits++;
        temp >>= 1;
    }
    int mask = (1 << numBits) - 1;
    cout << (N ^ mask) << endl;
    return 0;
}

```

Java

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int N = scanner.nextInt();
        if (N <= 0) {
            System.out.println("N must be greater than 0");
            return;
        }
        // Find the number of bits in N
        int numBits = Integer.toBinaryString(N).length();
    }
}

```

```

        // Create a bitmask of all 1's of the same length
        int mask = (1 << numBits) - 1;

        // Toggle the bits
        int result = N ^ mask;
        System.out.println(result);
    }
}

```

Python

```

N = int(input("Enter N: "))
if N <= 0:
    print("N must be greater than 0")
else:
    num_bits = N.bit_length()
    mask = (1 << num_bits) - 1
    print(N ^ mask)

```

25.

Given a NxN Matrix you have rotate the matrix by 90 degree in clockwise direction and print the resultant matrix a) N that is size of matrix b) Next NxN contains the values of elements of matrix

Constraints:

N should be greater than zero

Example Input

3

1 2 3

4 5 6

7 8 9

Output

7 4 1

8 5 2

9 6 3

Solutions

C++

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int N;
    cin >> N;
    if (N <= 0) {
        cout << "N must be greater than 0";
        return 0;
    }
    vector<vector<int>> mat(N, vector<int>(N));
    // Input
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            cin >> mat[i][j];
    // Transpose
    for (int i = 0; i < N; i++)
        for (int j = i + 1; j < N; j++)
            swap(mat[i][j], mat[j][i]);
    // Reverse each row
    for (int i = 0; i < N; i++)
        reverse(mat[i].begin(), mat[i].end());
    // Output
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
            cout << mat[i][j] << " ";
```

```

        cout << endl;
    }
    return 0;
}

```

Java

```

import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        int N = scanner.nextInt();
        if (N <= 0) {
            System.out.println("N must be greater than 0");
            return;
        }
        int[][] matrix = new int[N][N];
        // Input
        for (int i = 0; i < N; i++)
            for (int j = 0; j < N; j++)
                matrix[i][j] = scanner.nextInt();
        // Step 1: Transpose
        for (int i = 0; i < N; i++) {
            for (int j = i + 1; j < N; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }
        // Step 2: Reverse each row
        for (int i = 0; i < N; i++) {
            for (int j = 0, k = N - 1; j < k; j++, k--) {

```

```

        int temp = matrix[i][j];
        matrix[i][j] = matrix[i][k];
        matrix[i][k] = temp;
    }
}
// Output
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++)
        System.out.print(matrix[i][j] + " ");
    System.out.println();
}
}

```

Python

```

N = int(input())
if N <= 0:
    print("N must be greater than 0")
else:
    matrix = [list(map(int, input().split())) for _ in range(N)]
    # Transpose
    for i in range(N):
        for j in range(i+1, N):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    # Reverse each row
    for row in matrix:
        row.reverse()
    # Output
    for row in matrix:
        print(*row)

```

26.

Given a string in small case with repeating character in it number p will be threshold character count record the frequency of repeating character count of repeating characters. Write a program to print only first character from the string which has count greater than or equal to p and in case of multiple character print the character which is alphabetic smallest.(TCS NQT off campus 2023)

Solutions

C++

```
#include <iostream>
#include <map>
#include <set>
using namespace std;
int main() {
    string s;
    int p;
    getline(cin, s);
    cin >> p;
    map<char, int> freq;
    for (char ch : s) {
        freq[ch]++;
    }
    set<char> candidates;
    for (auto& it : freq) {
        if (it.second >= p) {
            candidates.insert(it.first);
        }
    }
    if (candidates.empty()) {
        cout << endl;
        return 0;
    }
    char smallestChar = *candidates.begin();
```

```

    for (char ch : s) {
        if (ch == smallestChar) {
            cout << ch << endl;
            break;
        }
    }
    return 0;
}

```

Java

```

import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int p = sc.nextInt();
        // Frequency map
        Map<Character, Integer> freq = new HashMap<>();
        for (char ch : s.toCharArray()) {
            freq.put(ch, freq.getDefault(ch, 0) + 1);
        }
        // Collect characters with freq >= p
        TreeSet<Character> candidates = new TreeSet<>();
        for (Map.Entry<Character, Integer> entry : freq.entrySet()) {
            if (entry.getValue() >= p) {
                candidates.add(entry.getKey());
            }
        }
        if (candidates.isEmpty()) {
            // No character meets threshold
            System.out.println();
        }
    }
}

```

```

        return;
    }
    // Alphabetically smallest character among those meeting freq >= p
    char smallestChar = candidates.first();
    // Print the first occurrence of this character in the string
    for (char ch : s.toCharArray()) {
        if (ch == smallestChar) {
            System.out.println(ch);
            break;
        }
    }
}

```

Python

```

s = input()
p = int(input())
from collections import Counter
freq = Counter(s)
# Characters with frequency >= p
candidates = [ch for ch, count in freq.items() if count >= p]
if not candidates:
    # No character meets threshold
    print()
else:
    smallest_char = min(candidates)
    # Print first occurrence of this character
    for ch in s:
        if ch == smallest_char:
            print(ch)
            break

```


27.

Time is integral part of our life. Soci horology which is the discipline of studying time and clock making. A watchmaker is passionate about delivering the best watches 7 with accurate time all over the world. Each watch he makes, he tests the time in it with a standard watch showing GMT of his place. The task here is to find if the new watch he made is working correctly and find the number of minutes (delay or early) such that: Compare the initial time (h, m) and the current time (h1,m1) in the new watch.

The time after X hours in the new watch may be correct or incorrect.. Display the number of minutes by which new watch is lagging or early from the actual time. If the time is lagging display positive integer an if the time is early display the negative integer. Given a initial time in h:m and current time h1:m2 and x display the delays in minutes in from the initial time after x hours (TCS off campus NQT 2023)

Solutions

C++

```
#include <iostream>

using namespace std;

int main() {
    int h, m, h1, m1, x;
    cin >> h >> m >> h1 >> m1 >> x;
    int correctTime = (h + x) * 60 + m;
    int newWatchTime = h1 * 60 + m1;
    int difference = newWatchTime - correctTime;
    cout << -difference << endl;
    return 0;
}
```

Java

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // Read initial time
```

```

int h = sc.nextInt();
int m = sc.nextInt();
// Read current time in new watch
int h1 = sc.nextInt();
int m1 = sc.nextInt();
// Read x hours after which time is to be validated
int x = sc.nextInt();
// Total minutes of correct time after x hours
int correctMinutes = (h + x) * 60 + m;
// Total minutes of new watch's time
int newWatchMinutes = h1 * 60 + m1;
// Calculate difference
int difference = newWatchMinutes - correctMinutes;
// If watch is ahead, show negative value
// If watch is behind, show positive value
System.out.println(-difference);
}
}

```

Python

```

# Read input
h, m = map(int, input().split())
h1, m1 = map(int, input().split())
x = int(input())

# Convert times to total minutes
expected_time = (h + x) * 60 + m
actual_time = h1 * 60 + m1
# Difference (delay or early)
difference = actual_time - expected_time
# If early, return negative; if delay, return positive

```

```
print(-difference)
```

28.

A mall maintains a pricing format for all its products. A value N is printed on each product. When the scanner reads the value N on the item, the absolute difference between the product of all the digits in the value N and the sum of all the digits is the price of the item. The task here is to design the software such that given the code of any item N and returns the value of that item

Solutions

C++

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    int N;
    cin >> N;
    int sum = 0, product = 1;
    int num = N;
    while (num > 0) {
        int digit = num % 10;
        sum += digit;
        product *= digit;
        num /= 10;
    }
    cout << abs(product - sum) << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);
int N = sc.nextInt();
int sum = 0, product = 1;
int num = N;
while (num > 0) {
    int digit = num % 10;
    sum += digit;
    product *= digit;
    num /= 10;
}
int price = Math.abs(product - sum);
System.out.println(price);
}
}

```

Python

```

N = int(input())
sum_digits = 0
product_digits = 1
num = N
while num > 0:
    digit = num % 10
    sum_digits += digit
    product_digits *= digit
    num //= 10
print(abs(product_digits - sum_digits))

```

29. NQT off campus 2023

Given:

- A string s (e.g., "apple")
- A key string key (e.g., "eapl")

Objective: Sort string *s* by following the character order defined in *key*.

Example

Input:

s = "apple"

key = "eapl"

Steps:

- According to *key*, the order is: $e \rightarrow a \rightarrow p \rightarrow l$
- From *s*, pick:

$e \rightarrow 1$ time

$a \rightarrow 1$ time

$p \rightarrow 2$ times

$l \rightarrow 1$ time

Output:

eappl

Solutions

C++

```
#include <iostream>
```

```
#include <unordered_map>
```

```
#include <string>
```

```
using namespace std;
```

```
int main() {
```

```
    string s, key;
```

```
    getline(cin, s);
```

```
    getline(cin, key);
```

```
    unordered_map<char, int> freq;
```

```
    for (char ch : s) {
```

```
        freq[ch]++;
```

```
    }
```

```

string result = "";
for (char ch : key) {
    if (freq.count(ch)) {
        result.append(freq[ch], ch);
        freq.erase(ch);
    }
}
for (auto &entry : freq) {
    result.append(entry.second, entry.first);
}
cout << result << endl;
return 0;
}

```

Java

```

import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();    // original string
        String key = sc.nextLine();  // key string
        // Count frequency of each character in s
        Map<Character, Integer> freq = new HashMap<>();
        for (char ch : s.toCharArray()) {
            freq.put(ch, freq.getOrDefault(ch, 0) + 1);
        }
        StringBuilder result = new StringBuilder();
        // Append characters based on key order
        for (char ch : key.toCharArray()) {
            if (freq.containsKey(ch)) {
                int count = freq.get(ch);

```

```

        for (int i = 0; i < count; i++) {
            result.append(ch);
        }
        freq.remove(ch); // remove once processed
    }
}

// Optional: append leftover characters not in key
for (Map.Entry<Character, Integer> entry : freq.entrySet()) {
    char ch = entry.getKey();
    int count = entry.getValue();
    for (int i = 0; i < count; i++) {
        result.append(ch);
    }
}

System.out.println(result.toString());
}
}

```

Python

```

from collections import Counter

s = input().strip()
key = input().strip()
freq = Counter(s)
result = []

# Append chars based on key order
for ch in key:
    if ch in freq:
        result.append(ch * freq[ch])
        del freq[ch]

# Optional: add leftover characters not in key
for ch in freq:

```

```
result.append(ch * freq[ch])
print("".join(result))
```

30.

In the mystical land of Numeria, a Relative Blood Number is defined as a number such that the sum of the factorials of its digits is equal to the number itself.

You are to write a program that checks whether a given positive integer is a Relative Blood Number or not.

Input:

145

Output:

Yes, it is a Relative Blood Number

Explanation:

$1! + 4! + 5! = 1 + 24 + 120 = 145 \rightarrow$ Matches the number itself.

Solutions

C++

```
#include <iostream>
```

```
using namespace std;
```

```
int factorial(int n) {
    if (n == 0 || n == 1) return 1;
    int f = 1;
    for (int i = 2; i <= n; ++i)
        f *= i;
    return f;
}
```

```
int main() {
    int num;
    cin >> num;
    int original = num, sum = 0;
```



```

while (num > 0) {
    int digit = num % 10;
    sum += factorial(digit);
    num /= 10;
}
if (sum == original)
    cout << "Yes, it is a Relative Blood Number\n";
else
    cout << "No, it is not a Relative Blood Number\n";
return 0;
}

```

Java

```

import java.util.*;

public class Main {
    public static int factorial(int n) {
        if (n == 0 || n == 1) return 1;
        int fact = 1;
        for (int i = 2; i <= n; i++) {
            fact *= i;
        }
        return fact;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt();
        int original = num;
        int sum = 0;
        while (num > 0) {
            int digit = num % 10;
            sum += factorial(digit);

```

```

        num /= 10;
    }
    if (sum == original)
        System.out.println("Yes, it is a Relative Blood Number");
    else
        System.out.println("No, it is not a Relative Blood Number");
    }
}

```

Python

```

def factorial(n):
    return 1 if n == 0 or n == 1 else n * factorial(n - 1)
num = int(input())
original = num
sum_fact = sum(factorial(int(d)) for d in str(num))
if sum_fact == original:
    print("Yes, it is a Relative Blood Number")
else:
    print("No, it is not a Relative Blood Number")

```

31. NQT off campus 2023

Bastin is struggling to extract numbers from strings. Your task is to extract all consecutive digit sequences (numbers) from a given string and print the largest number among those that do not contain the digit '9'.

Sample Input 1

hello this is alpha 5051 and 9475

Sample Output 1

5051

Explanation:

Extracted numbers: 5051, 9475

Only 5051 does not contain 9, so it's printed.

Solutions

C++

```
#include <iostream>
#include <string>
#include <sstream>
#include <cctype>
using namespace std;
bool contains9(const string& s) {
    return s.find('9') != string::npos;
}
int main() {
    string input;
    getline(cin, input);
    string num = "";
    int maxVal = -1;
    for (char ch : input) {
        if (isdigit(ch)) {
            num += ch;
        } else {
            if (!num.empty() && !contains9(num)) {
                maxVal = max(maxVal, stoi(num));
            }
            num = "";
        }
    }
    if (!num.empty() && !contains9(num)) {
        maxVal = max(maxVal, stoi(num));
    }
    cout << maxVal << endl;
    return 0;
}
```

Java

```

import java.util.*;
import java.util.regex.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        Pattern pattern = Pattern.compile("\\d+");
        Matcher matcher = pattern.matcher(s);
        int max = -1
        while (matcher.find()) {
            String num = matcher.group();
            if (!num.contains("9")) {
                int val = Integer.parseInt(num);
                if (val > max) max = val;
            }
        }
        System.out.println(max);
    }
}

```

Python

```

import re
def largest_number_without_9(s):
    numbers = re.findall(r'\d+', s)
    valid_numbers = [int(num) for num in numbers if '9' not in num]
    return max(valid_numbers) if valid_numbers else -1
# Input
s = input()
print(largest_number_without_9(s))

```

32.

Given a string S containing only double quotes (") and hash symbols (#), determine the difference in their counts. (NQT off campus 2023)

A string is considered valid if the number of " and # characters is equal.

- If " > # → Output a positive integer (difference)
- If # > " → Output a negative integer (difference)
- If both are equal → Output 0

Sample Input 1:

S = ##""

Sample Output 1:

0

Solutions

C++

```
#include <iostream>
using namespace std;
int main() {
    string s;
    getline(cin, s);
    int quoteCount = 0, hashCount = 0;
    for (char c : s) {
        if (c == "\"") quoteCount++;
        else if (c == '#') hashCount++;
    }
    cout << (quoteCount - hashCount) << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```

String s = sc.nextLine();
int quoteCount = 0, hashCount = 0;
for (char c : s.toCharArray()) {
    if (c == '"') quoteCount++;
    else if (c == '#') hashCount++;
}
System.out.println(quoteCount - hashCount);
}
}

```

Python

```

s = input().strip()
quote_count = s.count('"')
hash_count = s.count('#')
print(quote_count - hash_count)

```

33.

Given an array `arr[]` of `N` integers, replace each element in the array with its frequency in the original array.

Sample Input:

```
arr = [1, 2, 2, 6]
```

Sample Output:

```
[1, 2, 2, 1]
```

Explanation:

- Frequency of 1 $\rightarrow$ 1
 - Frequency of 2 $\rightarrow$ 2
 - Frequency of 6 $\rightarrow$ 1
- So, replace each value in the array with its frequency.

Solutions

C++

```
#include <iostream>
```

```

#include <unordered_map>
#include <vector>
using namespace std;
int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    vector<int> arr(n);
    unordered_map<int, int> freq;
    cout << "Enter elements: ";
    for (int i = 0; i < n; ++i) {
        cin >> arr[i];
        freq[arr[i]]++;
    }
    for (int i = 0; i < n; ++i) {
        cout << freq[arr[i]] << " ";
    }
    return 0;
}

```

Java

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        Map<Integer, Integer> freq = new HashMap<>();
        System.out.print("Enter elements: ");
        for (int i = 0; i < n; i++) {

```

```

        arr[i] = sc.nextInt();
        freq.put(arr[i], freq.getDefault(arr[i], 0) + 1);
    }
    for (int i = 0; i < n; i++) {
        System.out.print(freq.get(arr[i]) + " ");
    }
}
}

```

Python

```

from collections import Counter
arr = list(map(int, input("Enter array elements: ").split()))
freq = Counter(arr)
result = [freq[x] for x in arr]
print(result)

```

34.

Given a string convert the characters of the string into the opposite case

i.e if a character is the lower case then convert it into upper case and vice versa.

Example

Input

KNacademy

Output

KnACADEMY

Solutions

C++

```

#include <iostream>
using namespace std;
int main() {
    string str;

```



```

getline(cin, str);
for (char &ch : str) {
    if (isupper(ch))
        ch = tolower(ch);
    else
        ch = toupper(ch);
}
cout << str;
return 0;}

```

Java

```

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String str = scanner.nextLine();
        StringBuilder result = new StringBuilder();
        for (char ch : str.toCharArray()) {
            if (Character.isUpperCase(ch)) {
                result.append(Character.toLowerCase(ch));
            } else {
                result.append(Character.toUpperCase(ch));
            }
        }
        System.out.println(result.toString());
    }
}

```

Python

```

s = input("Enter the string: ")
print(s.swapcase())

```

35.

An accountant has a habit of typing numbers in reverse. For example, if he needs to enter 123, he types 321. His boss wants him to correct these numbers by re-entering them from right to left, which gives the corrected number. Your task is to help the boss find the corrected numbers.

Input: A number N.

Output: The corrected number.

Constraints and Notes:

Ignore any leading zeros in the corrected number. For example, for an input of 500, the reversed number is 005, which should be displayed as 5.

If the input number is negative, the sign should be retained after reversal. For example, if N = -4512, the correct number would be -2154.

Solutions

C++

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;
string reversePreserve(string s) {
    bool isNegative = false;
    if (s[0] == '-') {
        isNegative = true;
        s = s.substr(1); // remove negative sign for now
    }
    reverse(s.begin(), s.end());
    // Remove leading zeros after reversal
    size_t pos = s.find_first_not_of('0');
    if (pos != string::npos) {
        s = s.substr(pos);
    } else {
        s = "0"; // All zeros case
    }
}
```

```

    }
    if (isNegative)
        return "-" + s;
    return s;
}

int main() {
    string input;
    cout << "Enter the number: ";
    cin >> input;
    cout << reversePreserve(input) << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {

    public static String reversePreserve(String s) {
        boolean isNegative = s.startsWith("-");
        if (isNegative) {
            s = s.substring(1);
        }
        StringBuilder sb = new StringBuilder(s);
        String reversed = sb.reverse().toString().replaceFirst("^0+", "");
        if (reversed.isEmpty()) reversed = "0";
        return (isNegative ? "-" : "") + reversed;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine().trim();
        System.out.println(reversePreserve(s));
    }
}

```

```
}
```

Python

```
def reverse_preserve_format(s):  
    is_negative = s.startswith('-')  
    if is_negative:  
        s = s[1:]  
    reversed_str = s[::-1]  
    # Remove only leading zeros *after* reversing  
    reversed_str = reversed_str.lstrip('0')  
    if reversed_str == "":  
        reversed_str = '0' # if input was like '000'  
    return ('-' if is_negative else "") + reversed_str  
  
# Input as string  
s = input("Enter the number: ").strip()  
print(reverse_preserve_format(s))
```

36. Missing English Alphabets in a Sentence

Given: A sentence string str.

Task: Determine if the given sentence contains all letters of the English alphabet (a to z or A to Z).

Output:

- If all letters are present, print 0.
- Otherwise, print all missing letters of the alphabet.

Notes:

- All characters in the given sentence should be treated as case-insensitive (e.g., 'A' and 'a' are considered the same).
- The output should be in alphabetical order.
- The output should contain only lowercase alphabets.

Example 1:

- Input: "The four boxing wizard starts over the quickly."
- Output: "jmp"

- Explanation: The input sentence does not contain all letters from 'a' to 'z' or 'A' to 'Z' as 'j', 'm' and 'p' are missing.

Solutions

C++

```
#include <iostream>
#include <set>
#include <string>
#include <cctype>
using namespace std;
int main() {
    string s;
    getline(cin, s);
    set<char> letters;
    for (char ch : s) {
        if (isalpha(ch)) {
            letters.insert(tolower(ch));
        }
    }
    string missing = "";
    for (char ch = 'a'; ch <= 'z'; ++ch) {
        if (letters.find(ch) == letters.end()) {
            missing += ch;
        }
    }
    if (missing.empty())
        cout << 0 << endl;
    else
        cout << missing << endl;
    return 0;
}
```

Java

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        String sentence = sc.nextLine().toLowerCase();
        Set<Character> letters = new HashSet<>();
        for (char ch : sentence.toCharArray()) {
            if (Character.isLetter(ch)) {
                letters.add(ch);
            }
        }

        StringBuilder missing = new StringBuilder();
        for (char ch = 'a'; ch <= 'z'; ch++) {
            if (!letters.contains(ch)) {
                missing.append(ch);
            }
        }

        if (missing.length() == 0)
            System.out.println(0);
        else
            System.out.println(missing.toString());
    }
}
```

Python

```
def missing_alphabets(sentence):
    sentence = sentence.lower()
    found = set(ch for ch in sentence if ch.isalpha())
    missing = ''.join([chr(i) for i in range(ord('a'), ord('z')+1) if chr(i) not in found])
    if not missing:
```

```
        print(0)
    else:
        print(missing)
# Example usage
sentence = input()
missing_alphabets(sentence)
```

37. Knapsack Problem: Maximizing Value within Truck Capacity

Mr. Rao is relocating from place A to place B. The moving truck has a maximum capacity C. There are N items in the house, where each item has a corresponding value (V_i) and weight (W_i). Mr. Rao has to carry only the most valuable items whose total weight does not exceed the capacity of the truck. The task is to find those items (single or combination of items) whose total value (v) will be the maximum and their corresponding weight (w) will not exceed truck capacity (C).

Input:

- N: Number of items.
- C: Maximum capacity of the truck, an integer value.
- $W[0 \text{ to } N-1]$: An array consisting of the weight of each item.
- $V[0 \text{ to } N-1]$: An array consisting of the value of each item.

Example 1:

- Input:
 $N = 4$
 $C = 80$
 $V[] = [10, 45, 60, 90]$
 $W[] = [15, 20, 30, 40]$
- Output: 150

Solutions

C++

```
#include <iostream>
#include <vector>
#include <algorithm>
```

```
using namespace std;
```

```
int knapsack(int C, vector<int> &W, vector<int> &V, int N) {  
    vector<vector<int>> dp(N + 1, vector<int>(C + 1, 0));  
    for (int i = 1; i <= N; i++) {  
        for (int w = 0; w <= C; w++) {  
            if (W[i - 1] <= w) {  
                dp[i][w] = max(dp[i - 1][w], V[i - 1] + dp[i - 1][w - W[i - 1]]);  
            } else {  
                dp[i][w] = dp[i - 1][w];  
            }  
        }  
    }  
    return dp[N][C];  
}
```

```
int main() {  
    int N, C;  
    cin >> N >> C;  
    vector<int> V(N), W(N);  
    for (int i = 0; i < N; i++) cin >> V[i];  
    for (int i = 0; i < N; i++) cin >> W[i];  
    cout << knapsack(C, W, V, N) << endl;  
    return 0;  
}
```

Java

```
import java.util.Scanner;  
  
public class Main {  
    public static int knapsack(int C, int[] W, int[] V, int N) {  
        int[][] dp = new int[N + 1][C + 1];  
        for (int i = 1; i <= N; i++) {
```



```

        for (int w = 0; w <= C; w++) {
            if (W[i - 1] <= w) {
                dp[i][w] = Math.max(dp[i - 1][w], V[i - 1] + dp[i - 1][w - W[i - 1]]);
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }
    return dp[N][C];
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int N = sc.nextInt();
    int C = sc.nextInt();
    int[] V = new int[N];
    int[] W = new int[N];
    for (int i = 0; i < N; i++) V[i] = sc.nextInt();
    for (int i = 0; i < N; i++) W[i] = sc.nextInt();
    System.out.println(knapsack(C, W, V, N));
}
}

```

Python

```

def knapsack(N, C, V, W):
    dp = [[0] * (C + 1) for _ in range(N + 1)]
    for i in range(1, N + 1):
        for w in range(C + 1):
            if W[i - 1] <= w:
                dp[i][w] = max(dp[i - 1][w], V[i - 1] + dp[i - 1][w - W[i - 1]])
            else:
                dp[i][w] = dp[i - 1][w]

```

```
    return dp[N][C]
N, C = map(int, input().split())
V = list(map(int, input().split()))
W = list(map(int, input().split()))
print(knapsack(N, C, V, W))
```

38.

Given a positive integer number N. the task is to find the number of ways in which the given number can be represented as the product of two distinct numbers.

Example 1

Input:

14

Output:

2

Explanation:

14 can be represented as $1 * 14$ and $2 * 7$

Example 2

Input :

16

Output :

2

Explanation :

16 can be represented as $1 * 16$ and $2 * 8$

Note

$4 * 4$ is not valid way because both the numbers are same.

Solutions

C++

```
#include <iostream>
```

```
#include <cmath>
```

```
using namespace std;
```

```
int countDistinctProductPairs(int N) {
```

```

int count = 0;
int limit = (int)sqrt(N);
for (int i = 1; i <= limit; i++) {
    if (N % i == 0) {
        int j = N / i;
        if (i != j) count++;
    }
}
return count;
}

int main() {
    int N;
    cin >> N;
    cout << countDistinctProductPairs(N) << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {
    public static int countDistinctProductPairs(int N) {
        int count = 0;
        int limit = (int) Math.sqrt(N);
        for (int i = 1; i <= limit; i++) {
            if (N % i == 0) {
                int j = N / i;
                if (i != j) count++;
            }
        }
        return count;
    }
}

```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int N = sc.nextInt();
    System.out.println(countDistinctProductPairs(N));
}
}

```

Python

```

import math
def count_distinct_product_pairs(N):
    count = 0
    limit = int(math.isqrt(N))
    for i in range(1, limit + 1):
        if N % i == 0:
            j = N // i
            if i != j:
                count += 1
    return count
N = int(input())
print(count_distinct_product_pairs(N))

```

39.

You are given a string that contains a name hidden in it. Find the hidden name and try to find the Lucky number for it.

Input Format

The input contains a single string with a maximum of 32 characters in it.

Output Format:

The output contains a single integer representing the lucky number for the name decoded from the input.

Explanation :

Find the alpha characters and find the sum of alphabetical values. The lucky number is the digital sum reduced up to a single digit.

Sample Input 1:

D@h*o1n&i%

Sample Output 1:

5

Explanation:

The hidden name is Dhoni. So the sum will be $4+8+15+14+9 = 50$

Digital sum reduced up to single-digit will be $50 \Rightarrow 5+0 = 5$.

Solutions**C++**

```
#include <iostream>
#include <cctype>
using namespace std;
int main() {
    string s;
    getline(cin, s);
    int total = 0;
    for (char ch : s) {
        if (isalpha(ch)) {
            total += tolower(ch) - 'a' + 1;
        }
    }
    while (total > 9) {
        int sum = 0;
        while (total > 0) {
            sum += total % 10;
            total /= 10;
        }
        total = sum;
    }
    cout << total << endl;
    return 0;}
```

Java

```

import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int total = 0
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (Character.isLetter(ch)) {
                total += Character.toLowerCase(ch) - 'a' + 1;
            }
        }
        while (total > 9) {
            int sum = 0;
            while (total > 0) {
                sum += total % 10;
                total /= 10;
            }
            total = sum;
        }
        System.out.println(total);
    }
}

```

Python

```

s = input()
total = 0
for ch in s:
    if ch.isalpha():
        total += ord(ch.lower()) - ord('a') + 1
while total > 9:

```

```
total = sum(int(d) for d in str(total))  
print(total)
```

40. Binary Search

Problem: Search for a target element in a sorted array using the binary search method.

Test Case:

- Input: arr = [1, 2, 3, 4, 5, 6, 7], target = 4
- Output: 3

Solutions

C++

```
#include <iostream>  
using namespace std;  
int binarySearch(int arr[], int n, int target) {  
    int low = 0, high = n - 1;  
    while (low <= high) {  
        int mid = (low + high) / 2;  
        if (arr[mid] == target) return mid;  
        else if (arr[mid] < target) low = mid + 1;  
        else high = mid - 1;  
    }  
    return -1;  
}  
int main() {  
    int n, target;  
    cout << "Enter size of array: ";  
    cin >> n;  
    int arr[n];  
    cout << "Enter sorted array: ";  
    for (int i = 0; i < n; i++) cin >> arr[i];
```

```

    cout << "Enter target element: ";
    cin >> target;
    int result = binarySearch(arr, n, target);
    cout << "Output: " << result << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class Main {

    public static int binarySearch(int[] arr, int target) {
        int low = 0, high = arr.length - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (arr[mid] == target) return mid;
            else if (arr[mid] < target) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.print("Enter sorted array: ");
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        System.out.print("Enter target element: ");
        int target = sc.nextInt();
        int result = binarySearch(arr, target);
        System.out.println("Output: " + result);
    }
}

```



```
}  
}
```

Python

```
def binary_search(arr, target):  
    low = 0  
    high = len(arr) - 1  
    while low <= high:  
        mid = (low + high) // 2  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:  
            low = mid + 1  
        else:  
            high = mid - 1  
    return -1  
  
# Input  
arr = list(map(int, input("Enter sorted array elements: ").split()))  
target = int(input("Enter target element: "))  
result = binary_search(arr, target)  
print("Output:", result)
```

-----HCL-----

1.

You are given a numeric string X. Your task is to convert this string into another string Y such that:

- For each digit d in X, you can optionally replace it with 9 - d

- Choose the minimum value between d and 9 - d
- The resulting number must not begin with zero

Input:

4545

Output:

4444

Explanation:

In the given string 4545, only the 2nd and 4th character from left i.e. 5 is changed to 4 (as $9-5=4$). Others remain the same.

Solutions

C++

```
#include <iostream>
using namespace std;
string transformMinNumber(string X) {
    string result = "";
    for (int i = 0; i < X.length(); i++) {
        int digit = X[i] - '0';
        int alt = 9 - digit;
        if (i == 0 && alt == 0)
            result += X[i];
        else
            result += to_string(min(digit, alt));
    }
    return result;
}

int main() {
    string X;
    cout << "Enter the number string: ";
    cin >> X;
    cout << "Output: " << transformMinNumber(X) << endl;
```

```
    return 0;
}
```

Java

```
import java.util.Scanner;

public class Main {

    public static String transformMinNumber(String X) {
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < X.length(); i++) {
            int digit = X.charAt(i) - '0';
            int alt = 9 - digit;
            if (i == 0 && alt == 0) {
                result.append(digit);
            } else {
                result.append(Math.min(digit, alt));
            }
        }
        return result.toString();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number string: ");
        String X = sc.nextLine();
        System.out.println("Output: " + transformMinNumber(X));
    }
}
```

Python

```
def transform_min_number(X):
    result = ""
    for i in range(len(X)):
        digit = int(X[i])
        alt_digit = 9 - digit
```

```

        # Don't start with 0
        if i == 0 and alt_digit == 0:
            result += X[i]
        else:
            result += str(min(digit, alt_digit))
    return result

# Input
X = input("Enter the number string: ")
print("Output:", transform_min_number(X))

```

2. Given a string of length n which contains alphabets and some '#' characters scattered throughout the string, move all the '#' characters to the front of the string while maintaining the relative order of the other characters.

Input:

Move###Hash#to#Front

Output:

####MoveHashtoFront

Solutions

C++

```

#include <iostream>
using namespace std;
string moveHashesToFront(string str) {
    int hashCount = 0;
    string result = "";
    for (char ch : str) {
        if (ch == '#') hashCount++;
        else result += ch;
    }
    return string(hashCount, '#') + result;
}

```

```

int main() {
    string str;
    getline(cin, str);
    cout << moveHashesToFront(str) << endl;
    return 0;
}

```

Java

```

import java.util.Scanner;

public class MoveHashes {
    public static String moveHashesToFront(String str) {
        int count = 0;
        StringBuilder sb = new StringBuilder();
        for (char ch : str.toCharArray()) {
            if (ch == '#') count++;
            else sb.append(ch);
        }
        return "#".repeat(count) + sb.toString();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();
        System.out.println(moveHashesToFront(str));
    }
}

```

Python

```

def move_hashes_to_front(s):
    count = s.count('#')
    result = "".join([ch for ch in s if ch != '#'])
    return '#' * count + result

```

Input

```
s = input()
print(move_hashes_to_front(s))
```

3.

Spiral Matrix Traversal

Write code to traverse a given matrix in a spiral format and print its elements in the order of traversal. The traversal should start from the top-left corner and proceed clockwise.

Input Format:

3. The first line contains two integers, rows and cols, representing the number of rows and columns of the matrix, respectively.
4. The subsequent rows lines each contain cols space-separated integers, representing the elements of the matrix.

Output Format: Print the elements of the matrix in a single line, separated by spaces, in spiral order.

Test Case 1:

- **Input:**

5 4

1 2 3 4

5 6 7 8

9 10 11 12

13 14 15 16

17 18 19 20

- **Expected Output:**

1 2 3 4 8 12 16 20 19 18 17 13 9 5 6 7 11 15 14 10

- **Explanation:**

Right: 1, 2, 3, 4

Down: 8, 12, 16, 20

Left: 19, 18, 17

Up: 13, 9, 5

Right (Inner): 6, 7

Down (Inner): 11, 15

Left (Inner): 14

Up (Inner): 10

Combined: 1 2 3 4 8 12 16 20 19 18 17 13 9 5 6 7 11 15 14 10

Test Case 2:

Input:

3 3

1 2 3

4 5 6

7 8 9

Expected Output:

1 2 3 6 9 8 7 4 5

Solutions

C++

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int rows, cols;
```

```
    cin >> rows >> cols;
```

```
    int a[100][100];
```

```
    for (int i = 0; i < rows; ++i)
```

```
        for (int j = 0; j < cols; ++j)
```

```
            cin >> a[i][j];
```

```
    int top = 0, bottom = rows - 1, left = 0, right = cols - 1;
```

```
    while (top <= bottom && left <= right) {
```

```
        for (int i = left; i <= right; ++i) cout << a[top][i] << " ";
```

```
        top++;
```

```
        for (int i = top; i <= bottom; ++i) cout << a[i][right] << " ";
```

```
        right--;
```

```
        if (top <= bottom) {
```

```

        for (int i = right; i >= left; --i) cout << a[bottom][i] << " ";
        bottom--;
    }
    if (left <= right) {
        for (int i = bottom; i >= top; --i) cout << a[i][left] << " ";
        left++;
    }
}
return 0;
}

```

Java

```

import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int rows = sc.nextInt(), cols = sc.nextInt();
        int[][] a = new int[rows][cols];
        for (int i = 0; i < rows; ++i)
            for (int j = 0; j < cols; ++j)
                a[i][j] = sc.nextInt();

        int top = 0, bottom = rows - 1, left = 0, right = cols - 1;
        while (top <= bottom && left <= right) {
            for (int i = left; i <= right; ++i) System.out.print(a[top][i] + " ");
            top++;
            for (int i = top; i <= bottom; ++i) System.out.print(a[i][right] + " ");
            right--;
            if (top <= bottom) {
                for (int i = right; i >= left; --i) System.out.print(a[bottom][i] + " ");
                bottom--;
            }
        }
    }
}

```



```

        if (left <= right) {
            for (int i = bottom; i >= top; --i) System.out.print(a[i][left] + " ");
            left++;
        }
    }
}

```

Python

```

rows, cols = map(int, input().split())
matrix = [list(map(int, input().split())) for _ in range(rows)]
top, bottom, left, right = 0, rows - 1, 0, cols - 1
res = []
while top <= bottom and left <= right:
    for i in range(left, right + 1): res.append(matrix[top][i])
    top += 1
    for i in range(top, bottom + 1): res.append(matrix[i][right])
    right -= 1
    if top <= bottom:
        for i in range(right, left - 1, -1): res.append(matrix[bottom][i])
        bottom -= 1
    if left <= right:
        for i in range(bottom, top - 1, -1): res.append(matrix[i][left])
        left += 1
print(*res)

```

4. Preorder Traversal of Binary Tree

Solutions

C++

```

#include <bits/stdc++.h>
using namespace std;

```

```

// Structure of a Binary Tree Node
struct Node {
    int data;
    struct Node *left, *right;
    Node(int v)
    {
        data = v;
        left = right = nullptr;
    }
};

// Function to print Preorder traversal
void printPreorder(struct Node* node)
{
    if (node == nullptr)
        return;

    // Deal with the node
    cout << node->data << " ";
    // Recursion on left subtree
    printPreorder(node->left);
    // Recursion on right subtree
    printPreorder(node->right);
}

int main()
{
    struct Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
}

```

```

    root->right->right = new Node(6);
    printPreorder(root);
    return 0;
}

```

Java

```

import java.util.*;

// Structure of a Binary Tree Node
class Node {
    int data;
    Node left, right;
    Node(int v) {
        data = v;
        left = right = null;
    }
}

// Class for BinaryTree
class BinaryTree {
    Node root;

    // Function to print preorder traversal
    public static void printPreorder(Node node) {
        if (node == null)
            return;

        // First deal with the node
        System.out.print(node.data + " ");

        // Then recur on left subtree
        printPreorder(node.left);

        // Finally recur on right subtree
        printPreorder(node.right);
    }
}

```

```

class KNA {
    public static void main(String[] args) {
        BinaryTree tree = new BinaryTree();
        tree.root = new Node(1);
        tree.root.left = new Node(2);
        tree.root.right = new Node(3);
        tree.root.left.left = new Node(4);
        tree.root.left.right = new Node(5);
        tree.root.right.right = new Node(6);
        BinaryTree.printPreorder(tree.root);
    }
}

```

Python

Structure of a Binary Tree Node

class Node:

```

    def __init__(self, v):
        self.data = v
        self.left = None
        self.right = None

```

Function to print preorder traversal

def printPreorder(node):

if node is None:

return

Deal with the node

print(node.data, end=' ')

Recur on left subtree

printPreorder(node.left)

Recur on right subtree

printPreorder(node.right)

```
if __name__ == '__main__':  
    root = Node(1)  
    root.left = Node(2)  
    root.right = Node(3)  
    root.left.left = Node(4)  
    root.left.right = Node(5)  
    root.right.right = Node(6)  
    printPreorder(root)
```

5.

John works in a coin factory. Each coin has an ID. John has an array nums of size N, representing the coin IDs he needs to consider.

- The smallest number in nums is called the low-limit.
- The largest number in nums is called the high-limit.

For every coin ID from low-limit to high-limit (inclusive), John calculates the sum of the digits of the coin ID. The coin is placed in the box corresponding to that digit sum.

Task: Determine the box that contains the maximum number of coins and print the count of coins in that box.

Solutions

C++

```
#include <iostream>  
#include <unordered_map>  
#include <vector>  
  
using namespace std;  
  
int digitSum(int n) {  
    int sum = 0;  
    while (n) {  
        sum += n % 10;  
        n /= 10;  
    }  
}
```

```

    }
    return sum;
}
int main() {
    int N;
    cin >> N;
    vector<int> nums(N);
    int low = 1e9, high = 0;
    for (int i = 0; i < N; ++i) {
        cin >> nums[i];
        low = min(low, nums[i]);
        high = max(high, nums[i]);
    }
    unordered_map<int, int> boxCount;
    int maxCount = 0;
    for (int i = low; i <= high; ++i) {
        int s = digitSum(i);
        boxCount[s]++;
        maxCount = max(maxCount, boxCount[s]);
    }
    int countSame = 0, boxNo = -1;
    for (auto &entry : boxCount) {
        if (entry.second == maxCount) {
            boxNo = entry.first;
            countSame++;
        }
    }
    if (countSame == boxCount.size()) {
        cout << "All boxes have same number of coins\n";
    } else {

```

```
        cout << boxNo << " " << maxCount << endl;
    }
    return 0;
}
```

Java

```
import java.util.*;

public class Main {

    public static int digitSum(int n) {
        int sum = 0;
        while (n > 0) {
            sum += n % 10;
            n /= 10;
        }
        return sum;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        int[] nums = new int[N];
        int low = Integer.MAX_VALUE, high = Integer.MIN_VALUE;
        for (int i = 0; i < N; i++) {
            nums[i] = sc.nextInt();
            low = Math.min(low, nums[i]);
            high = Math.max(high, nums[i]);
        }
        Map<Integer, Integer> map = new HashMap<>();
        int maxCount = 0;
        for (int i = low; i <= high; i++) {
            int sum = digitSum(i);
            map.put(sum, map.getOrDefault(sum, 0) + 1);
        }
    }
}
```

```

        maxCount = Math.max(maxCount, map.get(sum));
    }
    int countSame = 0, boxNo = -1;
    for (int key : map.keySet()) {
        if (map.get(key) == maxCount) {
            boxNo = key;
            countSame++;
        }
    }
    if (countSame == map.size()) {
        System.out.println("All boxes have same number of coins");
    } else {
        System.out.println(boxNo + " " + maxCount);
    }
}
}

```

Python

```

def digit_sum(n):
    return sum(int(d) for d in str(n))

N = int(input())
nums = list(map(int, input().split()))
low = min(nums)
high = max(nums)
from collections import defaultdict
box = defaultdict(int)
for i in range(low, high + 1):
    s = digit_sum(i)
    box[s] += 1
max_count = max(box.values())
boxes_with_max = [k for k, v in box.items() if v == max_count]

```



```
if len(boxes_with_max) == len(box):  
    print("All boxes have same number of coins")  
else:  
    print(boxes_with_max[0], max_count)
```

6.

Tom and Jerry are very naughty they always fight each other even for little things. This time they started fighting over a small cookie. The situation is such that there is a stream of numbers on a number line where there is a cookie present at the kth largest element in the array. Since Tom always gets the cookie from the master. Can you help jerry in getting the cookie as fast as possible?

Note: The kth largest element should be considered with repetition.

Input Format

The first line contains an integer that denotes the number of elements in the stream.

The second line contains K whose large value has the cookie.

The third line contains the stream of numbers

Output Format

The output contains a single integer which denotes the number at which cookie is there

Testcase Input

30

5

10 10 10 1 1 1 9 10 7 2 6 8 7 7 1 5 1 4 5 6 4 6 5 7 8 2 7 9 2

Testcase Output

9

Explanation

- After sorting the array in descending order, including duplicates:

10, 10, 10, 9, 9, 8, 8, 7, 7, 7, ...

- The 5th largest element is 9.

Solutions

C++

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n, k;
    cin >> n >> k;
    vector<int> arr(n);
    for (int i = 0; i < n; ++i) cin >> arr[i];
    sort(arr.begin(), arr.end(), greater<int>()); // descending sort
    cout << arr[k - 1] << endl; // 0-based index
    return 0;
}

```

Java

```

import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int k = sc.nextInt();
        Integer[] arr = new Integer[n];
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();
        Arrays.sort(arr, Collections.reverseOrder()); // descending
        System.out.println(arr[k - 1]);
    }
}

```

Python

```

n = int(input())
k = int(input())
arr = list(map(int, input().split()))

```

```
arr.sort(reverse=True) # Sort in descending order
```

```
print(arr[k-1]) # k-th largest (1-based index)
```

7.

You are given two sorted arrays a1 (size m) and a2 (size n).

You need to find the median of the combined sorted array, which results from merging both arrays.

Sample Input

3 4

1 3 5

2 4 6 8

Sample Output

4.0

Explanation

Combined sorted array:

[1, 2, 3, 4, 5, 6, 8]

Length = 7 → odd → median = 4 (middle value)

If the length is even, median = average of two middle elements.

Solutions

C++

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int main() {
```

```
    int m, n;
```

```
    cin >> m >> n;
```

```
    vector<int> a1(m), a2(n), merged;
```

```
    for (int i = 0; i < m; i++) cin >> a1[i];
```

```
    for (int i = 0; i < n; i++) cin >> a2[i];
```

```
    merged.insert(merged.end(), a1.begin(), a1.end());
```

```

merged.insert(merged.end(), a2.begin(), a2.end());
sort(merged.begin(), merged.end());
int len = m + n;
if (len % 2 == 1)
    cout << (double)merged[len / 2];
else
    cout << (merged[len / 2 - 1] + merged[len / 2]) / 2.0;
return 0;
}

```

Java

```

import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int m = sc.nextInt(), n = sc.nextInt();
        int[] a1 = new int[m];
        int[] a2 = new int[n];
        for (int i = 0; i < m; i++) a1[i] = sc.nextInt();
        for (int i = 0; i < n; i++) a2[i] = sc.nextInt();
        int[] merged = new int[m + n];
        for (int i = 0; i < m; i++) merged[i] = a1[i];
        for (int i = 0; i < n; i++) merged[m + i] = a2[i];
        Arrays.sort(merged);
        int len = m + n;
        if (len % 2 == 1)
            System.out.println((double)merged[len / 2]);
        else
            System.out.println((merged[len / 2 - 1] + merged[len / 2]) / 2.0);
    }
}

```

Python

```
m, n = map(int, input().split())
a1 = list(map(int, input().split()))
a2 = list(map(int, input().split()))
merged = sorted(a1 + a2)
length = m + n
if length % 2 == 1:
    print(float(merged[length // 2]))
else:
    mid = length // 2
    median = (merged[mid - 1] + merged[mid]) / 2
    print(float(median))
```

8.

From your balcony, you can see N trees standing in a line numbered from 1 to N . The i th tree has height h_i . Using your binoculars, you can only look at K trees at once in a single frame. A frame is defined by the index of the leftmost tree in it and contains exactly K trees. Starting from frame 1 (the leftmost frame), you move your binoculars towards the right until you reach frame $N-K+1$ (the rightmost possible frame). You want to find the height of the highest tree in each frame that is captured by your binoculars

Input Format

The first line of the input contains two integers N and K denoting the total number of trees and number of trees in a frame.

The second line contains N integers $a_1, a_2, \dots, a_N$ where a_i represents the height of each tree.

Testcase Input

```
8 3
1 3 1 2 5 3 6 7
```

Testcase Output

```
3 3 5 5 6 7
```

Solutions

C++

```
#include <iostream>
#include <deque>
#include <vector>
using namespace std;
int main() {
    int N, K;
    cin >> N >> K;
    vector<int> arr(N);
    for(int i = 0; i < N; i++) cin >> arr[i];
    deque<int> dq;
    for(int i = 0; i < N; i++) {
        while (!dq.empty() && dq.front() <= i - K)
            dq.pop_front();
        while (!dq.empty() && arr[dq.back()] < arr[i])
            dq.pop_back();
        dq.push_back(i);
        if (i >= K - 1)
            cout << arr[dq.front()] << " ";
    }
    return 0;
}
```

Java

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt(), K = sc.nextInt();
        int[] arr = new int[N];
        for(int i = 0; i < N; i++) arr[i] = sc.nextInt();
    }
}
```

```

Deque<Integer> dq = new LinkedList<>();
for(int i = 0; i < N; i++) {
    while(!dq.isEmpty() && dq.peekFirst() <= i - K)
        dq.pollFirst();
    while(!dq.isEmpty() && arr[dq.peekLast()] < arr[i])
        dq.pollLast();
    dq.offerLast(i);
    if(i >= K - 1)
        System.out.print(arr[dq.peekFirst()] + " ");
}
}
}

```

Python

```

from collections import deque
N, K = map(int, input().split())
arr = list(map(int, input().split()))
dq = deque()
result = []
for i in range(N):
    # Remove indices outside current window
    while dq and dq[0] <= i - K:
        dq.popleft()
    # Remove smaller elements from back
    while dq and arr[dq[-1]] < arr[i]:
        dq.pop()
    dq.append(i)
    if i >= K - 1:
        result.append(arr[dq[0]])
print(*result)

```

9.

Given a string *s*, the task is to find the longest repeating non-overlapping substring in it. In other words, find identical substrings of maximum length which do not overlap. Return -1 if no such string exists.

Note: Multiple Answers are possible but we have to return the substring whose first occurrence is earlier.

Examples:

Input: *s* = "acdcdacdc"

Output: "acdc"

Explanation: The string "acdc" is the longest Substring of *s* which is repeating but not overlapping.

Input: *s* = "knacademy"

Output: "a"

Explanation: The string "a" is the longest subString of *s* which is repeating but not overlapping.

Solutions

C++

```
#include <bits/stdc++.h>
using namespace std;
string longestSubstring(string& s) {
    int n = s.length();
    string ans = "";
    int len = 0;
    int i = 0, j = 0;
    while (i < n && j < n) {
        string curr = s.substr(i, j - i + 1);
        if (s.find(curr, j + 1) != string::npos
            && j - i + 1 > len) {
            len = j - i + 1;
            ans = curr;
        }
        // Otherwise increment i
        i++;
    }
    return ans;
}
```



```

        i++;
        j++;
    }
    return len > 0 ? ans : "-1";
}

int main() {
    string s = "knacademy";
    cout << longestSubstring(s) << endl;
    return 0;
}

```

Java

```

class Main {
    static String longestSubstring(String s) {
        int n = s.length();
        String ans = "";
        int len = 0;
        int i = 0, j = 0;
        while (i < n && j < n) {
            String curr = s.substring(i, j + 1);
            // If substring exists, compare its length
            // with ans
            if (s.indexOf(curr, j + 1) != -1
                && j - i + 1 > len) {
                len = j - i + 1;
                ans = curr;
            }
            else
                i++;
            j++;
        }
    }
}

```

```

        return len > 0 ? ans : "-1";
    }

    public static void main(String[] args) {
        String s = "knacademy";
        System.out.println(longestSubstring(s));
    }
}

```

Python

```

def longestSubstring(s):
    n = len(s)
    ans = ""
    lenAns = 0
    i, j = 0, 0
    while i < n and j < n:
        curr = s[i:j + 1]
        # If substring exists, compare its length
        # with ans
        if s.find(curr, j + 1) != -1 and j - i + 1 > lenAns:
            lenAns = j - i + 1
            ans = curr
        # Otherwise increment i
        else:
            i += 1
            j += 1
    if lenAns > 0:
        return ans
    return "-1"

if __name__ == "__main__":
    s = "knacademy"
    print(longestSubstring(s))

```

10. Round the given number to nearest multiple of 10

Given a positive integer n, round it to nearest whole number having zero as last digit.

Examples:

Input : 4722
Output : 4720

Input : 38
Output : 40

Input : 10
Output: 10

Solutions

C++

```
#include <iostream>
using namespace std;
int roundToNearest10(int n) {
    int a = (n / 10) * 10;
    int b = a + 10;
    return (n - a > b - n) ? b : a;
}
int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << "Rounded: " << roundToNearest10(n) << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;
public class Main {
```

```

public static int roundToNearest10(int n) {
    int a = (n / 10) * 10;
    int b = a + 10;
    return (n - a > b - n) ? b : a;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter a number: ");
    int n = sc.nextInt();
    System.out.println("Rounded: " + roundToNearest10(n));
}
}

```

Python

```

def round_to_nearest_10(n):
    a = (n // 10) * 10
    b = a + 10
    if n - a > b - n:
        return b
    else:
        return a

# User input
n = int(input("Enter a number: "))
print("Rounded:", round_to_nearest_10(n))

```

11. Sort elements by frequency

Given an array of integers `arr[]`, sort the array according to the frequency of elements, i.e. elements that have higher frequency comes first. If the frequencies of two elements are the same, then the smaller number comes first.

Examples:

Input: `arr[] = [5, 5, 4, 6, 4]`

Output: `[4, 4, 5, 5, 6]`

Explanation: The highest frequency here is 2. Both 5 and 4 have that frequency. Now

since the frequencies are the same the smaller element comes first. So 4 comes first then comes 5. Finally comes 6. The output is 4 4 5 5 6.

Input: arr[] = [9, 9, 9, 2, 5]

Output: [9, 9, 9, 2, 5]

Solutions

C++

```
#include <iostream>
#include <vector>
#include <map>
#include <algorithm>
using namespace std;
int main() {
    int n, x;
    cin >> n;
    vector<int> arr(n);
    map<int, int> freq;
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
        freq[arr[i]]++;
    }
    sort(arr.begin(), arr.end(), [&](int a, int b) {
        if (freq[a] == freq[b]) return a < b;
        return freq[a] > freq[b];
    });
    for (int i : arr) cout << i << " ";
    return 0;
}
```

Java

```
import java.util.*;
```

```

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<Integer> arr = new ArrayList<>();
        for (int i = 0; i < n; i++) arr.add(sc.nextInt());
        Map<Integer, Integer> freq = new HashMap<>();
        for (int i : arr) freq.put(i, freq.getOrDefault(i, 0) + 1);
        arr.sort((a, b) -> {
            if (freq.get(a).equals(freq.get(b))) return Integer.compare(a, b);
            return Integer.compare(freq.get(b), freq.get(a));
        });
        for (int i : arr) System.out.print(i + " ");
    }
}

```

Python

```

n = int(input())
arr = list(map(int, input().split()))
freq = {x: arr.count(x) for x in arr}
arr.sort(key=lambda x: (-freq[x], x))
print(*arr)

```

12. Count of substrings that start and end with 1 in given Binary String

Given a binary string, count the number of substrings that start and end with 1.

Examples:

Input: "00100101"

Output: 3

Explanation: three substrings are "1001", "100101" and "101"

Solutions

C++

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s;
    cin >> s;
    int count = 0;
    for (char c : s)
        if (c == '1') count++;
    int result = (count * (count - 1)) / 2;
    cout << result << endl;
    return 0;
}
```

Java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int count = 0;
        for (char c : s.toCharArray())
            if (c == '1') count++;
        int result = (count * (count - 1)) / 2;
        System.out.println(result);
    }
}
```

Python

```
s = input().strip()
count = s.count('1')
result = count * (count - 1) // 2
```

```
print(result)
```

13.

Given a singly linked list and an integer k, the task is to rotate the linked list to the left by k places.

Example:

Input: linked list = 10 -> 20 -> 30 -> 40 -> 50, k = 4

Output: 50 -> 10 -> 20 -> 30 -> 40

Explanation:

After rotating the linked list to the left by 4 places, the 5th node, i.e node 50 becomes the head of the linked list and next pointer of node 50 points to node 10.

Solutions

C++

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
struct Node {
```

```
    int data;
```

```
    Node* next;
```

```
    Node(int x) : data(x), next(NULL) {}
```

```
};
```

```
Node* rotateLeft(Node* head, int k) {
```

```
    if (!head || k == 0) return head;
```

```
    int len = 1;
```

```
    Node* tail = head;
```

```
    while (tail->next) {
```

```
        tail = tail->next;
```

```
        len++;
```

```
    }
```

```
    k %= len;
```



```

    if (k == 0) return head;
    Node* newTail = head;
    for (int i = 1; i < k; ++i)
        newTail = newTail->next;
    Node* newHead = newTail->next;
    newTail->next = NULL;
    tail->next = head;
    return newHead;
}

int main() {
    vector<int> vals;
    int x;
    while (cin.peek() != '\n' && cin >> x)
        vals.push_back(x);
    int k;
    cin >> k;
    Node* head = new Node(vals[0]);
    Node* curr = head;
    for (int i = 1; i < vals.size(); ++i) {
        curr->next = new Node(vals[i]);
        curr = curr->next;
    }
    head = rotateLeft(head, k);
    while (head) {
        cout << head->data << " ";
        head = head->next;
    }
}

```

Java

```
import java.util.*;
```

```

class Node {
    int data;
    Node next;
    Node(int val) {
        data = val;
        next = null;
    }
}

public class Main {
    static Node rotateLeft(Node head, int k) {
        if (head == null || k == 0) return head;
        int len = 1;
        Node tail = head;
        while (tail.next != null) {
            tail = tail.next;
            len++;
        }
        k %= len;
        if (k == 0) return head;
        Node newTail = head;
        for (int i = 1; i < k; i++)
            newTail = newTail.next;
        Node newHead = newTail.next;
        newTail.next = null;
        tail.next = head;
        return newHead;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] vals = sc.nextLine().split(" ");
    }
}

```

```

int k = sc.nextInt();
Node head = new Node(Integer.parseInt(vals[0]));
Node curr = head;
for (int i = 1; i < vals.length; i++) {
    curr.next = new Node(Integer.parseInt(vals[i]));
    curr = curr.next;
}
head = rotateLeft(head, k);
while (head != null) {
    System.out.print(head.data + " ");
    head = head.next;}}}

```

Python

class Node:

```
def __init__(self, data):
```

```
    self.data = data
```

```
    self.next = None
```

```
def rotate_left(head, k):
```

```
    if not head or k == 0:
```

```
        return head
```

```
    # Count length and find tail
```

```
    length = 1
```

```
    tail = head
```

```
    while tail.next:
```

```
        tail = tail.next
```

```
        length += 1
```

```
    k %= length
```

```
    if k == 0:
```

```
        return head
```

```
    # Find new tail (k-th node)
```

```
    new_tail = head
```

```

    for _ in range(k - 1):
        new_tail = new_tail.next
    new_head = new_tail.next
    new_tail.next = None
    tail.next = head
    return new_head

# Input
values = list(map(int, input().split()))
k = int(input())

# Build list
head = Node(values[0])
curr = head
for val in values[1:]:
    curr.next = Node(val)
    curr = curr.next

# Rotate and print
head = rotate_left(head, k)
while head:
    print(head.data, end=' ')
    head = head.next

```

14.

Given an array `arr[]` of size `n`. For every element in the array, your task is to find the index of the farthest element in the array to the right which is smaller than the current element. If no such number exists then return -1.

Examples:

Input: `arr[] = [3, 1, 5, 2, 4]`

Output: `[3, -1, 4, -1, -1]`

Explanation: `arr[3]` is the farthest smallest element to the right of `arr[0]`.

arr[4] is the farthest smallest element to the right of arr[2].

And for the rest of the elements, there is no smaller element to their right.

Solutions

C++

```
#include <iostream>

#include <vector>

using namespace std;

vector<int> farthestSmallerRight(vector<int>& arr) {
    int n = arr.size();
    vector<int> res(n, -1);
    for (int i = 0; i < n; i++) {
        for (int j = n - 1; j > i; j--) {
            if (arr[j] < arr[i]) {
                res[i] = j;
                break;
            }
        }
    }
    return res;
}

int main() {
    int n, x;
    cin >> n; // number of elements
    vector<int> arr(n);
    for (int i = 0; i < n; ++i)
        cin >> arr[i];
    vector<int> res = farthestSmallerRight(arr);
    for (int idx : res)
```

```
        cout << idx << " ";  
    return 0;  
}
```

Java

```
import java.util.*;  
  
public class Main {  
    public static int[] farthestSmallerRight(int[] arr) {  
        int n = arr.length;  
        int[] res = new int[n];  
        Arrays.fill(res, -1);  
        for (int i = 0; i < n; i++) {  
            for (int j = n - 1; j > i; j--) {  
                if (arr[j] < arr[i]) {  
                    res[i] = j;  
                    break;  
                }  
            }  
        }  
        return res;  
    }  
  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        String[] parts = sc.nextLine().split(" ");  
        int[] arr = new int[parts.length];  
        for (int i = 0; i < parts.length; i++)  
            arr[i] = Integer.parseInt(parts[i]);  
        int[] res = farthestSmallerRight(arr);  
        for (int idx : res)  
            System.out.print(idx + " ");  
    }  
}
```

```
}  
}
```

Python

```
def farthest_smaller_right(arr):  
    n = len(arr)  
    result = [-1] * n  
    for i in range(n):  
        for j in range(n - 1, i, -1):  
            if arr[j] < arr[i]:  
                result[i] = j  
                break  
    return result  
  
# Input  
arr = list(map(int, input().split()))  
output = farthest_smaller_right(arr)  
print(output)
```

15.

Given a String containing only digits, the task is to restore it by returning all possible valid IP address combinations.

A valid IP address must be in the form of A.B.C.D, where A, B, C, and D are numbers from 0-255. The numbers cannot be 0 prefixed unless they are 0.

Examples:

Input: "255678166"

Output: ["25.56.78.166", "255.6.78.166", "255.67.8.166", "255.67.81.66"]

Explanation: These are the only valid possible IP addresses.

Input: "25505011535"

Output: []

Explanation: We cannot generate a valid IP address with this string.

Solutions

C++

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;
bool isValid(string &s) {
    if (s.empty() || (s[0] == '0' && s.size() > 1) || stoi(s) > 255)
        return false;
    return true;
}
void backtrack(string s, int start, vector<string> path, vector<string> &result) {
    if (path.size() == 4) {
        if (start == s.size()) {
            string ip = path[0] + "." + path[1] + "." + path[2] + "." + path[3];
            result.push_back(ip);
        }
        return;
    }
    for (int len = 1; len <= 3 && start + len <= s.size(); len++) {
        string part = s.substr(start, len);
        if (isValid(part)) {
            path.push_back(part);
            backtrack(s, start + len, path, result);
            path.pop_back();
        }
    }
}
vector<string> restoreIpAddresses(string s) {
```



```

        vector<string> result;
        backtrack(s, 0, {}, result);
        return result;
    }

    int main() {
        string s;
        cout << "Enter digit string: ";
        cin >> s;
        vector<string> ips = restoreIpAddresses(s);
        if (!ips.empty()) {
            cout << "Valid IPs:\n";
            for (string ip : ips)
                cout << ip << endl;
        } else {
            cout << "[]\n";
        }
        return 0;
    }

```

Java

```

import java.util.*;

public class IPAddressGenerator {
    public static boolean isValid(String s) {
        if (s.length() == 0 || (s.charAt(0) == '0' && s.length() > 1))
            return false;
        int val = Integer.parseInt(s);
        return val >= 0 && val <= 255;
    }

    public static void backtrack(String s, int start, List<String> path, List<String>
result) {
        if (path.size() == 4) {
            if (start == s.length())

```

```

        result.add(String.join(".", path));
    return;
}
for (int len = 1; len <= 3 && start + len <= s.length(); len++) {
    String part = s.substring(start, start + len);
    if (isValid(part)) {
        path.add(part);
        backtrack(s, start + len, path, result);
        path.remove(path.size() - 1);
    }
}
}
}

public static List<String> restoreIpAddresses(String s) {
    List<String> result = new ArrayList<>();
    backtrack(s, 0, new ArrayList<>(), result);
    return result;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter digit string: ");
    String s = sc.nextLine();
    List<String> ips = restoreIpAddresses(s);
    if (!ips.isEmpty()) {
        System.out.println("Valid IPs:");
        for (String ip : ips)
            System.out.println(ip);
    } else {
        System.out.println("");
    }
    sc.close();
}

```

```
}  
}
```

Python

```
def is_valid(segment):  
    return segment == "0" or (segment[0] != "0" and 0 < int(segment) <= 255)  
  
def restore_ip_addresses(s):  
    res = []  
  
    def backtrack(start, path):  
        if len(path) == 4:  
            if start == len(s):  
                res.append(".".join(path))  
            return  
        for end in range(start + 1, min(start + 4, len(s) + 1)):  
            seg = s[start:end]  
            if is_valid(seg):  
                backtrack(end, path + [seg])  
  
    backtrack(0, [])  
    return res  
  
# Input  
s = input("Enter digit string: ")  
ips = restore_ip_addresses(s)  
  
# Output  
if ips:  
    print("Valid IPs:")  
    for ip in ips:  
        print(ip)  
else:  
    print("[]")
```

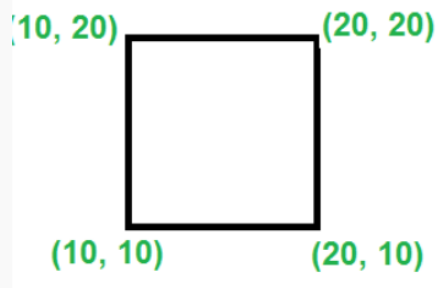
16.

Given coordinates of four points in a plane, find if the four points form a square or not.

Input: $p1 = \{ 20, 10 \}$, $p2 = \{ 10, 20 \}$, $p3 = \{ 20, 20 \}$, $p4 = \{ 10, 10 \}$

Output: Yes

Explanation:



Solutions

C++

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
// Function to calculate squared distance between two points
```

```
int distSq(pair<int, int> p1, pair<int, int> p2) {  
    return (p1.first - p2.first) * (p1.first - p2.first) +  
           (p1.second - p2.second) * (p1.second - p2.second);  
}
```

```
// Function to check if the given four points form a square
```

```
bool isSquare(pair<int, int> p1, pair<int, int> p2,  
              pair<int, int> p3, pair<int, int> p4) {  
    vector<int> d = {  
        distSq(p1, p2), distSq(p1, p3), distSq(p1, p4),  
        distSq(p2, p3), distSq(p2, p4), distSq(p3, p4)  
    };  
    sort(d.begin(), d.end());
```

```
    // Check if the 4 smallest distances (sides) are equal and the 2 largest (diagonals)
    are equal
```

```
    return d[0] > 0 && d[0] == d[1] && d[1] == d[2] && d[2] == d[3] && d[4] == d[5];
}
```

```
int main() {
```

```
    pair<int, int> p1, p2, p3, p4;
```

```
    cin >> p1.first >> p1.second;
```

```
    cin >> p2.first >> p2.second;
```

```
    cin >> p3.first >> p3.second;
```

```
    cin >> p4.first >> p4.second;
```

```
    if (isSquare(p1, p2, p3, p4))
```

```
        cout << "Yes\n";
```

```
    else
```

```
        cout << "No\n";
```

```
    return 0;
```

```
}
```

Java

```
import java.util.*;
```

```
public class CheckSquare {
```

```
    static int distSq(int[] p1, int[] p2) {
```

```
        return (p1[0] - p2[0]) * (p1[0] - p2[0]) +
               (p1[1] - p2[1]) * (p1[1] - p2[1]);
```

```
    }
```

```
    static boolean isSquare(int[] p1, int[] p2, int[] p3, int[] p4) {
```

```
        List<Integer> d = Arrays.asList(
            distSq(p1, p2), distSq(p1, p3), distSq(p1, p4),
            distSq(p2, p3), distSq(p2, p4), distSq(p3, p4)
        );
```

```
        Collections.sort(d);
```

```
        return d.get(0) > 0 &&
```

```
            d.get(0).equals(d.get(1)) &&
```

```

        d.get(1).equals(d.get(2)) &&
        d.get(2).equals(d.get(3)) &&
        d.get(4).equals(d.get(5));
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] p1 = {sc.nextInt(), sc.nextInt()};
        int[] p2 = {sc.nextInt(), sc.nextInt()};
        int[] p3 = {sc.nextInt(), sc.nextInt()};
        int[] p4 = {sc.nextInt(), sc.nextInt()};
        if (isSquare(p1, p2, p3, p4))
            System.out.println("Yes");
        else
            System.out.println("No");}}

```

Python

```

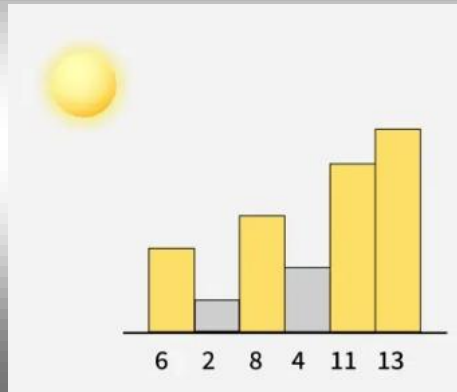
def dist_sq(p1, p2):
    return (p1[0] - p2[0])**2 + (p1[1] - p2[1])**2
def is_square(p1, p2, p3, p4):
    d = [
        dist_sq(p1, p2), dist_sq(p1, p3), dist_sq(p1, p4),
        dist_sq(p2, p3), dist_sq(p2, p4), dist_sq(p3, p4)
    ]
    d.sort()
    return d[0] > 0 and d[0] == d[1] == d[2] == d[3] and d[4] == d[5]
# Input
points = []
for _ in range(4):
    x, y = map(int, input().split())
    points.append((x, y))
# Output

```

```

if is_square(*points):
    print("Yes")
else:
    print("No")

```



17.

Given the array `arr[]` of heights of certain buildings that lie adjacent to each other, Sunlight starts falling from the left side of the buildings. If there is a building of a certain height, all the buildings to the right side of it having lesser heights cannot see the sun. The task is to find the total number of buildings that receive sunlight.

Examples:

Input: `arr[] = [6, 2, 8, 4, 11, 13]`

Output: 4

Explanation: Only buildings of height 6, 8, 11 and 13 can see the sun, hence output is 4.

Solutions

C++

```

#include <bits/stdc++.h>

using namespace std;

int longest(vector<int>& arr) {
    int n = arr.size();
    if (n == 0)
        return 0;
    int ans = 0;

```

```

    for (int i = 0; i < n; i++) {
        bool maxi = true;
        for (int j = 0; j < i; j++) {
            if (arr[j] > arr[i]) {
                maxi = false;
                break;
            }
        }
        if (maxi) {
            ans++;
        }
    }
    return ans;
}

int main() {
    vector<int> arr = {6, 2, 8, 4, 11, 13};
    cout << longest(arr) << endl;
    return 0;
}

```

Java

```

class KNA {
    static int longest(int[] arr) {
        int n = arr.length;
        if (n == 0) return 0;
        int ans = 0;
        for (int i = 0; i < n; i++) {
            boolean maxi = true;
            for (int j = 0; j < i; j++) {
                if (arr[j] > arr[i]) {
                    maxi = false;
                }
            }
            if (maxi) {
                ans++;
            }
        }
        return ans;
    }
}

```



```

        break;
    }
}
if (maxi) {
    ans++;
}
}
return ans;
}
public static void main(String[] args) {
    int[] arr = {6, 2, 8, 4, 11, 13};
    System.out.println(longest(arr));
}
}

```

Python

```

def longest(arr):
    n = len(arr)
    # base case
    if n == 0:
        return 0
    ans = 0
    for i in range(n):
        maxi = True
        for j in range(i):
            if arr[j] > arr[i]:
                maxi = False
                break
        if maxi:
            ans += 1
    return ans

```

```
if __name__ == "__main__":  
    arr = [6, 2, 8, 4, 11, 13]  
    print(longest(arr))
```

18.

Given, an array of size n. Find an element that divides the array into two sub-arrays with equal sums.

Examples:

Input: 1 4 2 5 0

Output: 2

Explanation: If 2 is the partition, subarrays are : [1, 4] and [5]

Solutions

C++

```
#include <iostream>  
#include <vector>  
using namespace std;  
int findPartitionElement(vector<int>& arr) {  
    int total = 0, left_sum = 0;  
    for (int num : arr) total += num;  
    for (int i = 0; i < arr.size(); i++) {  
        int right_sum = total - left_sum - arr[i];  
        if (left_sum == right_sum)  
            return arr[i];  
        left_sum += arr[i];  
    }  
    return -1;  
}  
int main() {  
    int n, x;  
    vector<int> arr;
```

```

cout << "Enter elements: ";
while (cin >> x)
    arr.push_back(x);
cout << findPartitionElement(arr) << endl;
return 0;
}

```

Java

```

import java.util.*;

public class Main {

    public static int findPartitionElement(int[] arr) {
        int total = 0, leftSum = 0;
        for (int val : arr) total += val;
        for (int i = 0; i < arr.length; i++) {
            int rightSum = total - leftSum - arr[i];
            if (leftSum == rightSum)
                return arr[i];
            leftSum += arr[i];
        }
        return -1;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<Integer> list = new ArrayList<>();
        while (sc.hasNextInt()) list.add(sc.nextInt());
        int[] arr = list.stream().mapToInt(i -> i).toArray();
        System.out.println(findPartitionElement(arr));}
}

```

Python

```

def find_partition_element(arr):
    total = sum(arr)
    left_sum = 0

```

```

for i in range(len(arr)):
    right_sum = total - left_sum - arr[i]
    if left_sum == right_sum:
        return arr[i]
    left_sum += arr[i]
return -1

arr = list(map(int, input("Enter the array: ").split()))
print(find_partition_element(arr))

```

19.

Given two strings s1 and s2 consisting of lowercase characters, the task is to check whether the two given strings are anagrams of each other or not. An anagram of a string is another string that contains the same characters, only the order of characters can be different.

Examples:

Input: s1 = "knacademy" s2 = "ymeadnack"

Output: true

Explanation: Both the string have same characters with same frequency. So, they are anagrams.

Solutions

C++

```

#include <algorithm>
#include <iostream>
using namespace std;

bool areAnagrams(string &s1, string &s2) {
    sort(s1.begin(), s1.end());
    sort(s2.begin(), s2.end());
    return s1 == s2;
}

int main() {
    string s1 = " knacademy ";

```

```

string s2 = " ymeadnack ";
    cout << (areAnagrams(s1, s2) ? "true" : "false") << endl;
return 0;
}

```

Java

```

import java.util.Arrays;
class KNAcademy {
    static boolean areAnagrams(String s1, String s2) {
        char[] s1Array = s1.toCharArray();
        char[] s2Array = s2.toCharArray();
        Arrays.sort(s1Array);
        Arrays.sort(s2Array);
        return Arrays.equals(s1Array, s2Array);
    }
    public static void main(String[] args) {
        String s1 = " knacademy ";
        String s2 = " ymeadnack ";
        System.out.println(areAnagrams(s1, s2));
    }
}

```

Python

```

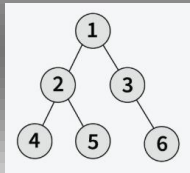
def areAnagrams(s1, s2):
    s1 = sorted(s1)
    s2 = sorted(s2)
    return s1 == s2
if __name__ == "__main__":
    s1 = "knacademy"
    s2 = "ymeadnack"
    print("true" if areAnagrams(s1, s2) else "false")

```

20.

Given a binary tree, the task is to connect the nodes that are at the same level. Given an additional nextRight pointer for the same. Initially, all the next right pointers point to garbage values, set these pointers to the point next right for each node.

Input:



Output:



Solutions

C++

```
#include <iostream>
#include <queue>
using namespace std;
struct Node {
    int data;
    Node *left, *right, *nextRight;
    Node(int val) : data(val), left(nullptr), right(nullptr), nextRight(nullptr) {}
};
void connect(Node *root) {
    if (!root) return;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        int size = q.size();
        Node* prev = nullptr;
```

```

    for (int i = 0; i < size; i++) {
        Node* curr = q.front(); q.pop();
        if (prev) prev->nextRight = curr;
        prev = curr;
        if (curr->left) q.push(curr->left);
        if (curr->right) q.push(curr->right);
    }
    prev->nextRight = nullptr;
}
}

void printLevels(Node* root) {
    while (root) {
        Node* temp = root;
        while (temp) {
            cout << temp->data << " ";
            temp = temp->nextRight;
        }
        cout << endl;
        if (root->left) root = root->left;
        else if (root->right) root = root->right;
        else break;
    }
}

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    root->right->right = new Node(6);

```

```
    connect(root);  
    printLevels(root);  
}
```

Java

```
import java.util.*;  
  
class Node {  
    int data;  
    Node left, right, nextRight;  
    Node(int val) {  
        data = val;  
        left = right = nextRight = null;  
    }  
}  
  
public class Main {  
    static void connect(Node root) {  
        if (root == null) return;  
        Queue<Node> q = new LinkedList<>();  
        q.add(root);  
        while (!q.isEmpty()) {  
            int size = q.size();  
            Node prev = null;  
            for (int i = 0; i < size; i++) {  
                Node curr = q.poll();  
                if (prev != null) prev.nextRight = curr;  
                prev = curr;  
                if (curr.left != null) q.add(curr.left);  
                if (curr.right != null) q.add(curr.right);  
            }  
            prev.nextRight = null;  
        }  
    }  
}
```



```

    }
    static void printLevels(Node root) {
        while (root != null) {
            Node temp = root;
            while (temp != null) {
                System.out.print(temp.data + " ");
                temp = temp.nextRight;
            }
            System.out.println();
            if (root.left != null) root = root.left;
            else if (root.right != null) root = root.right;
            else break;
        }
    }
}

public static void main(String[] args) {
    Node root = new Node(1);
    root.left = new Node(2);
    root.right = new Node(3);
    root.left.left = new Node(4);
    root.left.right = new Node(5);
    root.right.right = new Node(6);
    connect(root);
    printLevels(root);
}
}

```

Python

```

from collections import deque

class Node:
    def __init__(self, data):
        self.data = data

```

```

        self.left = self.right = self.nextRight = None
def connect(root):
    if not root:
        return
    q = deque([root])
    while q:
        size = len(q)
        prev = None
        for _ in range(size):
            curr = q.popleft()
            if prev:
                prev.nextRight = curr
            prev = curr
            if curr.left:
                q.append(curr.left)
            if curr.right:
                q.append(curr.right)
        prev.nextRight = None
def printLevels(root):
    while root:
        temp = root
        while temp:
            print(temp.data, end=" ")
            temp = temp.nextRight
        print()
        if root.left:
            root = root.left
        elif root.right:
            root = root.right
        else:

```

break

Sample usage

root = Node(1)

root.left = Node(2)

root.right = Node(3)

root.left.left = Node(4)

root.left.right = Node(5)

root.right.right = Node(6)

connect(root)

printLevels(root)